



BASI DI DATI MULTIMEDIALI

Alessandro Zappatore

Università degli studi di Torino
Anno accademico 2025/2026, 2° semestre

Indice

1	Introduzione	6
1.1	Che cos'è un medium?	6
1.2	Multimedialità	6
1.3	Ipermedialità	6
1.4	I Big Data e i Dati Multimediali	7
1.4.1	Le caratteristiche HMLE dei Big Data Multimediali	7
1.4.2	Ulteriori sfide dei dati multimediali (SIS)	7
1.5	Obiettivi del corso	8
1.6	Sense (Dare un senso ai dati)	8
1.7	Paradigmi di Interazione con i Dati	8
1.7.1	Operazioni di Exploratory Search	9
1.8	Caratteristiche Comuni dei Dati Multimediali	9
1.9	Livelli di Organizzazione dei Dati	10
1.9.1	Basi di Dati Strutturate	10
1.9.2	Basi di Dati Semi-strutturate	10
1.9.3	Basi di Dati Non Strutturate	10
1.10	Features (Caratteristiche estratte)	11
1.11	Gestione delle Imperfezioni	11
1.12	Tipologie di Query	12
1.12.1	Query sui Metadati	12
1.12.2	Query per Esempio (Query-By-Example)	12
1.12.3	Query Semantiche e Basate su Oggetti	12
1.13	QBE: Rappresentazione Visiva e Funzioni di Aggregazione	13
1.13.1	Proprietà fondamentale: La Monotonia	14
1.13.2	Le Funzioni di Aggregazione	14
1.14	Progressività della Ricerca	15
1.15	Come Rappresentare i Dati (Modelli Formali per i Surrogati)	15
1.15.1	1. Insiemi (Sets)	15
1.15.2	2. Vettori (Vectors)	15
1.15.3	3. Stringhe (Strings / Sequences)	15
1.15.4	4. Alberi (Trees)	16
1.15.5	5. Grafi (Graphs)	16
1.16	Tensori	16
1.16.1	Esempi nel contesto Multimediale	17
1.16.2	La sfida computazionale	17

2	Modello Vettoriale (Vector Space Model)	18
2.1	Dimensionalità e Rappresentazione dei Conteggi	18
2.2	Rappresentazione Vettoriale	19
2.2.1	Query come Vettore	19
2.2.2	Misura di Distanza o Similarità	20
2.2.3	Top-k Retrieval	20
2.2.4	Classificazione	20
2.3	Spazio vettoriale	21
2.4	Indipendenza lineare e Base	21
2.5	Prodotto interno e Ortogonalità	22
2.6	Norma e Basi Ortonormali	22
2.6.1	Base Ortonormale	22
2.6.2	Famiglia delle Norme L_p	23
2.7	Modelli Metrici e Distanze	23
2.7.1	Distanza di Minkowski (L_p Metric)	23
2.8	Vantaggi del Modello Metrico	24
2.9	Tipologie di Ricerca nello Spazio Metrico	24
2.10	Altre misure di distanza e similarità	24
2.10.1	Distanza di Mahalanobis	25
2.10.2	Approfondimento: Intuizione sulla Covarianza	25
2.10.3	Esempio Numerico Passo-Passo	26
2.10.4	Distanza Quadratica (Quadratic Form Distance)	27
2.10.5	Intersezione di Istogrammi (Intersection Similarity)	27
2.10.6	Misure basate sugli Angoli (Cosine Similarity)	28
2.10.7	Correlazione di Pearson	29
2.10.8	Divergenza di Kullback-Leibler (KL Divergence)	29
2.10.9	Earth Mover's Distance (EMD)	30
3	Quante features vogliamo?	31
3.1	Dimensionality Curse (Maledizione della Dimensionalità)	31
3.2	Effetto geometrico: Ipersfere e Gusci	31
3.2.1	Power Law	31
3.3	Criteri per la Selezione delle Feature	32
3.3.1	Caratteristiche di una "Buona Feature"	32
3.4	Entropia come misura di qualità	32
3.5	Feature Latenti e Trasformazione dello Spazio	32
3.5.1	Cosa sono le Feature Latenti	33
3.6	Rappresentazione dei testi e Feature Extraction	34
3.6.1	Rappresentazione vettoriale (Vector Space Model)	34
3.6.2	Distribuzione Zipfiana	35
3.6.3	Come definire i pesi vettoriali	35
3.7	Estrazione di Feature dalle Immagini	36
3.7.1	Rappresentazione del Colore	36
3.7.2	Rappresentazioni Lossy e Compressione	38
3.7.3	Istogramma dei colori	40
3.7.4	Istogrammi di colore locali (Spatial Grids)	41
3.7.5	Come scegliere il vocabolario dei colori	41
3.7.6	Confronti tra istogrammi dei colori	41
3.7.7	Texture	42
3.8	Segmentazione e Riconoscimento delle Forme	44
3.8.1	Segmentazione basata su Cluster (K-Means)	45
3.8.2	Region-Growing	45
3.8.3	Watershed (Spartiacque)	45
3.9	Rappresentazione e Descrittori di Forma	46
3.9.1	Rappresentazione Bitmap-based (Rigida)	46
3.9.2	Chain-Code (Codice a Catena di Freeman)	46

3.9.3	Approssimazione Poligonale (Rappresentazione vettoriale)	47
3.9.4	Time-Series (Firma Centroidale / Centroid Distance Signature)	48
3.10	Rappresentazione delle forme	49
3.10.1	Local Binary Patterns (LBP)	49
3.10.2	Histogram of Oriented Gradients (HOG)	49
3.11	Features locali	50
3.11.1	SIFT (Scale-Invariant Feature Transform)	50
4	Riduzione della Dimensionalità	54
4.1	Criteri di Trasformazione e Scelta	54
4.2	Trasformazioni Orto-normali e Proiezioni	54
4.2.1	Impatto sulle query (False Hit e Miss)	56
4.3	Singular Value Decomposition (SVD)	56
4.3.1	SVD Troncata e Approssimazione del Database	57
4.4	Principal Component Analysis (PCA)	57
4.4.1	Criteri per la scelta del numero di autovettori (k)	58
4.5	Multi-Dimensional Scaling (MDS)	58
4.5.1	Il Processo di Ottimizzazione	59
5	Tensori	60
5.1	Nomenclatura e Struttura	60
5.2	CP Decomposition (CANDECOMP / PARAFAC)	61
5.3	Tucker Decomposition	62
5.3.1	L'approccio Alternating Least Squares (ALS)	63
5.3.2	Tucker - HOSVD (Higher-Order SVD)	64
5.3.3	Tucker - HOOI (Higher-Order Orthogonal Iteration / ALS for Tucker)	64
6	Modelli Probabilistici	66
6.1	Aspect Model (Modello ad Aspetti)	66
6.2	Probabilistic LSA (PLSA)	66
6.2.1	L'algoritmo Expectation-Maximization (EM)	67
6.3	Latent Dirichlet Allocation (LDA)	68
6.3.1	Il Processo Generativo di LDA	68
6.4	Confronto tra SVD, PLSA e LDA	69
7	Indicizzazione Spaziale e Riduzione della Dimensionalità	71
7.1	Curve che riempiono lo spazio (Space-Filling Curves)	71
7.1.1	Ordine per Righe e per Colonne (Row-Major / Column-Major)	71
7.1.2	Ordine a Serpente (Righe Prime / Boustrophedon)	71
7.1.3	Ordine Diagonale (Cantor)	72
7.1.4	Curva Z-order (Codifica di Morton)	72
7.1.5	Curva di Peano-Hilbert	73
7.2	Come indicizzare sfruttando la multidimensionalità	73
7.2.1	File a Griglia (Grid File)	74
7.3	Alberi di punti (Point Trees)	75
7.3.1	Point Quadtree (Quadtree di punti)	75
7.4	K-D Trees (K-Dimensional Trees)	79
7.4.1	Estensione a dimensioni $k > 2$	79
7.5	MX Quadrees (Matrix Quadrees)	80
7.6	R-Trees (Region/Rectangle Trees)	80
7.6.1	Criteri di Splitting in fase di Inserimento	81
7.6.2	R+ Trees	82
7.6.3	Ricerca Nearest Neighbor (NN) e le Metriche di Pruning	82
7.7	X-Trees (eXtended Node Trees)	83
7.8	Pyramid Trees	84
7.9	TV Trees (Telescopic Vector Trees)	84

8	Hashing per Nearest Neighbor Search	86
8.1	Locality Sensitive Hashing (LSH)	86
8.1.1	La Griglia Randomizzata	86
8.1.2	Costruzione delle funzioni LSH per specifiche metriche	88
9	Clustering	89
9.1	Potatura dello spazio di ricerca	89
9.2	Metodi sound (non iterativi)	90
9.2.1	Componenti connesse (Single-Linkage)	90
9.2.2	Clique (Complete-Linkage)	91
9.3	Misure di qualità dei cluster	91
9.3.1	Diametro	91
9.3.2	Omogeneità e Compattezza	91
9.3.3	Dimensione bilanciata	92
9.3.4	Separazione	92
9.3.5	Integrità intra-cluster	93
9.3.6	Modularità	93
9.3.7	Clustering spettrale	93
9.4	Metodi iterativi	94
9.4.1	Single-pass (leader-based)	94
9.4.2	Assenza di soglia	94
9.4.3	Max-a-min	94
9.4.4	Assenza di distanza	95
9.4.5	Clustering adattivo	96
10	Classificazione	97
10.1	Support Vector Machines (SVM)	97
10.1.1	Classificazione N-aria (Multiclasse)	98
10.2	K-Nearest Neighbors (k-NN)	98
10.2.1	Meccanismo di classificazione	99
10.2.2	La scelta del parametro k	99
10.2.3	Variante: Weighted k-NN (Voto pesato)	99
10.3	Decision tree (Alberi di decisione)	99
10.3.1	Selezione delle feature	100
10.3.2	Overfitting e Potatura (Pruning)	101
10.4	Classificazione Ensemble	101
10.5	Classificazione Bayesiana	102
10.5.1	Naive Bayes (Ipotesi di Indipendenza)	102
10.5.2	Gestione delle dipendenze e Informazione Mutua	102
10.5.3	Grafo di dipendenza (Approssimazione di Chow-Liu)	103
11	Classificazione con reti neurali	105
11.1	Decomposizione LR	105
11.1.1	Architettura Encoder/Decoder	105
11.2	Deep Auto-Encoder	106
11.3	Estrazione di Feature nelle Immagini	106
11.3.1	Operazione di Convoluzione e Livelli Stacked	107
11.3.2	Operazione di Pooling (Sottocampionamento)	108
11.4	Classificazione Finale	108
11.5	Architetture Notevoli e Framework	108
12	Relevance Feedback	109
12.1	Soggettività vs Oggettività	109
12.2	Query e Manipolazione dello Spazio di Ricerca	109
12.3	Spostamento della Query (Algoritmo di Rocchio)	112
12.4	Ricalibrazione dei Pesi delle Feature	114

12.4.1 Come stimare le probabilità marginali	114
12.5 Ranking Bayesiano dei Documenti (Dimostrazione dei passaggi)	115

1 Introduzione

1.1 Che cos'è un medium?

Il termine **medium** (plurale media) indica il mezzo o lo strumento attraverso il quale vengono comunicati e trasmessi dei contenuti. La sua natura può variare notevolmente: può trattarsi di testo, immagini, suoni, video, ecc.

È importante notare che, anche all'interno di una singola tipologia di media, possono coesistere diversi **codici comunicativi**. Ad esempio, un'immagine statica non è solo "una foto", ma un insieme complesso di linee, forme, colori e spazi che il nostro cervello decodifica per estrarne un significato.

1.2 Multimedialità

Si parla di **multimedialità** quando avviene la combinazione simultanea di più media per veicolare un messaggio complesso.

- *Esempio*: Un servizio del telegiornale è un prodotto multimediale perché unisce video (immagini in movimento), audio (parlato e suoni ambientali) e spesso testo (sottopancia o titoli).

Dato, Informazione e Conoscenza Quando interagiamo con un media, avviene un processo di elaborazione cognitiva che trasforma il semplice dato in conoscenza.

- **Dato**: L'elemento grezzo percepito (es. vedo un casco con disegnata una tigre).
- **Informazione**: L'elaborazione del dato (es. riconosco che è un casco da sci).
- **Conoscenza**: L'informazione associata al bagaglio culturale pregresso.
 - *Esempio*: Vedo il casco con la tigre (dato), capisco che è attrezzatura da sci (informazione), ma grazie alla mia conoscenza pregressa so che quello è un casco e non una tigre o un pallone data la sua forma tondeggiante.

Soggettività La fruizione di un media è intrinsecamente legata alla **soggettività**. La stessa informazione, veicolata dallo stesso identico media, può essere decodificata in modo diverso da individui diversi.

- *Esempio*: L'immagine di un paesaggio innevato può evocare sensazioni di "bellezza e pace" in un osservatore, e sensazioni di "freddo e desolazione" in un altro.

1.3 Ipermedialità

L'**ipermedialità** riguarda la struttura della navigazione e l'interattività. Un sistema ipermediale prevede che l'informazione non sia fruita in modo lineare (dall'inizio alla fine), ma attraverso **cammini prestabiliti** o scelti dall'utente, tipicamente tramite l'uso di link (collegamenti).

Differenza tra Multimediale e Ipermediale È fondamentale distinguere i due concetti, che possono presentarsi da soli o combinati:

1. **Multimediale ma NON Ipermediale**: Un media che usa diversi codici (audio + video) ma ha una fruizione lineare.
 - *Esempio*: Un film o un video su YouTube (se guardato passivamente). L'utente preme play e il contenuto scorre dall'inizio alla fine senza bivi decisionali.
2. **Ipermediale (o Iperestuale) ma NON Multimediale**: Un contenuto che permette la navigazione non lineare, ma usa un solo tipo di media (solitamente il testo).
 - *Esempio*: Una pagina di Wikipedia composta di solo testo con link blu che rimandano ad altre pagine di solo testo. Questo è tecnicamente un Iperestesto.

3. **Ipermediale E Multimediale:** La combinazione completa. Abbiamo diversi tipi di media (testo, immagini, video) e una struttura di navigazione a link.

- *Esempio:* Un sito web moderno o un videogioco. Posso leggere, guardare video, cliccare su un'immagine per ingrandirla o seguire un link per cambiare argomento.

1.4 I Big Data e i Dati Multimediali

I **Big Data** sono insiemi di dati così grandi e complessi da richiedere tecnologie e metodi analitici avanzati per essere elaborati. Tradizionalmente, vengono definiti attraverso il modello delle "3 V":

- **Volume:** La quantità enorme di dati generati ogni secondo.
- **Velocità:** La rapidità con cui i dati vengono non solo generati, ma anche elaborati e analizzati (spesso in tempo reale).
- **Varietà:** La diversità dei formati e delle tipologie di dati (strutturati come i database tradizionali, o non strutturati come foto, video e post sui social).

L'analisi dei Big Data è fondamentale per individuare tendenze, fare previsioni e supportare le decisioni in settori cruciali come il marketing, la sanità e la finanza. **Queste tre caratteristiche fondanti si applicano perfettamente anche ai dati multimediali.**

1.4.1 Le caratteristiche HMLE dei Big Data Multimediali

Quando i Big Data incontrano il mondo multimediale, assumono tratti specifici che possono essere riassunti nell'acronimo **HMLE**:

- **H - High-dimensional** (Alta dimensionalità): Per descrivere accuratamente un dato multimediale è necessario un numero elevatissimo di variabili. Se per analizzare un testo abbiamo bisogno di uno spazio dimensionale pari all'intero vocabolario di una lingua, per un'immagine in alta risoluzione o un video dobbiamo elaborare milioni di pixel, frequenze audio e frammenti cromatici.
- **M - Multi-modal** (Multimodalità): I dati multimediali combinano diverse "modalità" (o nature) di informazione simultaneamente. Ad esempio, un video di TikTok è multimodale perché contiene immagini in movimento (video), parlato o musica (audio) e spesso sottotitoli o hashtag (testo).
- **L - Inter-Linked** (Interconnessione): I dati non sono isole, ma formano reti complesse. Queste connessioni possono essere:
 - *Esplicite:* Un'amicizia su un social network o un link diretto tra un video e l'articolo di blog che lo incorpora.
 - *Implicite:* Due utenti che non si conoscono ma che hanno guardato per intero lo stesso identico video, creando un legame comportamentale utile per gli algoritmi di raccomandazione.
- **E - Evolving** (In evoluzione): I dati multimediali sono dinamici e cambiano nel tempo. Un testo può essere editato, un video può ricevere nuovi commenti o essere remixato da altri utenti, generando un flusso informativo in continuo mutamento.

1.4.2 Ulteriori sfide dei dati multimediali (SIS)

Oltre al modello HMLE, gestire archivi multimediali enormi comporta l'affrontare tre ulteriori problematiche:

- **S - Sparsity** (Sparsità): All'interno di matrici di dati immense, spesso mancano moltissime informazioni (es. in un catalogo di milioni di film, un singolo utente ne avrà visti solo alcune decine). Questo richiede algoritmi avanzati capaci di gestire gli spazi vuoti e applicare tecniche di imputazione per stimare, nel modo più accurato possibile, il valore mancante o il comportamento probabile dell'utente.

- **I - Imprecision** (Imprecisione): I dati reali contengono spesso "rumore" o difetti intrinseci. Una foto può essere sfocata, un audio disturbato dal vento, o i metadati di un sensore GPS/termico possono avere un certo margine di errore hardware.
- **S - Subjectivity** (Soggettività): Come già visto nell'introduzione, i media sono soggetti all'interpretazione umana. L'analisi automatizzata fatica a catalogare dati soggettivi: un commento testuale può essere letterale o ironico, e un'immagine può trasmettere gioia o tristezza a seconda del contesto culturale dell'osservatore.

1.5 Obiettivi del corso

Il nostro obiettivo è quello di smembrare questi media in modo da poterli memorizzare e successivamente interrogare, per estrarre informazioni utili e significative.

Per esempio se volessimo riconoscere un pomodoro in una foto, dovremmo trovare un modo per distinguere una qualsiasi cosa di colore rosso da un pomodoro. Per fare questo, potremmo cercare di identificare caratteristiche specifiche del pomodoro (forma, dimensione, texture) e confrontarle con quelle presenti nell'immagine.

L'utente finale delle nostre analisi potrebbe essere un essere umano, quindi ci dobbiamo preoccupare della parte di visualizzazione dei risultati.

1.6 Sense (Dare un senso ai dati)

Oggi acquisiamo costantemente dati multimediali e multimodali dall'ambiente circostante. Il nostro compito non è solo memorizzarli ed elaborarli, ma soprattutto assegnare loro un significato nel modo più automatico possibile, riducendo al minimo l'intervento manuale dell'utente. Dobbiamo, in breve, "dare un senso" a questi dati.

Il termine **Sense** racchiude una doppia valenza:

1. **Percezione fisica**: L'atto di acquisire il dato dal mondo reale attraverso dei sensori (es. una fotocamera che cattura la luce, un microfono che registra onde sonore, un sensore tattile).
2. **Comprensione semantica**: L'atto cognitivo o computazionale di attribuire un significato logico all'informazione percepita.

Il Semantic Gap (Divario Semantico) Il nostro obiettivo principale è colmare il **gap semantico**. Si tratta della distanza logica e concettuale che separa il dato grezzo (es. una matrice di pixel colorati) e la sua interpretazione concettuale (es. "questa è l'immagine di un cane che corre in un prato"). Colmare questo divario permette ai sistemi informatici di utilizzare i dati per compiere azioni autonome o aiutare l'essere umano a prendere decisioni informate.

1.7 Paradigmi di Interazione con i Dati

A seconda di quanto l'utente abbia chiari i propri obiettivi e di quali strumenti abbia a disposizione, possiamo distinguere quattro modalità principali di interazione con i dati:

1. Data Processing (Elaborazione dei dati) L'utente sa esattamente *cosa* vuole ottenere e possiede le *funzioni specifiche* per farlo. È un'operazione oggettiva e priva di soggettività.

- *Esempio*: Dato un documento di testo, l'utente applica una funzione per contare quante volte compare la parola "multimediale". Non c'è ambiguità: il sistema esegue un calcolo statistico deterministico.

2. Querying (Interrogazione) L'utente sa *cosa* sta cercando e sa come descriverlo, ma **non ha** la funzione esatta per estrarlo. Delega quindi al sistema il compito di trovare il percorso e il metodo migliore per soddisfare la richiesta.

- *Esempio:* L'utente carica una foto di una scarpa in un motore di ricerca visiva e chiede: "Trova tutte le immagini simili a questa". Il sistema si fa carico di capire quali caratteristiche analizzare per restituire i risultati pertinenti.

3. Navigation (Navigazione) Il livello di astrazione aumenta: l'utente ha un'idea generale di ciò che vuole, ma non sa come descriverlo in modo preciso al sistema, oppure non ha gli strumenti di input adeguati per formulare una query esatta.

- *Esempio:* Un utente vuole comprare dei pantaloni su un e-commerce. Non sa il modello esatto, quindi naviga attraverso il menu a tendina "Abbigliamento → Uomo → Pantaloni", osservando i risultati finché non trova qualcosa che rispecchia la sua necessità implicita.

4. Exploration (Esplorazione) L'utente **non ha un obiettivo preciso**. Si muove all'interno del sistema spinto dalla curiosità o dalla scoperta casuale (serendipity).

- *Esempio:* Aprire TikTok o YouTube e scorrere la home page. L'utente non sta cercando un video specifico, ma esplora i contenuti suggeriti dall'algoritmo fermandosi su quelli che catturano la sua attenzione.

1.7.1 Operazioni di Exploratory Search

Anche l'esplorazione apparentemente casuale segue delle logiche e richiede dei sistemi in grado di supportarla attraverso specifiche operazioni:

- **Analisi:** La ricerca di pattern ricorrenti (tendenze) e di anomalie o eccezioni (outlier) all'interno di grandi moli di dati.
- **Comparazione:** Per navigare tra i dati è necessario che il sistema sappia definire il concetto di "simile" e "diverso", confrontando oggetti complessi attraverso metriche di distanza matematica.
- **Aggregazione:** La capacità di raggruppare (clustering) oggetti simili in categorie logiche per facilitarne la consultazione.
- **Trasformazione:** La modifica della rappresentazione del dato per convenienza computazionale, per estrarre informazioni più salienti o per migliorare le performance degli algoritmi.
 - *Esempio:* Passare dalla codifica colore RGB (Rosso, Verde, Blu) a quella YUV (Luminanza e Crominanza) tramite una trasformazione lineare. Questo non serve necessariamente a comprimere il dato, ma a separare l'informazione sulla luminosità da quella del colore, risultando molto più utile per algoritmi di computer vision o per la compressione video.
- **Visualizzazione:** La traduzione dell'analisi algoritmica (matrici, vettori, grafi) in interfacce grafiche intuitive per l'occhio umano.

1.8 Caratteristiche Comuni dei Dati Multimediali

I dati complessi che cerchiamo di analizzare presentano spesso queste caratteristiche ricorrenti:

- **Multi-dimensionali:** Per descrivere il dato sono necessarie numerose variabili (feature). Ad esempio, un'immagine in scala di grigi richiede una dimensione per ogni singolo pixel per indicarne l'intensità luminosa.
- **Multi-modali:** Contengono nature diverse di informazioni (es. un video contiene contemporaneamente audio, sequenze di immagini e testo sotto forma di sottotitoli).
- **Multi-layer (Multilivello):** Lo stesso ambiente fisico o set di dati può essere descritto attraverso strati sovrapposti.

- *Esempio*: Una mappa cittadina su Google Maps. Il "livello 1" mostra la rete stradale per le automobili, il "livello 2" mostra le piste ciclabili, il "livello 3" la rete dei mezzi pubblici. Sono dati diversi che descrivono lo stesso spazio.
- **Multi-resolution (Multirisoluzione)**: I dati possono essere analizzati a granularità differenti, spaziali o temporali.
 - *Esempio*: I dati del traffico urbano possono essere aggregati e visualizzati ogni 15 minuti, ogni ora o come media settimanale, cambiando completamente la prospettiva dell'analisi.
- **Imprecisi / Ambigui / Soggettivi**: Contengono rumore di fondo (es. un audio fruscante) o dipendono dall'interpretazione umana (es. una recensione sarcastica di un film).

Serie temporali (Time Series): Dati raccolti sequenzialmente nel tempo. Sono *multivariate* se misurano più parametri simultaneamente (es. un macchinario ospedaliero che registra contemporaneamente frequenza cardiaca, pressione sanguigna e saturazione dell'ossigeno in ogni istante t).

1.9 Livelli di Organizzazione dei Dati

1.9.1 Basi di Dati Strutturate

Si basano su una logica rigida, definita a priori tramite uno schema rigido (es. tabelle SQL). Ogni informazione è inserita in campi precisi.

- **Vantaggi**: Dati prevedibili, interrogazioni (query) estremamente veloci e precise, facilità di ottimizzazione.
- **Svantaggi**: Totale incapacità di rappresentare la complessità, la soggettività e la natura non omogenea dei media multimediali.

1.9.2 Basi di Dati Semi-strutturate

Offrono maggiore flessibilità. Lo schema non è rigido ma si "auto-describe" insieme ai dati (es. file JSON, XML o database NoSQL a documenti). Le query si basano sull'attraversamento di cammini o alberi logici.

- **Vantaggi**: Buona organizzazione unita a flessibilità; permette di aggiungere nuovi campi senza rompere la struttura precedente.
- **Svantaggi**: Ancora insufficienti per analizzare il contenuto semantico interno di immagini, video o audio grezzi.

1.9.3 Basi di Dati Non Strutturate

Non esiste nessuno schema predefinito. Il dato è memorizzato nella sua forma grezza. Ne sono un esempio file di testo libero, immagini, audio, video e post sui social network.

Il Concetto di Surrogato Nessun sistema di analisi multimediale lavora mai direttamente confrontando bit per bit il dato originale (sarebbe troppo costoso e inutile). I sistemi lavorano invece su un **Surrogato**. Il surrogato è una rappresentazione formale e compatta che contiene *tutte e sole* le proprietà essenziali del dato originale utili per l'analisi.

- *Esempio*: Se cerco un pomodoro in un database di immagini, il sistema non confronta la foto intera. Estrae dalla foto di input un surrogato (es. un vettore matematico che riassume la forte presenza di rosso e una forma sferica) e lo confronta con i surrogati di tutte le immagini salvate nel database.

1.10 Features (Caratteristiche estratte)

Le **features** sono proprio quegli elementi misurabili che compongono il surrogato. La loro natura cambia radicalmente in base al tipo di media analizzato e all'obiettivo dell'analisi:

- **Nelle Immagini:** Le features classiche includono **colore** (istogrammi), **texture** (ruvidità o pattern visivo) e **forma** (shape). Il rilevamento dei *bordi* (edge detection) è una feature fondamentale per segmentare l'immagine e distinguere dove finisce un pomodoro e inizia lo sfondo, permettendo al sistema di isolare la forma dell'oggetto.
- **Nel Testo:** Frequenza delle parole chiave, analisi sintattica, lunghezza delle frasi, autori, metadati bibliografici.
- **Nei Video (per Sistemi di Raccomandazione):** Invece di analizzare i pixel, se lo scopo è raccomandare un film a un utente (come su Netflix), le features utili saranno i metadati: genere, regista, attori principali, anno di uscita e le valutazioni storiche (rating) lasciate dagli altri utenti.

1.11 Gestione delle Imperfezioni

I sistemi multimediali operano nel mondo reale, il quale è intrinsecamente caotico. Qualsiasi processo che va dall'acquisizione del dato fino alla sua interrogazione è soggetto a diverse forme di imperfezione, errore o approssimazione.

1. Imprecisione nei dati catturati Il dato grezzo può arrivare al sistema già compromesso o rumoroso a causa di limiti fisici o hardware dei sensori.

- *Esempio:* Una fotocamera con la lente sporca, un sensore ottico in condizioni di scarsa luminosità che genera "grana" (rumore digitale), o un microfono esposto al vento. Il sistema deve essere in grado di tollerare e filtrare questo rumore iniziale.

2. Imprecisione nell'estrazione delle feature (Quantizzazione) Quando trasformiamo il dato grezzo in un surrogato (estraendo le feature), siamo costretti a semplificare, introducendo un margine di errore o approssimazione.

- *Esempio (Vocabolario limitato):* Se analizziamo un'immagine a milioni di colori ma decidiamo di estrarre una feature basata su una tavolozza (vocabolario) di soli 16 colori base per risparmiare spazio, il sistema "arrotonderà" i pixel al colore più simile disponibile. Molte sfumature andranno perse.
- *Percezione Macchina vs Umano:* Una macchina può distinguere due codici colore RGB matematicamente diversi che l'occhio umano percepisce come identici. Usare una precisione eccessiva a livello algoritmico potrebbe rivelarsi controproducente, introducendo differenze ("rumore") dove per l'utente umano non esistono.

3. Similarità e Correlazione ingannevole nelle features Le feature scelte potrebbero non essere sufficientemente discriminanti per il dominio applicativo, portando il sistema a confondere oggetti concettualmente diversi ma visivamente simili.

- *Esempio (Pomodoro vs Palla):* Se utilizziamo solo la "forma" (cerchio) e la "dimensione" come feature, il sistema considererà identici un pomodoro e una pallina rossa, fallendo nel recuperarli separatamente.
- *Tolleranza alla deformazione:* Al contrario, a volte desideriamo che il sistema *ignori* alcune imperfezioni. In un sistema di ricerca basato su schizzi a mano libera (Sketch-based Retrieval), l'utente disegna un cerchio imperfetto. Il sistema non deve cercare altre "patate tremolanti", ma deve estrarre la feature astratta "forma circolare" e restituire immagini di cerchi perfetti o oggetti tondi.

4. Imperfezione nella formulazione delle query L'utente spesso fatica a tradurre il proprio intento di ricerca in una query precisa (una manifestazione del *semantic gap*).

- *Esempio*: Nella **Query-By-Example (QBE)**, l'utente fornisce un'immagine di un tramonto sul mare molto arancione per cercare "altri paesaggi marini al tramonto". Il sistema, analizzando i colori, potrebbe restituire immagini di zucche, arance o fiamme (che condividono l'istogramma cromatico), ignorando la composizione (sole + acqua) che l'utente aveva in mente ma non ha saputo specificare.

Il Relevance Feedback (Feedback di Rilevanza): Per mitigare questa incapacità di formulare la query perfetta al primo tentativo, si introduce un ciclo interattivo. Il sistema mostra dei risultati preliminari e l'utente indica quali sono pertinenti ("Mi piace questo", "Non c'entra nulla quest'altro"). Il sistema ricalcola dinamicamente i pesi delle feature (ad esempio, capendo che deve dare meno importanza al colore arancione e più importanza alle linee orizzontali del mare) e affina la ricerca successiva.

1.12 Tipologie di Query

I sistemi informativi multimediali supportano diversi paradigmi per interrogare le basi di dati.

1.12.1 Query sui Metadati

Sono le interrogazioni più tradizionali, basate su testo e logica booleana (Vero/Falso). Lavorano sui dati strutturati (autori, date, tag) associati al media, non sul suo contenuto intrinseco.

- *Esempio*: "Trovami tutti i dipinti di Van Gogh completati nel 1889".
- *Esempio*: "Trovami tutti i film di genere 'Azione' con Brad Pitt".
- *Vantaggio*: Risultati esatti e certi. *Svantaggio*: Richiedono che qualcuno abbia precedentemente e manualmente etichettato a mano tutti i media.

1.12.2 Query per Esempio (Query-By-Example)

L'utente non usa parole, ma fornisce al sistema un oggetto multimediale (un'immagine, uno spezzone audio) chiedendo di trovare elementi correlati. Si dividono in due approcci:

- **Ricerca Esatta (Exact Match)**: Cerca esattamente lo stesso file (es. tramite un controllo crittografico o *hashing*). Utile per trovare copie pirata di un video o immagini duplicate in un archivio.
- **Ricerca per Similarità (Partial/Similarity Match)**: Cerca i record "più vicini" all'esempio fornito nello spazio delle feature. Per funzionare, richiede di definire una metrica di distanza e una soglia di accettabilità (ϵ).
 - *Range Query*: "Restituisci tutte le immagini la cui distanza dall'esempio è inferiore a X ".
 - *k-Nearest Neighbors (k-NN)*: "Restituisci le prime K immagini più simili, indipendentemente da quanto lo siano in termini assoluti".

1.12.3 Query Semantiche e Basate su Oggetti

Sono le query più avanzate, che cercano di comprendere il contenuto profondo della scena scendendo nel dettaglio dei singoli oggetti che la compongono.

- **Similarità Visuale (Low-level)**: Ricerca basata strettamente sull'aspetto grafico grezzo. "Trova immagini con la stessa distribuzione di colori, gli stessi bordi predominanti o la stessa texture di sfondo".
- **Similarità Semantica (High-level)**: Ricerca basata sul significato (concetti). Richiede l'uso di reti neurali o classificatori. "Trova un'immagine che contenga il concetto di 'cane' e il concetto di 'parco'", a prescindere dal fatto che il cane sia bianco, nero, grande o piccolo (le feature visive cambiano, il significato semantico resta).

- **Similarità Spaziale (Topologica):** Non si limita a verificare l'esistenza di oggetti, ma analizza le loro **relazioni spaziali** all'interno della scena (sopra, sotto, a destra, dentro, adiacente).
 - *Esempio:* Una query semantica standard troverebbe un'immagine con una "persona" e un "cane". Una query con vincoli spaziali permette di cercare specificamente "un'immagine in cui una persona si trova *sopra* o *tiene in mano* un cane" (escludendo le foto in cui la persona è sullo sfondo e il cane in primo piano staccato).

1.13 QBE: Rappresentazione Visiva e Funzioni di Aggregazione

Nel paradigma **Query-By-Example (QBE)**, il processo di ricerca non avviene tramite parole chiave, ma attraverso il confronto visivo.

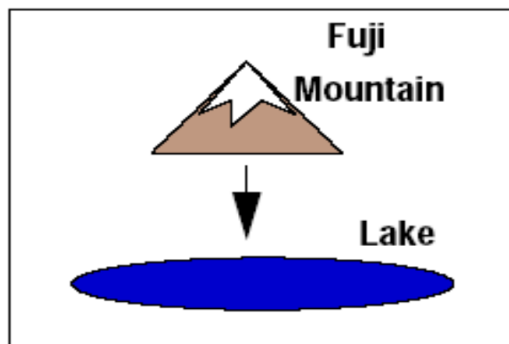


Figura 1: *Input:* L'utente fornisce un'immagine di esempio (es. il Monte Fuji) come query.

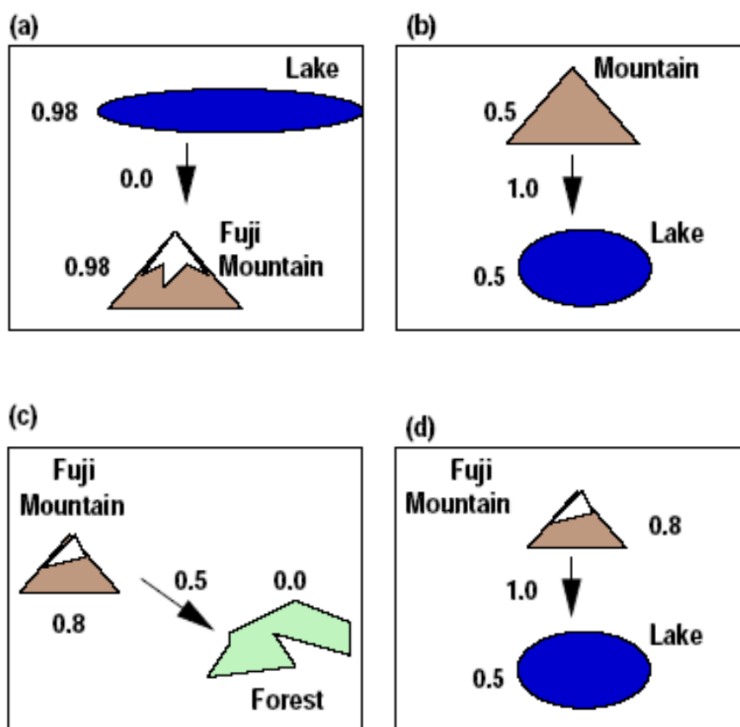


Figura 2: *Output:* Il sistema valuta la similarità visuale e restituisce immagini con caratteristiche (colore, forma) affini.

Quando confrontiamo un'immagine di query con quelle nel database, spesso analizziamo diverse **feature** separatamente (es. una per il colore, una per la texture, una per la forma). Ognuna di queste produce uno "score parziale" di similarità. Dobbiamo poi combinare questi punteggi in un unico **score globale** utilizzando una **funzione di aggregazione** *g*.

1.13.1 Proprietà fondamentale: La Monotonia

La funzione di aggregazione scelta deve essere **MONOTONA**. Una funzione $g(x_1, \dots, x_m)$ è monotona se, aumentando il valore di uno qualsiasi degli input x_i (migliorando quindi una similarità parziale), il valore del risultato finale non diminuisce mai.

- **Perché è importante?** La monotonia è fondamentale per l'efficienza degli algoritmi di ricerca (come l'algoritmo di Fagin o il Threshold Algorithm). Ci permette di "potare" (pruning) lo spazio di ricerca: se sappiamo che il punteggio parziale massimo possibile per i restanti oggetti non può superare il punteggio del miglior oggetto trovato finora, possiamo fermarci senza scansionare tutto il database.

1.13.2 Le Funzioni di Aggregazione

Esistono diversi modi per combinare gli score parziali, che rispecchiano diverse logiche decisionali:

1. Minimo (Logica AND - Congiunzione) Questa funzione restituisce il valore più basso tra tutti gli score parziali.

$$Score_{globale} = \min(s_{colore}, s_{forma}, s_{texture})$$

- **Comportamento:** Il risultato peggiore "domina" (abbassa) l'intero punteggio.
- **Significato logico:** Corrisponde a una **Congiunzione (AND)**. Richiedo che l'immagine sia simile per colore **E** per forma **E** per texture. Se anche solo una caratteristica è diversa (score basso), l'oggetto viene scartato o penalizzato pesantemente.
- **Esempio:** Cerco una "pallina da tennis gialla". Se trovo un oggetto giallo (score colore alto) ma quadrato (score forma basso), il Minimo prenderà lo score basso della forma. L'oggetto non va bene.

2. Massimo (Logica OR - Disgiunzione) Questa funzione restituisce il valore più alto tra tutti gli score parziali.

$$Score_{globale} = \max(s_{colore}, s_{forma}, s_{texture})$$

- **Comportamento:** Il risultato migliore "domina" (alza) l'intero punteggio.
- **Significato logico:** Corrisponde a una **Disgiunzione (OR)**. Mi basta che l'immagine sia simile per colore **OPPURE** per forma. È una ricerca più lasca.
- **Esempio:** Cerco "qualcosa che ricordi il mare". Potrebbe essere un'immagine blu (colore simile) oppure un'onda disegnata in bianco e nero (forma simile). Basta che una delle due feature sia presente affinché l'oggetto sia considerato rilevante.

3. Media (Compromesso) Calcola la somma dei valori diviso il loro numero, spesso usando una media ponderata per dare più importanza a certe feature rispetto ad altre.

$$Score_{globale} = \frac{1}{n} \sum_{i=1}^n s_i \quad \text{oppure} \quad \sum_{i=1}^n w_i \cdot s_i$$

- **Comportamento:** Tutti i punteggi contribuiscono al risultato finale. Un punteggio molto basso può essere compensato da uno molto alto.
- **Significato:** È la funzione più comune perché offre un bilanciamento tra le varie caratteristiche senza essere drastica come il Minimo o il Massimo.

1.14 Progressività della Ricerca

La **progressività** è una caratteristica fondamentale dei sistemi avanzati di recupero dell'informazione (Information Retrieval) e riguarda l'efficienza computazionale durante l'esplorazione dei risultati. Spesso un utente non vuole vedere tutti i milioni di risultati in una volta sola, ma procede a scaglioni (es. "Mostrami i primi 5 risultati più simili", poi "Mostrami i successivi 5"). Vogliamo che il sistema sia in grado di restituire risultati progressivamente più "lontani" (meno simili all'input) **senza sprecare il lavoro computazionale già svolto**.

- *Il problema:* Se ricalcolassimo da zero le distanze tra la query e l'intero database ogni volta che l'utente chiede la "pagina successiva", il sistema collasserebbe.
- *La soluzione:* Quando il sistema individua il j -esimo elemento più simile, deve mantenere uno stato (ad esempio tramite una coda di priorità o un indice indicizzato). In questo modo, per ottenere l'elemento $(j + 1)$ -esimo, il costo computazionale è minimo, riutilizzando i calcoli precedenti.

1.15 Come Rappresentare i Dati (Modelli Formali per i Surrogati)

Una volta deciso quali feature estrarre da un media, dobbiamo scegliere una struttura matematica e logica per memorizzarle. Questa scelta determinerà quali tipi di query potremo eseguire e con quale efficienza. Di seguito i modelli formali più utilizzati:

1.15.1 1. Insiemi (Sets)

Un insieme è una collezione non ordinata e priva di ripetizioni di elementi. In questo modello, un dato è rappresentato semplicemente dalla presenza o assenza di determinate caratteristiche (logica booleana: 1 o 0).

- **Caratteristiche:** Posso affermare se una proprietà esiste, ma non posso quantificarla.
- *Esempio:* In un'immagine, posso dire che $S = \{\text{rosso}, \text{blu}, \text{albero}\}$, indicando che ci sono questi elementi. Non posso però dire *quanto* rosso c'è o *quanti* alberi ci sono.
- **Uso:** È sufficiente per rispondere a query di tipo esistenziale ("Trova tutte le foto che contengono un gatto"). Tuttavia, è raramente usato da solo per i dati multimediali complessi, poiché perde troppe sfumature essenziali.

1.15.2 2. Vettori (Vectors)

È in assoluto una delle rappresentazioni più utilizzate (il cosiddetto **Vector Space Model**). Un vettore è un array matematico monodimensionale $\mathbf{v} = [v_1, v_2, \dots, v_n]$. La dimensione del vettore (n) è fissa e corrisponde al numero totale di feature che il sistema è in grado di memorizzare. Il vettore supera i limiti degli insiemi perché permette di memorizzare valori continui o discreti per ogni singola feature, catturando due aspetti fondamentali:

- **Conteggio (Frequenza):** Valori interi che indicano la molteplicità. *Esempio:* Quanti pixel rossi ci sono in un'immagine, o quante volte compare la parola "computer" in un testo.
- **Peso (Importanza):** Valori continui (spesso normalizzati tra 0 e 1) che indicano la rilevanza di una feature. *Esempio:* Un valore di 0.8 all'indice corrispondente al "rosso" indica che l'immagine ha una forte dominanza di quel colore, mentre 0.1 indica una presenza trascurabile.

1.15.3 3. Stringhe (Strings / Sequences)

Se per descrivere il mio dato multimediale l'**ordine** degli elementi è fondamentale, i vettori e gli insiemi falliscono. Una stringa è una sequenza ordinata di simboli estratti da un alfabeto.

- *Esempio testuale:* Il vettore mi direbbe che le lettere 'A', 'M', 'O', 'R', 'E' sono presenti una volta ciascuna. Ma solo la stringa mi permette di distinguere la parola "AMORE" dalla parola "ROMA" o "ORME".

- *Esempio multimediale*: L'analisi di una sequenza di note musicali (una melodia) o la sequenza temporale dei fotogrammi in un video (dove l'ordine degli eventi cambia completamente la trama).

1.15.4 4. Alberi (Trees)

Quando le feature hanno una natura intrinsecamente **gerarchica** o tassonomica, si utilizza una struttura ad albero.

- *Esempio spaziale*: La scomposizione di un'immagine. La radice dell'albero è l'immagine intera; i figli sono i 4 quadranti in cui viene divisa; i nipoti sono ulteriori sotto-quadranti (come nel modello Quad-tree).
- *Esempio semantico*: Un documento strutturato in Capitoli, Paragrafi e Frasi (es. file XML o JSON).

1.15.5 5. Grafi (Graphs)

Se l'obiettivo principale è rappresentare le **relazioni** e le interconnessioni tra oggetti diversi, il grafo è il modello matematico perfetto. È composto da Nodi (le entità) e Archi (le relazioni tra di esse).

- *Esempio Sociale*: In un social network, ogni utente è un nodo, e un "Mi piace" o un'amicizia è un arco direzionale che li collega.
- *Esempio Visivo (Scene Graph)*: In un'immagine complessa, un nodo è "Persona", un altro nodo è "Cavallo". L'arco che li unisce ha l'etichetta "Cavalca". Questo permette di distinguere un uomo a cavallo da un uomo in piedi accanto a un cavallo.

Esiste una notazione migliore in assoluto? La risposta è **no**. Non esiste un modello universalmente superiore (principio noto in informatica come *No Free Lunch theorem*). Ogni struttura dati è progettata per ottimizzare la risoluzione di un problema specifico:

- Usa i **Vettori** se devi calcolare similarità matematiche veloci (es. riconoscimento volti).
- Usa le **Stringhe** se stai analizzando sequenze temporali (es. riconoscimento vocale).
- Usa i **Grafi** se devi fare inferenza semantica o studiare reti sociali (es. algoritmi di raccomandazione).

Spesso, nei sistemi moderni di Big Data multimediali, questi modelli vengono utilizzati congiuntamente in architetture ibride.

1.16 Tensori

I **tensori** rappresentano la generalizzazione matematica e logica delle strutture dati che abbiamo visto finora. Se pensiamo ai dati come contenitori di numeri organizzati nello spazio, esiste una gerarchia precisa:

- **Scalare (0D)**: Un singolo numero (es. 7).
- **Vettore (1D)**: Una singola riga o colonna di numeri (es. [1, 5, 7]).
- **Matrice (2D)**: Una tabella di numeri con righe e colonne (es. un'immagine in scala di grigi 1024×768).
- **Tensore (n D)**: Una struttura che generalizza il concetto a n dimensioni (o "assi").

Sebbene nell'immaginario comune ci si fermi spesso alla terza dimensione (un cubo di numeri), i tensori possono avere un numero arbitrario di dimensioni ($n > 3$).

1.16.1 Esempi nel contesto Multimediale

Nel mondo dei Big Data e del Deep Learning, i tensori sono la struttura standard per rappresentare l'informazione complessa:

- **Tensore 3D (Immagine a colori):** Un'immagine digitale RGB non è piatta; ha tre livelli di profondità. Viene rappresentata come $Altezza \times Larghezza \times 3$ (Canali Colore).
- **Tensore 4D (Video o Batch):**
 - Un video è una sequenza di immagini nel tempo: $Tempo \times Altezza \times Larghezza \times 3$.
 - Nel training delle reti neurali, si processano gruppi di immagini (batch) contemporaneamente: $NumeroImmagini \times Altezza \times Larghezza \times 3$.

1.16.2 La sfida computazionale

La caratteristica critica dei tensori è il costo di elaborazione. **Le operazioni sui tensori crescono esponenzialmente all'aumentare delle dimensioni.** Se immaginiamo un tensore con D dimensioni, e ogni dimensione ha una grandezza N , il numero totale di elementi da elaborare è N^D .

- *Esempio:* Passare da una matrice 10×10 ($10^2 = 100$ elementi) a un tensore $10 \times 10 \times 10$ ($10^3 = 1000$ elementi) aumenta i dati di un fattore 10 aggiungendo una sola dimensione.

Questo porta spesso alla cosiddetta "**Maledizione della dimensionalità**" (Curse of Dimensionality): all'aumentare delle dimensioni, i dati diventano radi (sparsi) e gli algoritmi di ricerca o di calcolo della distanza (come la Distanza Euclidea) diventano estremamente lenti o meno efficaci nel discriminare la similarità.

2 Modello Vettoriale (Vector Space Model)

Il modello vettoriale è il metodo fondamentale per trasformare dati astratti (immagini, testi, audio) in entità matematiche elaborabili.

2.1 Dimensionalità e Rappresentazione dei Conteggi

Il concetto chiave è che ogni feature misurabile corrisponde a una dimensione geometrica. Il valore della feature (es. un conteggio o un peso) diventa la coordinata su quell'asse.

- **1 Dimensione (Retta):** Se consideriamo una sola feature (es. "quanto è rossa l'immagine"), ogni oggetto viene mappato su una retta.

$$v = [5]$$



Figura 3: Rappresentazione di oggetti su una retta in 1D

- **2 Dimensioni (Piano Cartesiano):** Se consideriamo due feature (es. "quanto è rossa" e "quanto è luminosa"), ogni oggetto è un punto (x, y) su un piano 2D.

$$v = [5, 7]$$

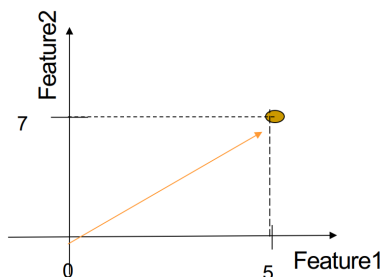


Figura 4: Rappresentazione di oggetti su un piano in 2D

- **3 Dimensioni (Spazio Euclideo):** Con tre feature, l'oggetto diventa un punto (x, y, z) nello spazio 3D.

$$v = [5, 7, 3]$$

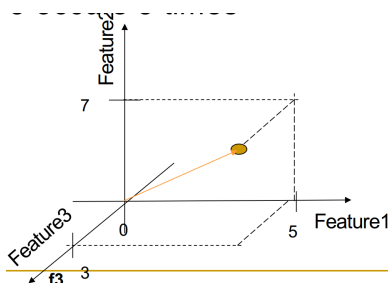


Figura 5: Rappresentazione di oggetti su uno spazio 3D

- **N Dimensioni (Iperspazio):** Generalizzando, se estraiamo N feature, ogni dato multimediale diventa un punto in un iperspazio N -dimensionale. Anche se non possiamo visualizzarlo geometricamente, le regole matematiche rimangono le stesse.

2.2 Rappresentazione Vettoriale

In questo modello, ogni oggetto multimediale O viene rappresentato come un vettore di feature $\mathbf{v}$:

$$\mathbf{v}_O = (f_1, f_2, \dots, f_N)$$

Dove f_i è il valore numerico della i -esima feature.

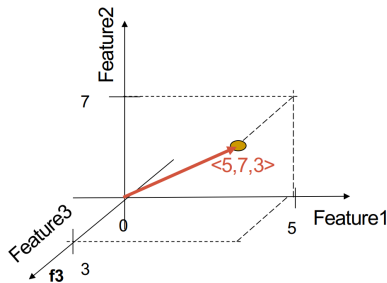


Figura 6: Rappresentazione vettoriale di un oggetto

L'Analogia Spaziale Possiamo immaginare l'intero database come una nuvola di punti in uno spazio multidimensionale (lo *Spazio delle Feature*).

- Immagini visivamente simili avranno vettori con valori simili.
- Di conseguenza, i punti corrispondenti a immagini simili si troveranno vicini nello spazio geometrico.

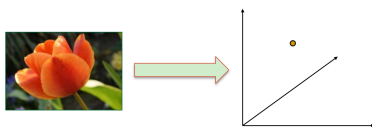


Figura 7: Rappresentazione di un immagine su uno spazio 2D

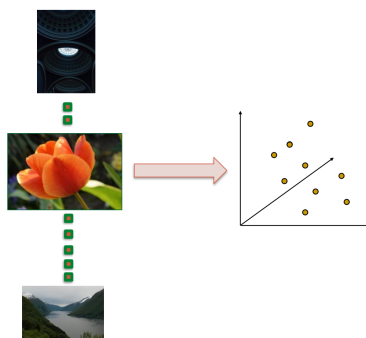


Figura 8: Rappresentazione di più immagini su uno spazio 2D

2.2.1 Query come Vettore

Nel modello vettoriale, la query non è un comando esterno, ma viene tradotta essa stessa in un vettore $\mathbf{q}$ nello stesso spazio dei dati.

- *Esempio*: Se cerco un'immagine simile a una foto di input (Query-By-Example), il sistema estrae le feature dalla foto di input e crea il vettore query $\mathbf{v}_q$.
- L'obiettivo diventa quindi trovare i vettori $\mathbf{v}_d$ (documenti/dati) che sono geometricamente più vicini a $\mathbf{v}_q$.

2.2.2 Misura di Distanza o Similarità

Per quantificare il concetto di "vicinanza" tra la query q e un oggetto d , usiamo delle metriche matematiche. Esiste una relazione inversa tra distanza e similarità:

$$\text{Distanza} \approx \frac{1}{\text{Similarità}}$$

Le metriche più comuni sono:

- **Distanza Euclidea (L_2):** Misura la distanza in linea d'aria tra due punti. È la classica distanza geometrica. Se la distanza è 0, gli oggetti sono identici.

$$d(\mathbf{q}, \mathbf{d}) = \sqrt{\sum_{i=1}^N (q_i - d_i)^2}$$

- **Similarità del Coseno:** Misura l'angolo tra i due vettori, ignorando la loro lunghezza (magnitudine). È molto usata quando conta la proporzione delle feature più che la loro quantità assoluta.

$$\cos(\theta) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|}$$

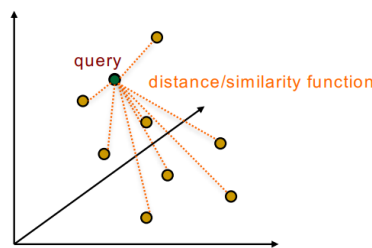


Figura 9: Rappresentazione della distanza tra vettori in uno spazio 2D

2.2.3 Top-k Retrieval

Una volta calcolata la distanza tra il vettore query q e tutti gli altri vettori nel database, il sistema ordina i risultati. Una query Top-k richiede di restituire i k oggetti più vicini alla query. Geometricamente, questo equivale a tracciare una sfera centrata nel punto q e allargarla finché non ingloba esattamente k punti.

2.2.4 Classificazione

Il modello vettoriale è la base per la classificazione automatica. Se osserviamo lo spazio delle feature, spesso i dati appartenenti alla stessa categoria (es. "foto di mare" vs "foto di montagna") tendono a raggrupparsi in cluster distinti.

- Un classificatore è un algoritmo che cerca di trovare una superficie di separazione (una linea in 2D, un piano in 3D, un iperpiano in N D) che divida lo spazio in regioni.
- Se un nuovo punto (una nuova immagine) cade nella regione "A", il sistema la classificherà automaticamente come appartenente alla classe "A".

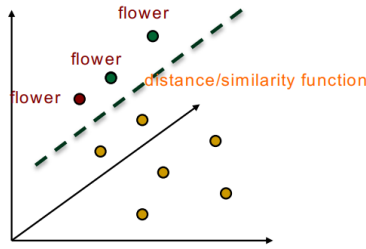


Figura 10: Rappresentazione di oggetti su uno spazio 2D in cui è stata individuata la classe dei fiori

2.3 Spazio vettoriale

Definizione Un insieme $\mathbb{S}$ è definito **spazio vettoriale** su un campo di scalari (come $\mathbb{R}$) se e solo se, per ogni $\vec{v}_i, \vec{v}_j, \vec{v}_k \in \mathbb{S}$ e per ogni scalare $c, d \in \mathbb{R}$, sono soddisfatti i seguenti assiomi:

- **Chiusura e proprietà della somma:**
 - *Commutatività:* $\vec{v}_i + \vec{v}_j = \vec{v}_j + \vec{v}_i$
 - *Associatività:* $\vec{v}_i + (\vec{v}_j + \vec{v}_k) = (\vec{v}_i + \vec{v}_j) + \vec{v}_k$
 - *Elemento neutro:* Esiste un vettore $\vec{0}$ tale che $\vec{v}_i + \vec{0} = \vec{v}_i$
 - *Inverso additivo:* Esiste un vettore $-\vec{v}_i$ tale che $\vec{v}_i + (-\vec{v}_i) = \vec{0}$
- **Chiusura e proprietà del prodotto per scalare:**
 - *Distributività (somma scalari):* $(c + d) \cdot \vec{v}_i = c \cdot \vec{v}_i + d \cdot \vec{v}_i$
 - *Distributività (somma vettori):* $c \cdot (\vec{v}_i + \vec{v}_j) = c \cdot \vec{v}_i + c \cdot \vec{v}_j$
 - *Compatibilità prodotto:* $(cd) \cdot \vec{v}_i = c \cdot (d\vec{v}_i)$
 - *Identità moltiplicativa:* $1 \cdot \vec{v}_i = \vec{v}_i$

Gli elementi di $\mathbb{S}$ sono chiamati **vettori**.

2.4 Indipendenza lineare e Base

Definizione di Indipendenza Sia $V = \{\vec{v}_1, \vec{v}_2, \dots, \vec{v}_n\}$ un insieme di vettori in uno spazio $\mathbb{S}$. Si dice che i vettori sono **linearmente indipendenti** se l'unica combinazione lineare che produce il vettore nullo è quella in cui tutti i coefficienti sono nulli:

$$\sum_{i=1}^n c_i \cdot \vec{v}_i = \vec{0} \quad \Rightarrow \quad c_1 = c_2 = \dots = c_n = 0$$

Se questa condizione non è verificata, significa che almeno un vettore è "ridondante" (può essere espresso come somma degli altri).

Definizione di Base Un insieme V è detto **base** di $\mathbb{S}$ se soddisfa due condizioni:

1. **Indipendenza Lineare:** I vettori non sono ridondanti.
2. **Completezza (Generazione):** Ogni vettore $\vec{u} \in \mathbb{S}$ può essere generato come combinazione lineare dei vettori della base:

$$\vec{u} = \sum_{i=1}^n c_i \cdot \vec{v}_i$$

Il numero di vettori che compongono la base definisce la **dimensione** dello spazio vettoriale.

Esempio Pratico Se descriviamo un oggetto usando 3 feature, stiamo operando in uno spazio a 3 dimensioni. Avere una quarta feature che è una semplice combinazione matematica delle prime tre (es. $f_4 = f_1 + f_2$) non aggiunge informazione, ma solo ridondanza. Una base efficiente conterrebbe solo le prime 3 feature.

2.5 Prodotto interno e Ortogonalità

Definizione Un **prodotto interno** (o scalare) su $\mathbb{S}$ è una funzione $\mathbb{S} \times \mathbb{S} \rightarrow \mathbb{R}$ che associa a due vettori un numero reale, soddisfacendo:

- **Commutatività:** $\vec{u} \cdot \vec{v} = \vec{v} \cdot \vec{u}$
- **Distributività:** $(c_1 \vec{u} + c_2 \vec{v}) \cdot \vec{w} = c_1(\vec{u} \cdot \vec{w}) + c_2(\vec{v} \cdot \vec{w})$
- **Positività:** $\vec{u} \cdot \vec{u} > 0$ se $\vec{u} \neq \vec{0}$

Ortogonalità Due vettori sono **ortogonali** se il loro prodotto interno è nullo:

$$\vec{u} \cdot \vec{v} = 0$$

In termini geometrici, significa che formano un angolo di 90° . Un insieme di vettori mutuamente ortogonali è sempre linearmente indipendente.

Interpretazione Geometrica Il prodotto interno è legato alla proiezione di un vettore sull'altro:

$$\vec{u} \cdot \vec{v} = \|\vec{u}\| \|\vec{v}\| \cos(\theta)$$

Dove θ è l'angolo tra i due vettori. Se i vettori sono normalizzati (lunghezza 1), il prodotto scalare corrisponde esattamente al coseno dell'angolo (Cosine Similarity).

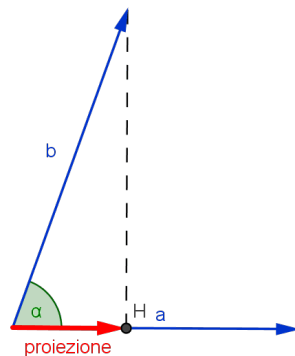


Figura 11: Il prodotto interno rappresenta la proiezione di un vettore sull'altro

2.6 Norma e Basi Ortonormali

Definizione La **norma** $\|\vec{v}\|$ è una funzione che assegna una lunghezza (magnitudine) strettamente positiva a un vettore.

2.6.1 Base Ortonormale

Una base $V = \{\vec{v}_1, \dots, \vec{v}_n\}$ è **ortonormale** se valgono due condizioni:

1. I vettori sono a due a due ortogonali (prodotto scalare = 0).
2. I vettori sono normalizzati (prodotto scalare con se stessi = 1).

Formalmente, usando il delta di Kronecker $\delta_{i,j}$:

$$\vec{v}_i \cdot \vec{v}_j = \delta_{i,j} = \begin{cases} 1 & \text{se } i = j \\ 0 & \text{se } i \neq j \end{cases}$$

2.6.2 Famiglia delle Norme L_p

La definizione generalizzata di lunghezza è data dalla norma L_p . Dato un vettore $\vec{x} = (x_1, \dots, x_n)$:

$$\|\vec{x}\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}$$

Casi notevoli:

- $p = 1$ (**Norma Manhattan**): Somma dei valori assoluti.
- $p = 2$ (**Norma Euclidea**): La classica lunghezza geometrica "in linea d'aria".
- $p = \infty$ (**Norma di Chebyshev**): Determina la lunghezza basandosi solo sulla componente più grande del vettore:

$$\|\vec{x}\|_\infty = \max_i |x_i|$$

2.7 Modelli Metrici e Distanze

Per confrontare due oggetti (surrogati) $\vec{x}$ e $\vec{y}$, definiamo una funzione di **distanza** $d(\vec{x}, \vec{y})$.

Una **metrica** è una regola matematica che assegna un numero alla distanza tra due oggetti in modo coerente. Quel numero deve essere interpretabile come una vera nozione di "quanto sono diversi" due punti nello spazio delle feature. In pratica, la metrica è ciò che trasforma la similarità in un problema geometrico: più il valore è piccolo, più gli oggetti sono simili; più è grande, più sono diversi.

Questa proprietà è fondamentale perché algoritmi come *range search*, *k-NN*, clustering e classificazione si basano proprio sull'ordinamento dei punti in base alle distanze. Se la funzione di distanza non è una metrica, il concetto di "vicinanza" può diventare ambiguo e i risultati del retrieval possono essere instabili o poco affidabili.

Una funzione è una metrica valida se soddisfa gli assiomi:

1. **Non negatività**: $d(\vec{x}, \vec{y}) \geq 0$ e $d(\vec{x}, \vec{x}) = 0$.
2. **Simmetria**: $d(\vec{x}, \vec{y}) = d(\vec{y}, \vec{x})$.
3. **Disuguaglianza triangolare**: $d(\vec{x}, \vec{y}) \leq d(\vec{x}, \vec{z}) + d(\vec{z}, \vec{y})$.

2.7.1 Distanza di Minkowski (L_p Metric)

È la generalizzazione della distanza tra due punti A e B in uno spazio N -dimensionale:

$$d_p(A, B) = \left(\sum_{i=1}^N |a_i - b_i|^p \right)^{1/p}$$

1. Distanza Manhattan (L_1 Metric) Si ottiene ponendo $p = 1$. Corrisponde alla distanza percorsa muovendosi solo lungo gli assi (come un taxi in una città a griglia).

$$d_1(A, B) = \sum_{i=1}^N |a_i - b_i|$$

Utilizzo: Spesso preferita nella Computer Vision per confrontare istogrammi di colore, in quanto cattura meglio la similarità percepita dall'occhio umano ed è robusta agli outlier.

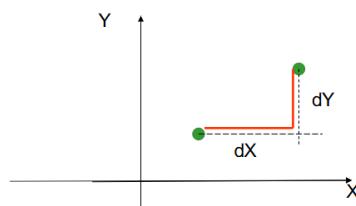


Figura 12: Distanza L1 (Manhattan): somma dei cateti

2. Distanza Euclidea (L_2 Metric) Si ottiene ponendo $p = 2$. È la distanza più comune.

$$d_2(A, B) = \sqrt{\sum_{i=1}^N (a_i - b_i)^2}$$

Utilizzo: Standard per molti algoritmi di clustering, ma soffre in spazi ad altissima dimensionalità.

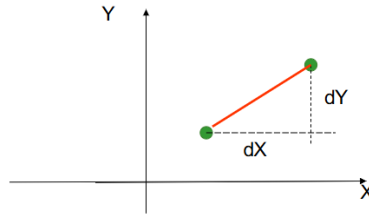


Figura 13: Distanza L2 (Euclidea): ipotenusa

3. Distanza di Chebyshev (L_∞ Metric) Si ottiene con $p \rightarrow \infty$. La distanza è determinata unicamente dalla singola dimensione in cui i due oggetti differiscono maggiormente.

$$d_\infty(A, B) = \max_i |a_i - b_i|$$

2.8 Vantaggi del Modello Metrico

Il modello metrico offre diversi vantaggi pratici nella valutazione della similarità tra immagini e oggetti multimediali:

- È particolarmente adatto per alcune forme di valutazione della similarità, ad esempio i confronti basati sul colore.
- È coerente con approcci ampiamente utilizzati nelle comunità di *computer vision* e *pattern recognition*.
- I risultati sperimentali suggeriscono che la metrica L_1 può catturare meglio la nozione umana di similarità tra immagini.
- Rende relativamente semplice l'indicizzazione dei dati, modellati come vettori di proprietà, mediante tecniche classiche di indicizzazione multidimensionale.

2.9 Tipologie di Ricerca nello Spazio Metrico

Una volta definita la metrica, possiamo eseguire interrogazioni sul database:

- **Range Search (Ricerca per raggio):** "Trova tutti i vettori che distano meno di r dalla query". Geometricamente, si selezionano i punti dentro un'ipersfera di raggio r .
- **k-Nearest Neighbors (k-NN o Top-k):** "Trova i k vettori più vicini alla query". Il raggio di ricerca non è fisso, ma si espande finché non ingloba k elementi.

2.10 Altre misure di distanza e similarità

Oltre alle metriche L_p standard, esistono misure più sofisticate progettate per gestire correlazioni tra variabili, distribuzioni di probabilità o invarianze di scala.

2.10.1 Distanza di Mahalanobis

La Distanza di Mahalanobis è una misura di distanza tra un punto e una distribuzione (o tra due punti in uno spazio con assi correlati). A differenza della distanza Euclidea, che tratta tutte le dimensioni allo stesso modo e come indipendenti, la Mahalanobis tiene conto della correlazione tra le variabili e della varianza di ciascuna. Geometricamente, mentre la distanza Euclidea forma "sfere" intorno al centro, la distanza di Mahalanobis forma "ellissoidi" orientati secondo la distribuzione dei dati.

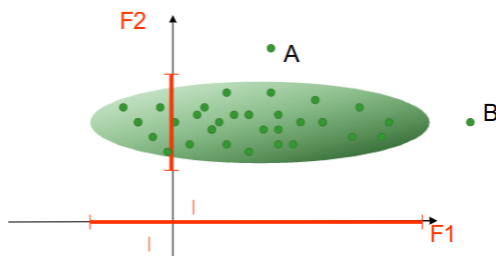


Figura 14: Distanza di Mahalanobis

Il ruolo della Covarianza Se la direzione di massima variabilità dei dati non è allineata con gli assi cartesiani (es. c'è una correlazione tra le feature x e y), la distanza Euclidea può essere ingannevole. La Mahalanobis "schiaccia" o "dila" lo spazio in base alla matrice di covarianza S , trasformando l'ellissoide di distribuzione in una sfera unitaria.

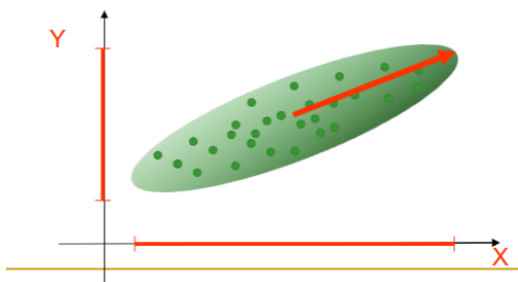


Figura 15: Distanza di Mahalanobis in presenza di feature non allineate con gli assi

Definizioni Formali Siano $\vec{a}$ e $\vec{b}$ due vettori (o un vettore e la media della distribuzione).

- **Matrice di Covarianza (S o Σ):** Una matrice simmetrica che descrive come le feature variano insieme. L'elemento (i, j) è la covarianza tra la feature i e la feature j .

$$S_{xy} = Cov(x, y) = E((x - \mu_x)(y - \mu_y))$$

- **Formula della Distanza:**

$$\Delta_{Mah}(\vec{a}, \vec{b}) = \sqrt{(\vec{a} - \vec{b})^T S^{-1} (\vec{a} - \vec{b})}$$

L'uso dell'inversa della matrice di covarianza (S^{-1}) serve a "normalizzare" i dati, rimuovendo l'effetto delle correlazioni e delle diverse scale. Se le variabili sono incorrelate e hanno varianza unitaria (S è la matrice identità), la distanza di Mahalanobis si riduce alla distanza Euclidea.

2.10.2 Approfondimento: Intuizione sulla Covarianza

Per comprendere la distanza di Mahalanobis, è essenziale capire cosa rappresenta la matrice di covarianza S (o Σ). Essa non ci dice solo quanto i dati sono "sparsi" (varianza), ma anche come si muovono insieme.

Immaginiamo di avere due feature, x (Altezza) e y (Peso):

- **Varianza (σ^2):** È la diagonale della matrice. Ci dice quanto variano i dati lungo un singolo asse. Se la varianza di x è alta e quella di y è bassa, la nuvola di punti sarà allungata orizzontalmente.

- **Covarianza (σ_{xy}):** Sono gli elementi fuori dalla diagonale. Ci dicono se esiste una relazione lineare:
 - **Positiva:** Se x cresce, anche y tende a crescere (es. altezza e peso). La nuvola di punti è inclinata verso destra (/).
 - **Negativa:** Se x cresce, y tende a diminuire. La nuvola è inclinata verso sinistra (\).
 - **Zero:** Le variabili sono indipendenti. La nuvola è circolare o un'ellisse allineata perfettamente agli assi (nessuna inclinazione).

Perché invertiamo la matrice (S^{-1})? Nella formula usiamo l'inversa della covarianza. Intuitivamente, dividere per la varianza significa penalizzare le deviazioni lungo gli assi dove c'è poca variabilità (dove siamo molto sicuri del valore medio) e perdonare le deviazioni lungo gli assi con alta variabilità (dove è normale trovare dati sparsi).

2.10.3 Esempio Numerico Passo-Passo

Immaginiamo uno scenario in cui la distanza Euclidea fallisce nel giudicare la similarità, mentre la Mahalanobis ci dà la risposta corretta.

Sia data una distribuzione di punti con media nell'origine $\vec{\mu} = [0, 0]^T$. Le feature x e y sono positivamente correlate (es. $y \approx x$). Definiamo la matrice di covarianza S :

$$S = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

Consideriamo due punti da analizzare, A e B :

- Punto $A = [1, 1]^T$ (allineato con la correlazione).
- Punto $B = [1, -1]^T$ (opposto alla correlazione).

1. Calcolo della Distanza Euclidea (L_2) Calcoliamo la distanza geometrica semplice dall'origine $(0, 0)$.

$$\Delta_{Euc}(A, \mu) = \sqrt{(1-0)^2 + (1-0)^2} = \sqrt{2} \approx 1.41$$

$$\Delta_{Euc}(B, \mu) = \sqrt{(1-0)^2 + (-1-0)^2} = \sqrt{2} \approx 1.41$$

Risultato Euclideo: I due punti sono considerati **identici** in termini di distanza. Il sistema non riesce a distinguere che A è "normale" (segue il trend) e B è un'anomalia (va contro il trend).

2. Calcolo della Distanza di Mahalanobis Per prima cosa, dobbiamo calcolare l'inversa della matrice di covarianza S^{-1} . Il determinante è $|S| = (2 \cdot 2) - (1 \cdot 1) = 3$.

$$S^{-1} = \frac{1}{|S|} \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} = \begin{bmatrix} 0.66 & -0.33 \\ -0.33 & 0.66 \end{bmatrix}$$

Applichiamo la formula $\Delta_{Mah} = \sqrt{\vec{x}^T S^{-1} \vec{x}}$ per i due punti.

Per il punto A (1, 1):

$$\begin{aligned} \Delta_{Mah}^2(A) &= [1 \quad 1] \cdot \frac{1}{3} \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 1 \end{bmatrix} \\ &= \frac{1}{3} [1 \quad 1] \cdot \begin{bmatrix} (2-1) \\ (-1+2) \end{bmatrix} = \frac{1}{3} [1 \quad 1] \cdot \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \frac{1}{3}(1+1) = 0.66 \\ \Delta_{Mah}(A) &\approx 0.81 \end{aligned}$$

Per il punto B (1, -1):

$$\begin{aligned} \Delta_{Mah}^2(B) &= [1 \quad -1] \cdot \frac{1}{3} \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ -1 \end{bmatrix} \\ &= \frac{1}{3} [1 \quad -1] \cdot \begin{bmatrix} (2+1) \\ (-1-2) \end{bmatrix} = \frac{1}{3} [1 \quad -1] \cdot \begin{bmatrix} 3 \\ -3 \end{bmatrix} = \frac{1}{3}(3+3) = 2 \\ \Delta_{Mah}(B) &\approx 1.41 \end{aligned}$$

Conclusione dell'esempio Nonostante geometricamente siano alla stessa distanza dal centro, la distanza di Mahalanobis ci dice che:

- Il punto A (distanza 0.81) è molto **più vicino** statisticamente, perché si trova lungo la direzione principale dei dati.
- Il punto B (distanza 1.41) è molto **più lontano**, perché si trova in una zona "rara" o anomala rispetto alla distribuzione della covarianza.

2.10.4 Distanza Quadratica (Quadratic Form Distance)

Questa distanza è fondamentale quando le dimensioni del vettore feature non sono indipendenti, ma esiste una somiglianza intrinseca tra di esse (fenomeno noto come "Crosstalk" o correlazione tra bin).

- *Esempio:* Nel confronto di istogrammi di colore. Nella distanza Euclidea standard, un pixel "Rosso" è equidistante da un pixel "Arancione" e da un pixel "Blu". Percettivamente, però, il rosso è molto più simile all'arancione. Un errore di classificazione tra rosso e arancione dovrebbe pesare meno di un errore tra rosso e blu.

Per modellare questo comportamento, definiamo prima una **matrice di similarità** A , dove ogni elemento a_{ij} rappresenta quanto la feature i è simile alla feature j (valore alto = molto simile). La formula della distanza quadratica utilizza l'**inversa** di questa matrice (A^{-1}):

$$\Delta_{quad}(\vec{x}, \vec{y}) = \sqrt{(\vec{x} - \vec{y})^T A^{-1} (\vec{x} - \vec{y})}$$

Interpretazione dell'Inversa (A^{-1}) L'utilizzo della matrice inversa è cruciale per la corretta ponderazione degli errori:

1. **Similarità e Pesì:** Se due feature i e j sono molto simili (es. Rosso e Arancione), il valore corrispondente nella matrice di similarità A_{ij} sarà **alto**.
2. **Inversione:** Passando alla matrice inversa A^{-1} , la relazione si inverte. Intuitivamente, a valori alti in A corrispondono valori ridotti (o pesi minori) nella struttura di A^{-1} che agisce sulle differenze incrociate.
3. **Effetto sulla Distanza:** Quando calcoliamo la distanza, la differenza tra i valori delle feature ($\vec{x} - \vec{y}$) viene moltiplicata per questi pesi contenuti in A^{-1} .

In sintesi:

- **Feature simili** (A_{ij} alto) $\rightarrow$ Peso nell'inversa (A^{-1}) basso $\rightarrow$ La differenza impatta **poco** sulla distanza totale.
- **Feature diverse** (A_{ij} basso o nullo) $\rightarrow$ Peso nell'inversa più rilevante $\rightarrow$ La differenza impatta **molto** sulla distanza totale.

2.10.5 Intersezione di Istogrammi (Intersection Similarity)

È una misura di similarità molto usata in Computer Vision per il riconoscimento di oggetti basato sul colore. Calcola l'area di sovrapposizione tra due istogrammi.

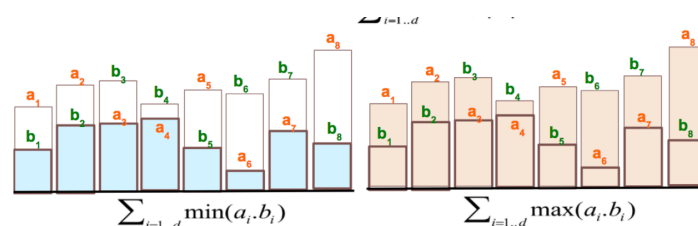


Figura 16: Intersezioni tra due istogrammi

$$sim_{int}(\vec{a}, \vec{b}) = \frac{\sum_{i=1}^n \min(a_i, b_i)}{\sum_{i=1}^n \max(a_i, b_i)}$$

Il risultato varia da 0 (nessuna sovrapposizione) a 1 (istogrammi identici). È robusta a variazioni di sfondo e occlusioni parziali.

2.10.6 Misure basate sugli Angoli (Cosine Similarity)

Queste misure valutano l'orientamento dei vettori nello spazio, ignorando (o normalizzando) la loro magnitudine (lunghezza). Sono particolarmente utili quando la "quantità" assoluta non conta quanto la "proporzione".

- *Esempio Immagini*: Due foto identiche, ma una più luminosa dell'altra, avranno vettori di colore con lunghezze diverse ma stessa direzione.
- *Esempio Testo*: Un documento breve e uno lungo che trattano lo stesso argomento avranno frequenze di parole diverse, ma distribuzione proporzionale simile.

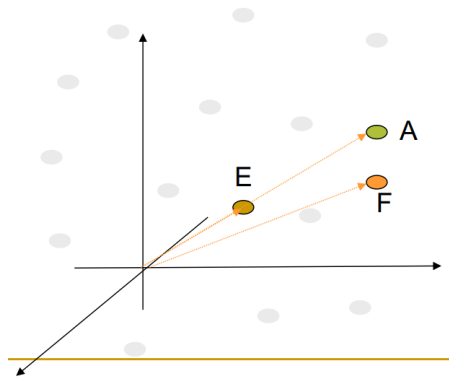


Figura 17: 2 vettori con stessa direzione ma lunghezza diversa

In questo esempio, i vettori A ed F sono più vicini quindi più simili tra loro, ma il vettore E e il vettore A hanno la stessa direzione, quindi stesso angolo rispetto all'origine, quindi sono più simili tra loro rispetto al vettore F per altri fattori.

Dati due vettori $\vec{a}$ e $\vec{b}$:

1. Dot Product (Prodotto Scalare)

$$sim_{dot}(\vec{a}, \vec{b}) = \vec{a} \cdot \vec{b} = \sum_{i=1}^n a_i b_i$$

Il valore dipende dalla lunghezza dei vettori, quindi non è limitato.

2. Cosine Similarity Normalizza il prodotto scalare per le lunghezze dei vettori, ottenendo il coseno dell'angolo θ tra essi:

$$sim_{cos}(\vec{a}, \vec{b}) = \cos(\theta) = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|}$$

- 1: Vettori allineati (massima similarità).
- 0: Vettori ortogonali (non correlati).
- -1: Vettori opposti.

2.10.7 Correlazione di Pearson

La correlazione di Pearson misura la relazione lineare tra due variabili. Nel contesto dei vettori, può essere vista come una Cosine Similarity applicata ai vettori "centrati" (cioè sottraendo la media di ciascun vettore dai suoi componenti).

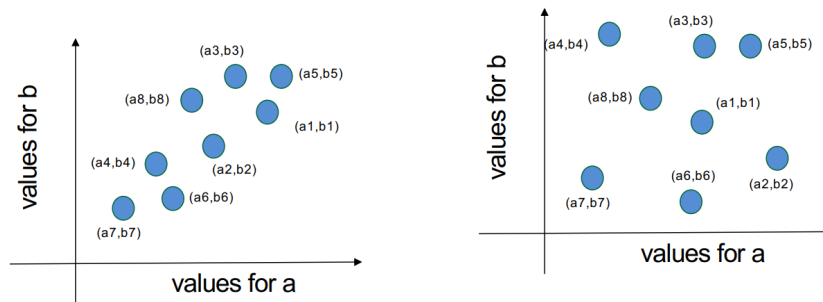


Figura 18: 2 esempi di correlazione di Pearson: a sinistra correlazione positiva, a destra nessuna correlazione

$$\rho(\vec{a}, \vec{b}) = \frac{\sum (a_i - \bar{a})(b_i - \bar{b})}{\sqrt{\sum (a_i - \bar{a})^2} \sqrt{\sum (b_i - \bar{b})^2}}$$

È utile per capire se al variare delle feature di un oggetto, variano linearmente anche quelle dell'altro, indipendentemente dai valori assoluti (shift e scale invariant).

2.10.8 Divergenza di Kullback-Leibler (KL Divergence)

La KL Divergence non è propriamente una distanza (non è simmetrica: $\Delta_{KL}(a, b) \neq \Delta_{KL}(b, a)$), ma misura la divergenza tra due distribuzioni di probabilità P e Q . È spesso usata quando i vettori rappresentano istogrammi normalizzati ($\sum p_i = 1$).

$$\Delta_{KL}(\vec{a}, \vec{b}) = \sum_i a_i \log \left(\frac{a_i}{b_i} \right)$$

Interpretazione della Teoria dell'Informazione L'Entropia $H(P)$ misura l'incertezza o la quantità media di informazione necessaria per descrivere eventi di una distribuzione P :

$$H(P) = - \sum p_i \log_2(p_i)$$

La KL Divergence misura la "sorpresa" o l'inefficienza (bit extra necessari) quando si usa la distribuzione Q per approssimare la vera distribuzione P . Se $\Delta_{KL}(\vec{a}, \vec{b}) = 0$, le distribuzioni sono identiche. Più alto è il valore, più informazione si perde usando b per descrivere a .

Lo spazio dedicato alla singola codifica è:

$$sp(e_1) = \log_2 \frac{1}{prob}$$

Quando devo comprimere un documento costituito da N termini, se gli eventi fossero equiprobabili potrei usare il logaritmo, invece se non lo sono, avrò che per ciascuno $\log \frac{1}{p_i}$, per comprimere l'intero documento mi serviranno $\sum_{i=1}^N p_i \log \frac{1}{p_i}$.

Nella kl distance uso $\log_2 \frac{1}{q_i}$ mi chiedo quanto sarà l'errore di codifica cambiando distribuzione di probabilità e lo ottengo

$$\sum_{i=1}^N p_i \log_2 \frac{1}{q_i} - \sum_{i=1}^N p_i \log_2 \frac{1}{p_i} = \sum_{i=1}^N p_i \log_2 \frac{p_i}{q_i}$$

Così facendo ottengo la formula della kl divergence.

Se voglio renderla simmetrica, posso fare la media tra le due versioni.

2.10.9 Earth Mover's Distance (EMD)

L'EMD (nota anche come metrica di Wasserstein per distribuzioni discrete) è una misura robusta per confrontare due distribuzioni di probabilità o istogrammi. A differenza delle distanze bin-to-bin (come la Minkowski) che confrontano solo indici corrispondenti, l'EMD tiene conto della "distanza di terra" (*ground distance*) tra i bin, permettendo di gestire shift e deformazioni.

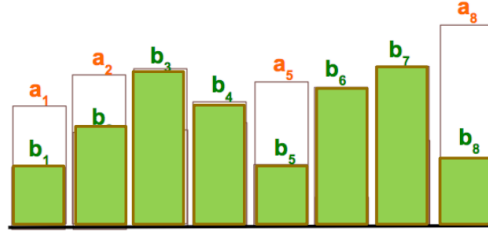


Figura 19: Esempio di Earth Mover's Distance: spostamento di massa tra due istogrammi

L'Analogia del Trasporto Immaginiamo due istogrammi come due accumuli di terra:

- Il vettore $\vec{a} = \langle a_1, \dots, a_n \rangle$ rappresenta la configurazione iniziale (le colline di terra).
- Il vettore $\vec{b} = \langle b_1, \dots, b_n \rangle$ rappresenta la configurazione target (le buche da riempire).

L'EMD calcola il **costo minimo** di lavoro necessario per trasformare $\vec{a}$ in $\vec{b}$ spostando la "massa" di probabilità.

Definizione Formale del Flusso Definiamo ab_{ij} come la quantità di flusso (probabilità) spostata dal bin i di $\vec{a}$ al bin j di $\vec{b}$. Ogni spostamento ha un costo unitario c_{ij} , che rappresenta la distanza semantica o fisica tra la feature i e la feature j . Il problema viene modellato come un problema di *Trasporto Ottimo* (Linear Programming):

1. Minimizzare il Lavoro Totale:

$$W = \sum_{i=1}^n \sum_{j=1}^n c_{ij} \cdot ab_{ij}$$

2. Soggetto ai vincoli:

- **Non negatività:** $ab_{ij} \geq 0$ (non posso spostare massa negativa).
- **Vincolo sulla Sorgente:** La somma della massa che esce da a_i deve eguagliare il valore originale a_i :

$$\sum_{j=1}^n ab_{ij} = a_i$$

- **Vincolo sulla Destinazione:** La somma della massa che arriva in b_j deve eguagliare il valore target b_j :

$$\sum_{i=1}^n ab_{ij} = b_j$$

- **Conservazione Totale:** La massa totale spostata è normalizzata (solitamente a 1 per distribuzioni di probabilità):

$$\sum_{i=1}^n \sum_{j=1}^n ab_{ij} = 1$$

La distanza finale è il lavoro normalizzato per il flusso totale:

$$\Delta_{EMD}(\vec{a}, \vec{b}) = \frac{\sum_{i=1}^n \sum_{j=1}^n c_{ij} \cdot ab_{ij}}{\sum_{i=1}^n \sum_{j=1}^n ab_{ij}} = \sum_{i=1}^n \sum_{j=1}^n cost_{ij}$$

Se i vettori sono normalizzati (somma = 1), il denominatore è 1 e la formula si semplifica nella somma dei costi ponderati.

3 Quante features vogliamo?

La scelta del numero di dimensioni N è un compromesso critico. Vorremmo il minor numero possibile di features per efficienza, ma un numero sufficiente per non perdere potere descrittivo.

3.1 Dimensionality Curse (Maledizione della Dimensionalità)

Al crescere del numero di dimensioni N , si verificano fenomeni controintuitivi che degradano le prestazioni dei sistemi di retrieval.

Problema di Efficienza (Indici) Le strutture di indicizzazione spaziale (come R-Trees o KD-Trees) diventano esponenzialmente meno efficienti.

- Richiedono spazio di memorizzazione crescente.
- Il tempo di attraversamento dell'albero aumenta.
- Sopra una certa soglia (spesso $N > 10-15$), la ricerca indicizzata diventa più lenta di una scansione sequenziale dell'intero database (Sequential Scan).

Problema di Efficacia (Distanze) Più dimensioni aggiungiamo, più dati servono per evitare l'*overfitting*. Ma il problema geometrico è più grave: in spazi ad alta dimensionalità, la distribuzione delle distanze si "appiattisce". La distanza tra il punto di query e il suo vicino più prossimo (d_{min}) tende ad avvicinarsi alla distanza con il vicino più lontano (d_{max}).

$$\lim_{N \rightarrow \infty} \frac{d_{max} - d_{min}}{d_{min}} \rightarrow 0$$

Se tutti i punti sono quasi equidistanti, il concetto di "similarità" perde significato.

3.2 Effetto geometrico: Ipersfere e Gusci

Al crescere delle dimensioni, il volume dello spazio si concentra nei "gusci" esterni o negli angoli dell'iper-cubo, lasciando il centro vuoto (*Empty Space Phenomenon*). Supponiamo che lo spazio sia diviso in gusci concentrici attorno alla query. Se il numero di punti nella sfera più interna è I , il volume (e quindi il numero di punti attesi in distribuzione uniforme) nei gusci esterni cresce esponenzialmente:

- 1° strato (interno): I
- 2° strato: $\approx O(2^{n-1}I)$
- 3° strato: $\approx O(3^{n-1}I)$

Questo implica che la stragrande maggioranza dei dati si trova in uno strato periferico lontanissimo dalla query, rendendo la discriminazione basata sulla distanza estremamente difficile.

3.2.1 Power Law

Spesso la distribuzione delle distanze o la frequenza delle feature segue una legge di potenza (Power Law). Pochi termini sono molto frequenti (o poche dimensioni portano molta informazione), mentre una "coda lunga" di dimensioni aggiunge rumore o informazione marginale, contribuendo però negativamente al calcolo delle distanze (fenomeno di concentrazione della misura).

3.3 Criteri per la Selezione delle Feature

Per scegliere il sottoinsieme ottimo di feature, ci basiamo su:

1. **Semantica dell'applicazione:** Le feature devono avere senso nel dominio specifico (es. texture per tessuti, forma per loghi, colore per quadri).
2. **Percezione utente:** Se l'occhio umano non distingue due sfumature (JND - Just Noticeable Difference), è inutile usare due bin separati. La granularità deve rispecchiare la percezione.
3. **Potere descrittivo:** La feature deve descrivere l'oggetto in modo fedele.
4. **Potere discriminante:** È il criterio più importante per il retrieval. La feature deve saper distinguere oggetti diversi. Se una feature ha lo stesso valore per tutto il database, è inutile.
5. **Query Workload:** Scartiamo feature che non verranno mai interrogate dagli utenti (es. feature audio in un DB dove si cercano solo immagini per colore).

3.3.1 Caratteristiche di una "Buona Feature"

Una feature è utile se è **significativa** e **discriminante**. Esiste un legame diretto con la teoria dell'informazione: un evento (o feature) è significativo se porta informazione.

- **Evento comune:** "Oggi c'è il sole" → Bassa sorpresa → Bassa informazione.
- **Evento raro:** "Oggi c'è un'eclissi" → Alta sorpresa → Alta informazione.

Vogliamo feature che catturino eventi "rari" o distintivi degli oggetti, non caratteristiche piatte comuni a tutti.

3.4 Entropia come misura di qualità

Possiamo formalizzare il potere discriminante di una feature usando l'Entropia di Shannon $H(X)$. Data una feature X che può assumere valori x con probabilità $p(x)$:

$$H(X) = \sum_{x \in X} p(x) \log_2 \frac{1}{p(x)} = - \sum_{x \in X} p(x) \log_2 p(x)$$

- **Alta Entropia (vicino a $\log_2 N$):** La distribuzione dei valori è uniforme (istogramma piatto). La feature è molto varia e imprevedibile. **Ottima per discriminare.**
- **Bassa Entropia (vicino a 0):** La distribuzione è concentrata su un solo valore (istogramma a picco). Tutti gli oggetti hanno quasi lo stesso valore per questa feature. **Pessima per discriminare.**

3.5 Feature Latenti e Trasformazione dello Spazio

In un caso in cui ho due feature, entrambe parallele agli assi, ma devo sceglierne solo una, quale scelgo? La scelta migliore è quella che ha più potere discriminante, cioè quella con entropia più alta (in poche parole quella che se scelgo un intervallo di valori, restituisce un numero di oggetti più piccolo possibile).

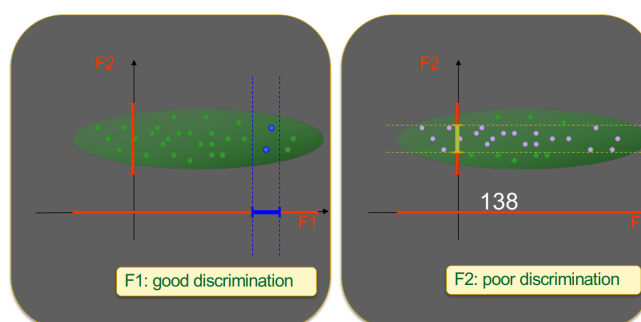


Figura 20: Esempio di scelta tra due feature parallele agli assi

Spesso le feature misurate ("native") sono ridondanti o correlate (es. altezza e peso). Questo spreca dimensioni e peggiora la dimensionality curse. L'obiettivo è trovare delle **feature latenti** (nascoste) che sintetizzino l'informazione.

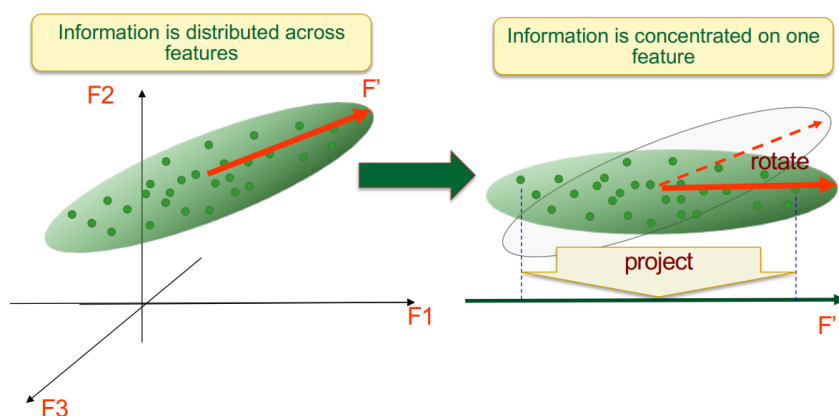


Figura 21: Esempio di feature latente

Il processo (es. PCA/SVD)

1. **Analisi della Correlazione:** Se nessuna feature è allineata con gli assi, i dati formano una nuvola obliqua.
2. **Rotazione:** Cerchiamo la direzione di *massima variabilità* dei dati. Questa direzione diventa il nuovo primo asse.
3. **Combinazione Lineare:** La nuova feature (latente) non è più una singola proprietà fisica, ma una combinazione matematica delle feature originali (es. $0.7 \cdot \text{Altezza} + 0.3 \cdot \text{Peso}$).
4. **Proiezione:** Tutti i dati (e le future query) vengono proiettati su questo nuovo sistema di riferimento.

3.5.1 Cosa sono le Feature Latenti

Le feature latenti sono proprietà **nascoste** dei dati che non vengono misurate direttamente, ma estratte matematicamente analizzando le correlazioni tra le feature originali. Mentre le feature misurate descrivono i dati grezzi, le feature latenti rappresentano i concetti astratti o le strutture fondamentali che governano quei dati, ripuliti dal rumore e dalla ridondanza.

Esempio: Dai Termini ai Concetti (Text Mining) Immaginiamo un database di documenti dove le feature misurate sono le singole parole. Se l'analisi rileva che i termini "motore", "ruote", "freni" e "carrozzeria" compaiono quasi sempre insieme (alta covarianza), significa che sono fortemente correlati.

Il sistema matematico aggrega queste variabili in un'unica feature latente. Questa nuova feature non corrisponde a una parola scritta, ma rappresenta il Topic "Automobili".

- **Prima:** Avevamo 4 dimensioni (4 parole diverse).
- **Dopo:** Abbiamo 1 dimensione (il concetto di "Automobile").

Il sistema ha ridotto la dimensionalità catturando la semantica nascosta dietro le parole.

Vantaggi

- **Compressione:** Spesso le prime k feature latenti catturano il 95% dell'informazione totale. Possiamo scartare le restanti $N - k$ dimensioni (riduzione dimensionale).
- **Decorrelazione:** Le feature latenti sono ortogonali tra loro.
- **Robustezza:** L'utente non interroga direttamente le feature latenti (non hanno un nome semantico intuitivo), ma il sistema le usa per calcolare distanze più accurate e veloci.

3.6 Rappresentazione dei testi e Feature Extraction

Nel campo dell'Information Retrieval e del Natural Language Processing, ogni documento testuale viene tipicamente modellato attraverso l'approccio **Bag of Words (BoW)**. In questo modello, un documento è rappresentato come un **multi-insieme (multi-set)** di parole chiave.

Parliamo di *multi-insieme* e non di semplice *insieme* perché viene mantenuta l'informazione sulla **frequenza** di ciascuna parola all'interno del documento. È proprio la frequenza (il numero di occorrenze) che ci permette di caratterizzare e distinguere un documento rispetto a un altro.

In questa rappresentazione strutturata **l'ordine delle parole viene ignorato**. Di conseguenza, frasi come *"il gatto insegue il topo"* e *"il topo insegue il gatto"* possiedono esattamente la stessa rappresentazione vettoriale, perdendo l'informazione legata alla sintassi.

Affinché la rappresentazione sia efficace, le parole chiave estratte devono essere **portatrici di contenuto**. È quindi fondamentale distinguere il vocabolario in due categorie:

- **Content words (Terms):** Sono le parole che veicolano il significato del testo e caratterizzano un documento (es. sostantivi, verbi, aggettivi).

Questi termini non vengono quasi mai mantenuti nella loro forma flessa originale. Per ridurre la dimensionalità del vocabolario e aggregare parole con lo stesso significato, si applicano due processi principali:

- **Stemming:** Taglia la parte finale della parola per estrarne la radice (es. "correre", "correva", "corso" → "corr").
- **Lemmatizzazione:** Riconduce la parola alla sua forma base, o *lemma*, tipicamente quella che troveremmo sul dizionario (es. maschile singolare per gli aggettivi, infinito per i verbi).

Questo processo aumenta la frequenza aggregata dei termini, migliorandone il potere discriminante. Inoltre, i sistemi più avanzati riconoscono le **parole composte** (o *n-grammi*), come "casa editrice", trattandole come un concetto unico (feature singola) anziché come la somma dei singoli termini "casa" ed "editrice". L'insieme di tutte queste feature raffinate costituisce il **vocabolario** utilizzato per costruire i vettori.

- **Non-content words (Stop words):** Sono parole di uso comune che fungono da collante sintattico ma non portano informazione semantica rilevante (es. articoli, preposizioni, congiunzioni). Essendo presenti in modo omogeneo in quasi tutti i documenti della collezione, non aiutano a discriminare un testo dall'altro. Dal punto di vista della Teoria dell'Informazione, avendo una probabilità di comparsa altissima, queste parole apportano un **contenuto informativo (self-information) quasi nullo**. Per questo motivo, vengono filtrate e rimosse in fase di pre-processing.

3.6.1 Rappresentazione vettoriale (Vector Space Model)

Dato un vocabolario di termini estratti dalla collezione, ogni documento d_i viene rappresentato come un vettore numerico:

$$d_i = \langle w_{i1}, w_{i2}, w_{i3}, \dots, w_{in} \rangle$$

La dimensione del vettore corrisponde all'esatto numero di termini n presenti nell'intero vocabolario.

Il peso w_{ij} quantifica l'importanza (o la frequenza) del termine j -esimo all'interno del documento i -esimo:

- $w_{ij} = 0$: se il termine j **non compare** nel documento d_i . Spesso questi vettori sono *sparsi*, contenendo moltissimi zeri.
- $w_{ij} > 0$: se il termine j **compare** nel documento d_i , con un valore che cresce in base alla sua rilevanza.

3.6.2 Distribuzione Zipfiana

La distribuzione delle frequenze dei termini in un linguaggio naturale (e quindi nei documenti) non è uniforme, ma segue la **Legge di Zipf**. Essa stabilisce che la frequenza della k -esima parola più comune in una collezione è inversamente proporzionale al suo rango k , ed è pari a $(\frac{1}{k})^\Theta$ volte la frequenza della parola più comune (con il parametro $\Theta \approx 1$).

Questo significa che poche parole occorrono spessissimo (le *stop words*), mentre la stragrande maggioranza dei termini compare molto raramente.

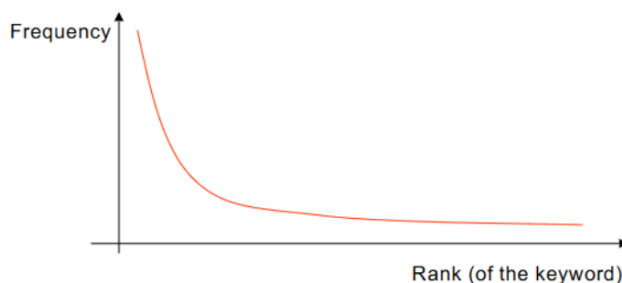


Figura 22: Rappresentazione grafica della Legge di Zipf: si nota il rapido decadimento della frequenza all'aumentare del rango del termine.

3.6.3 Come definire i pesi vettoriali

I pesi vettoriali devono quantificare quanto una determinata feature sia "brava" a descrivere il contenuto di uno specifico documento all'interno di una collezione.

1. Term Frequency (TF) La *Term Frequency* è una misura locale che indica la densità di un termine all'interno di un *singolo* documento. È definita come:

$$tf = \frac{n}{K}$$

dove n è il numero di occorrenze del termine nel documento e K è il numero totale di termini in quel documento.

Esempio: Se in un documento breve la parola "Partita" compare 3 volte, e in un documento molto lungo compare 30 volte, dividere per K (normalizzazione sulla lunghezza del documento) ci permette di capire in quale dei due documenti il concetto di "Partita" è in proporzione più centrale e rilevante.

Tuttavia, la TF da sola non basta per valutare il **potere discriminante** di una feature. Se una parola è frequente in un documento, ma è ugualmente frequente in *tutti* gli altri documenti della collezione, allora non ci aiuta a distinguerli. Al contrario, un termine raro nella collezione generale, ma molto frequente nel nostro documento, risulta essere un'ottima feature distintiva.

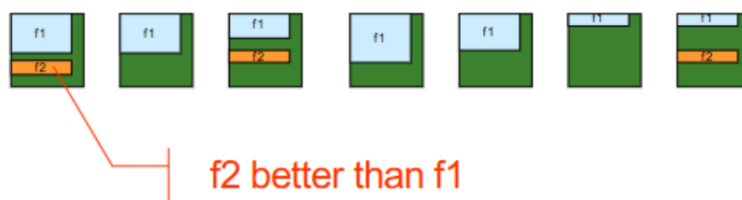


Figura 23: Esempio intuitivo: una feature presente in pochi documenti ha un potere discriminante molto più elevato rispetto a una feature ubiquitaria.

2. Inverse Document Frequency (IDF) L'*Inverse Document Frequency* è una misura globale che valuta quanto un termine sia raro o comune in **tutta la collezione** di documenti:

$$idf = \log \left(\frac{N}{m} \right)$$

dove N è il numero totale di documenti della collezione e m è il numero di documenti in cui compare almeno una volta il termine in esame. Più il termine è raro (ovvero m è piccolo), maggiore sarà il valore di $\frac{N}{m}$, e di conseguenza l'IDF sarà più alto.

Nota architetturale: Poiché l'IDF dipende da N (che cambia man mano che si inseriscono nuovi documenti nel sistema), i motori di ricerca e i database documentali solitamente non memorizzano l'IDF staticamente, ma aggiornano o calcolano questo valore dinamicamente.

3. TF-IDF Il peso finale più utilizzato è il prodotto tra TF e IDF, chiamato **TF-IDF**. Questa metrica bilancia l'importanza locale del termine e la sua specificità globale:

$$tf_idf = tf \cdot idf = \frac{n}{K} \cdot \log\left(\frac{N}{m}\right)$$

Interpretazione: Due termini che compaiono con la stessa frequenza (TF) in un documento avranno pesi finali (TF-IDF) diversi: il termine presente in meno documenti della collezione (IDF più alto) riceverà un peso maggiore perché è più caratteristico.

Gestione degli edge cases (Smoothing): Un potenziale problema della formula standard dell'IDF si presenta quando $m = 0$ (un termine cercato non è presente in nessun documento della collezione), portando il denominatore a zero e generando un asintoto a infinito. Per risolvere questo problema, nei sistemi reali si utilizza spesso una versione "lisciata" (smoothed), aggiungendo 1 al denominatore: $idf = \log\left(\frac{N}{m+1}\right)$.

4. Normalizzazione del TF-IDF La componente TF ($\frac{n}{K}$) assume per sua natura valori compresi tra 0 e 1. L'IDF, invece, è un logaritmo che può crescere notevolmente per termini estremamente rari in collezioni immense. Per evitare che il fattore globale (IDF) "schiacci" completamente il fattore locale (TF), si adotta spesso una versione **normalizzata** della metrica:

$$norm_tf_idf = \frac{n}{K} \cdot \frac{\log\left(\frac{N}{m}\right)}{\max idf}$$

Dividendo per il valore massimo di IDF calcolato nella collezione, si forza anche il termine IDF in un range tra 0 e 1. In questo modo si dà pari potenziale alle due componenti. Nel caso in cui si desideri asimmetria (es. privilegiare la rarità globale), si possono inserire dei coefficienti di peso correttivi (α, β).

3.7 Estrazione di Feature dalle Immagini

Per poter analizzare e confrontare le immagini tramite algoritmi, è necessario passare dalla matrice grezza di pixel a una **rappresentazione vettoriale** (feature vector).

Una rappresentazione intermedia molto comune è l'utilizzo degli **istogrammi**, che riassumono la distribuzione globale di una certa caratteristica nell'immagine. Un istogramma può essere interpretato come un conteggio delle frequenze di valori esatti, oppure come l'accumulo di valori all'interno di intervalli predefiniti (*bin*).

Le feature estraibili da un'immagine possono essere classificate in base al loro livello di astrazione:

- **Low-level features:** Colori, Edges (bordi/contorni), Texture (trama).
- **Mid-level features:** Segments (regioni o segmenti di immagine con caratteristiche omogenee).
- **High-level features:** Objects (oggetti riconosciuti), Metadata (informazioni EXIF), Captions e Tags.

3.7.1 Rappresentazione del Colore

Modello RGB Nel modello **RGB**, ciascun colore è ottenuto tramite la sintesi additiva di tre componenti fondamentali: Rosso (R), Verde (G) e Blu (B). Queste componenti sono considerate *linearmente indipendenti*, ovvero nessuna delle tre può essere ottenuta come combinazione matematica delle altre due. Fungono da "vettori base" per generare tutti gli altri colori dello spettro visibile a monitor.

Lavorando tipicamente su 8 bit per canale, i valori spaziano da 0 a 255: la terna (0, 0, 0) rappresenta il nero (assenza totale di luce), mentre (255, 255, 255) rappresenta il bianco (massima intensità luminosa di tutti i canali).

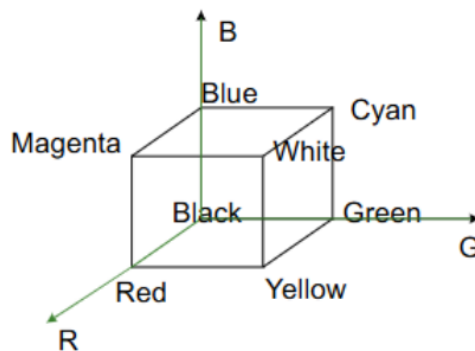


Figura 24: Scomposizione di un'immagine nei tre canali del modello RGB.

Modello HSV Se l'obiettivo è elaborare l'immagine basandosi su come gli esseri umani "pensano" al colore, il modello RGB risulta poco intuitivo. Il modello **HSV** separa la componente cromatica da quella di illuminazione, risultando molto più vicino alla percezione umana:

- **Hue (H - Tonalità)**: Rappresenta il tipo di colore (es. rosso, verde, blu). Geometricamente è spesso rappresentato come un angolo (da 0° a 360°) su una ruota dei colori.
- **Saturation (S - Saturazione)**: Indica la purezza o "pienezza" del colore (da 0 a 1). A 0 si ha una sfumatura di grigio, a 1 si ha il colore puro senza contaminazioni di bianco.
- **Value (V - Valore/Luminosità)**: Indica la brillantezza del colore (da 0 a 1). A 0 il colore è completamente nero, a 1 ha la massima luminosità possibile.

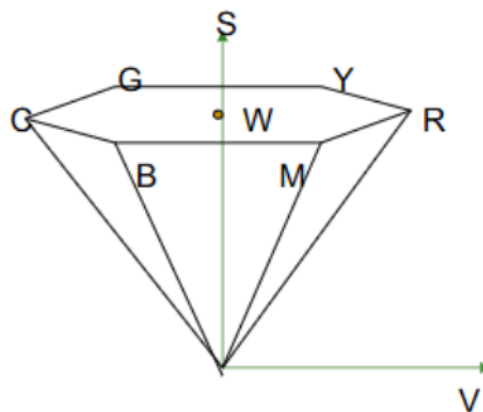


Figura 25: Rappresentazione cilindrica e scomposizione dello spazio colore HSV.

Modello CIELAB (o Lab*) Il modello CIELAB è uno spazio colore *perceptivamente uniforme*. Questo significa che una variazione numerica fissa all'interno di questo spazio corrisponde a una variazione visiva di pari entità percepita dall'occhio umano, cosa che non avviene in RGB o HSV. È lo standard per misurare le differenze oggettive tra colori.

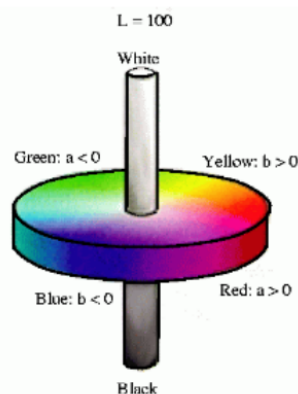


Figura 26: Esempio di rappresentazione nello spazio colore uniforme CIELAB.

3.7.2 Rappresentazioni Lossy e Compressione

Un'immagine grezza ad alta risoluzione contiene spesso micro-dettagli o variazioni cromatiche che l'occhio umano non è fisicamente in grado di percepire. Utilizzare un modello troppo dettagliato, mantenendo tutte queste informazioni, è controproducente: aumenta drasticamente lo spazio di archiviazione, i tempi di calcolo, e rischia di far distinguere all'algoritmo feature che in realtà sono semplice "rumore" visivo, peggiorando il potere discriminante del modello.

Per questo si ricorre a tecniche di **riduzione della dimensionalità** e compressione **lossy** (con perdita di dati, non reversibile, come il formato JPEG).

Il principio biologico fondamentale su cui si basano queste compressioni è che **l'occhio umano è molto più sensibile alle variazioni di luminosità (contrasto) che alle variazioni di colore**.

Modello YUV Per sfruttare questo limite biologico, si passa dallo spazio RGB a spazi colore che separano la luminosità dal colore, come lo **YUV** (una trasformazione lineare di RGB):

- **Y (Luma)**: Rappresenta la luminanza, ovvero l'immagine in scala di grigi. Contiene il dettaglio e il contrasto dell'immagine.
- **U e V (Chroma)**: Rappresentano la cromaticità (l'informazione sul colore). In particolare, *U* mappa l'asse blu-giallo e *V* l'asse rosso-ciano.

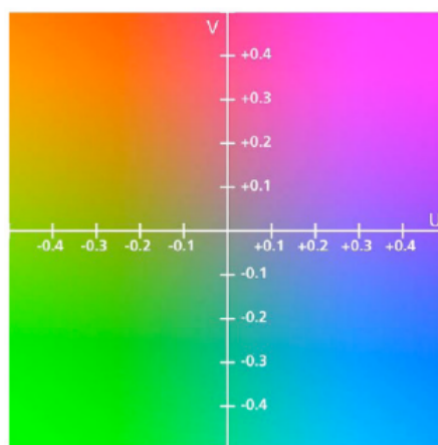


Figura 27: Schema del piano colore YUV.

Poiché la componente Y è la più importante per l'occhio umano, in fase di compressione possiamo preservarla al massimo e "sacrificare" le componenti U e V (tecnica nota come *Chroma Subsampling*).

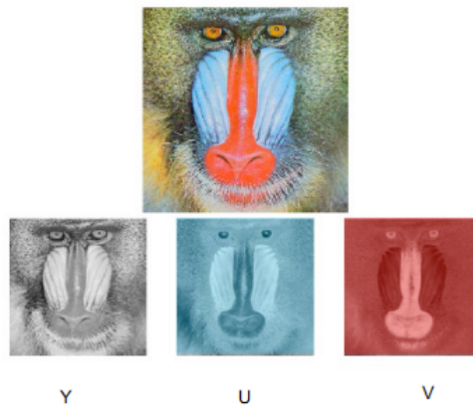


Figura 28: Scomposizione di un'immagine nei canali Y, U e V. Si nota come il canale Y contenga quasi tutta l'informazione strutturale.

Esempio pratico: Se in RGB usiamo 8 bit per canale (24 bit totali per pixel), non possiamo scartare bit senza alterare drasticamente l'immagine, poiché i tre canali sono ugualmente importanti. In YUV, potremmo decidere di usare 8 bit per la componente Y (mantenendo intatti i dettagli di luce) e solo 4 bit per U e 4 bit per V, portando il peso del pixel da 24 a 16 bit totali senza che l'occhio umano percepisca un degrado significativo della qualità.

Modello YIQ Oltre a essere più sensibile alla luminosità che al colore, il sistema visivo umano ha un'ulteriore particolarità: discrimina i dettagli molto meglio nelle sfumature tra l'arancione e il blu rispetto a quelle tra il verde e il magenta. Il modello **YIQ** (storicamente usato per le trasmissioni televisive NTSC) sfrutta proprio questo:

- **Y:** Luminanza (scala di grigi).
- **I (In-phase):** Rappresenta l'asse cromatico arancione-blu (a cui siamo più sensibili).
- **Q (Quadrature):** Rappresenta l'asse cromatico verde-magenta (a cui siamo meno sensibili).

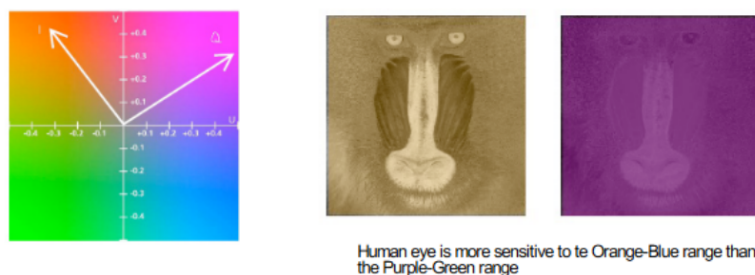


Figura 29: Rappresentazione dei canali nello spazio colore YIQ.

Esempio di compressione estrema: In fase di codifica, conoscendo questi limiti biologici, potremmo assegnare 8 bit al canale Y (massima fedeltà per i contorni), 6 bit al canale I (buona fedeltà per i colori arancione/blu) e soli 2 bit al canale Q (bassissima fedeltà per verde/magenta). In questo modo, l'immagine viene fortemente compressa pur mantenendo un'apparenza visiva eccellente per un osservatore umano.

Momenti dei colori I momenti dei colori sono feature globali compatte che descrivono statisticamente la distribuzione dei valori di colore all'interno di un'immagine. Dato un canale colore (ad esempio, il rosso R), i primi tre momenti sono:

- **Primo momento (Media - Mean):**

$$Avg_R = \frac{1}{N} \sum_{x,y} R(x,y)$$

dove N è il numero totale di pixel. Rappresenta l'intensità media di quel colore nell'immagine. Un Avg_R alto indica un'immagine prevalentemente rossa.

- **Secondo momento (Deviazione standard - Standard Deviation):**

$$STD_R = \sqrt{\frac{1}{N} \sum_{x,y} (R(x,y) - Avg_R)^2}$$

Misura la varianza, ovvero quanto i pixel si discostano in media dal valore centrale. Un valore alto indica un elevato contrasto o una grande varietà di sfumature di quel colore (es. un tramonto); un valore basso indica un colore molto uniforme (es. un cielo sereno o uno sfondo a tinta unita).

- **Terzo momento (Asimmetria - Skewness):**

$$Skew_R = \sqrt[3]{\frac{1}{N} \sum_{x,y} (R(x,y) - Avg_R)^3}$$

Indica l'asimmetria della distribuzione dei colori rispetto alla media.

- Se è **positivo**, la distribuzione ha una "coda" verso i valori alti: la maggior parte dei pixel ha un valore di rosso inferiore alla media, ma ci sono pochi pixel con rosso molto intenso.
- Se è **negativo**, la coda è verso i valori bassi: l'immagine ha molti pixel di rosso intenso, con pochi pixel scuri che abbassano la media.

Si potrebbe proseguire con momenti di ordine superiore (es. la *Curtosi*), ma in pratica i primi tre momenti sono sufficienti a fornire un'ottima sintesi. Poiché l'immagine ha tipicamente 3 canali (es. RGB o HSV), un vettore di feature basato sui momenti avrà dimensione $3 \times 3 = 9$ feature in totale.

Essendo misure globali, i momenti perdono completamente l'informazione sulla **disposizione spaziale** dei colori. Per questo, spesso si integrano con istogrammi o partizioni locali.

3.7.3 Istogramma dei colori

Un istogramma rappresenta la distribuzione globale dei colori quantizzati. Per ogni "bin" (che rappresenta un colore o un range di colori del vocabolario scelto), l'istogramma conta il numero assoluto o la percentuale di pixel che assumono quel colore nell'immagine.

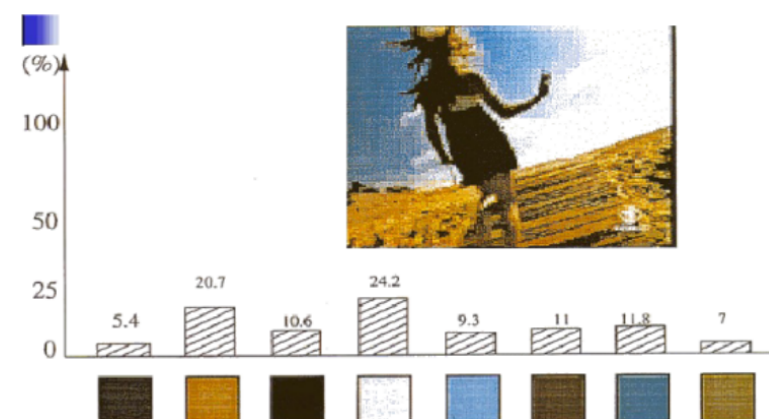


Figura 30: Esempio di istogramma dei colori: l'asse X rappresenta i colori del vocabolario, l'asse Y la frequenza dei pixel.

Problemi degli istogrammi standard Il limite principale degli istogrammi standard è la mancanza di correlazione semantica tra i bin. Il conteggio considera ogni colore come indipendente: un pixel rosso fuoco e un pixel rosso corallo, se assegnati a due bin diversi, vengono considerati distanti quanto un pixel rosso e uno blu. Questo problema viene risolto utilizzando distanze avanzate (come la forma quadratica) che considerano il *cross-talk* tra i colori.

3.7.4 Istogrammi di colore locali (Spatial Grids)

Per recuperare l'informazione spaziale persa dagli approcci globali, l'immagine viene frammentata in una griglia fissa (es. 3×3 o 4×4 celle). Per ogni cella viene calcolato un istogramma (o i momenti dei colori) indipendente. Il vettore di feature finale è la concatenazione delle feature locali. In questo modo, il sistema è in grado di distinguere, ad esempio, un'immagine con il "cielo azzurro in alto e prato verde in basso" da una con "prato verde in alto e mare azzurro in basso", che globalmente avrebbero lo stesso istogramma.

3.7.5 Come scegliere il vocabolario dei colori

Per costruire un istogramma è necessario discretizzare lo spazio continuo dei colori in un numero finito di *bin*. Esistono due approcci principali:

1. Vocabolario definito a priori (Data-independent) Lo spazio colore viene partizionato staticamente, indipendentemente dal dataset. Ad esempio, si dividono uniformemente gli assi R, G e B in 4 intervalli ciascuno, ottenendo $4^3 = 64$ bin.

- **Pro:** Sistema flessibile, pronto ad accogliere qualsiasi nuova immagine senza dover ricalcolare il modello.
- **Contro:** Molti bin potrebbero rimanere vuoti (es. se nel database non ci sono immagini viola, il bin del viola sarà un inutile spreco di spazio).

2. Vocabolario definito dai dati (Data-driven) Se il database è relativamente statico e si conosce il dominio delle immagini, si analizza la distribuzione reale dei pixel in tutto il dataset per creare un vocabolario su misura. In questo modo si ottimizza lo spazio: ci saranno più bin dedicati ai colori dominanti del dataset e pochissimi per i colori rari.

Algoritmo Median-Cut È l'algoritmo classico per la quantizzazione data-driven dei colori. Si basa su una divisione iterativa dello spazio RGB:

1. Si inseriscono tutti i pixel del database in un unico grande contenitore (cluster).
2. Si individua il contenitore con la massima varianza lungo uno dei tre assi colore (R, G o B). Ad esempio, si trova che i colori variano maggiormente lungo il canale del Verde.
3. Si ordinano i pixel di quel contenitore rispetto al canale scelto e si calcola il valore **mediano**.
4. Si "taglia" il contenitore in due esatte metà basandosi su questo valore mediano, creando due nuovi cluster.
5. Si ripete il processo sui nuovi cluster fino a raggiungere il numero di bin desiderato (K).

Il risultato è un vocabolario ottimizzato dove ogni bin ha quasi lo stesso numero di pixel complessivi, garantendo un'alta entropia e un ottimo potere discriminante.

3.7.6 Confronti tra istogrammi dei colori

Siano H e K due istogrammi normalizzati (frequenze relative) con n bin.

- **Distanza Euclidea** (L_2 norm):

$$D_{Euc}(H, K) = \sqrt{\sum_{i=1}^n (H_i - K_i)^2}$$

Semplice e veloce, ma non tiene conto della somiglianza tra colori di bin diversi (problema dell'indipendenza dei bin).

- **Intersection Similarity (Histogram Intersection):**

$$S_{Int}(H, K) = \frac{\sum_{i=1}^n \min(H_i, K_i)}{\sum_{i=1}^n K_i}$$

Misura l'area di sovrapposizione tra i due istogrammi. Se gli istogrammi sono identici, il valore è 1. È molto robusta ai rumori di fondo e occlusioni parziali.

Distanza in Forma Quadratica (Quadratic Form Distance) Per risolvere il problema dell'indipendenza dei bin, si utilizza una matrice di pesi A .

$$D_{quad}^2(H, K) = (H - K)^\top A (H - K) = \sum_{i=1}^n \sum_{j=1}^n A_{ij} (H_i - K_i) (H_j - K_j)$$

Il fattore A_{ij} è detto *cross-talk* e quantifica la somiglianza percettiva tra il colore del bin i e quello del bin j .

- Se A è la matrice identità (tutti zeri tranne la diagonale), la formula si riduce alla Distanza Euclidea standard.
- Se A_{ij} è alto, l'algoritmo "perdona" l'immagine se un pixel è caduto nel bin del rosso chiaro anziché in quello del rosso scuro, considerandoli affini.

Problema: La complessità computazionale diventa quadratica $O(n^2)$. Confrontare istogrammi molto densi su database enormi diventa troppo lento.

Filtro spaziale e Lower-Bounding (Filter-and-Refine) Per accelerare la ricerca senza perdere precisione, si applica la teoria del *Lower Bounding* (alla base di algoritmi come GEMINI).

Consideriamo le medie dei canali RGB delle due immagini come un vettore a 3 dimensioni: $\mu_x = [R_{avgx}, G_{avgx}, B_{avgx}]$ e $\mu_y = [R_{avgy}, G_{avgy}, B_{avgy}]$. È possibile dimostrare matematicamente che esiste una costante c tale per cui la distanza euclidea tra le medie è sempre inferiore o uguale alla complessa distanza quadratica tra gli istogrammi:

$$d_{Euc}^2(\mu_x, \mu_y) \leq c \cdot D_{quad}^2(H_x, H_y)$$

Questa disuguaglianza ci permette di adottare un'architettura a due fasi:

1. **Filtro (Filter):** Si scansiona il database calcolando solo la velocissima distanza tra i momenti medi. Se questa distanza "sottostimata" è già superiore a una nostra soglia di tolleranza ϵ , possiamo scartare l'immagine con assoluta certezza matematiche (Zero Falsi Negativi o *No False Dismissals*).
2. **Raffinamento (Refine):** Si recupera il piccolo sottoinsieme di immagini "sopravvissute" al filtro e solo su queste si calcola la pesante distanza in forma quadratica per stilare il ranking definitivo.

Questo stratagemma abbatta drasticamente i tempi di calcolo garantendo esattamente gli stessi risultati di una scansione quadratica completa.

3.7.7 Texture

La texture (o trama) descrive la presenza di pattern regolari o quasi-regolari che si ripetono all'interno di una regione dell'immagine. A differenza del colore, che è una proprietà puntuale del singolo pixel, la texture è una **proprietà spaziale e di vicinato**: non ha senso calcolare la texture di un singolo pixel isolato.

Esistono molteplici proprietà percettive per descrivere le texture, tra cui:

- Livello di grigio (luminosità media del pattern)
- Forma degli elementi base (texton)
- Omogeneità e Levigatezza

- Finezza e Granulosità
- Ruvidezza
- Regolarità e Periodicità
- Linearità e Direzionalità

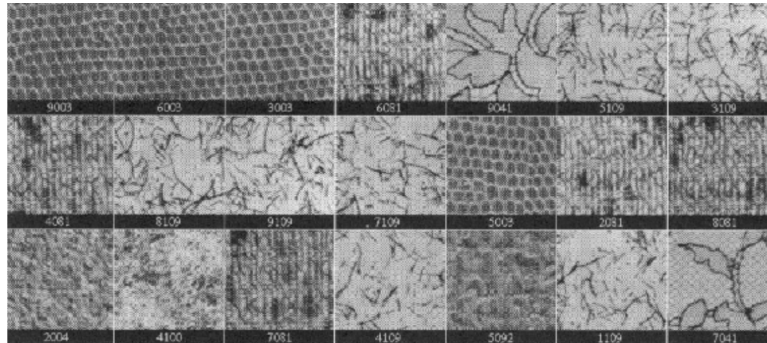


Figura 31: Esempi di texture con differenti gradi di direzionalità, ruvidezza e regolarità.

Estrazione dei Gradienti (Filtri di Sobel) Per analizzare la texture, il primo passo è tipicamente la conversione dell'immagine in **scala di grigi**, poiché l'informazione strutturale risiede nei contrasti di luminosità (luma) piuttosto che nelle variazioni cromatiche.

L'obiettivo è misurare la **direzione** e l'**intensità** (magnitudo) della variazione di luminosità per ogni pixel rispetto ai suoi vicini. Questo si ottiene calcolando il gradiente spaziale dell'immagine tramite l'operazione di **convoluzione**. La convoluzione applica una trasformazione lineare locale utilizzando una matrice di pesi detta *kernel* o *filtro*, facendola scorrere pixel per pixel.

I filtri più utilizzati per questo scopo sono i **filtri di Sobel**, che approssimano le derivate parziali dell'immagine lungo l'asse orizzontale (x) e verticale (y):

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} \quad G_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

Il filtro G_x calcola il gradiente orizzontale (enfaticizzando i bordi verticali), assegnando un peso maggiore ai pixel immediatamente a destra (+2) e a sinistra (-2) del pixel centrale. Analogamente, G_y calcola il gradiente verticale.

Per ogni pixel, le due convoluzioni restituiscono due valori scalari, δ_x e δ_y . Da questi, calcoliamo le due componenti fondamentali del gradiente:

- **Intensità (Magnitudo)** della variazione:

$$|\nabla| = \sqrt{\delta_x^2 + \delta_y^2}$$

- **Direzione** della variazione (fase):

$$\theta = \arctan\left(\frac{\delta_y}{\delta_x}\right)$$

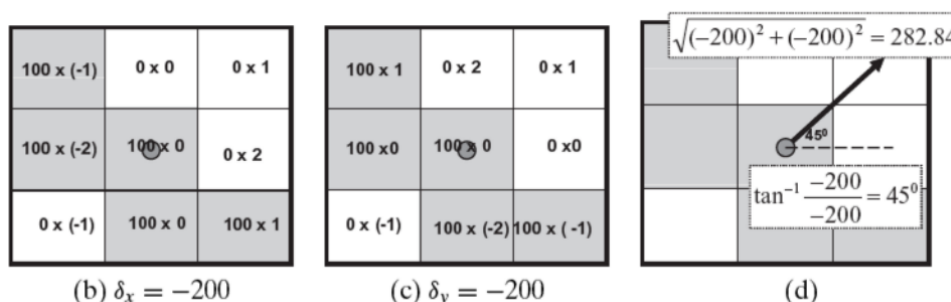


Figura 32: Esempio di estrazione dei gradienti di un'immagine in scala di grigi utilizzando i filtri di Sobel.

Istogramma delle Texture e dei Gradienti A differenza dell'istogramma dei colori, dove i "bin" sono i colori stessi, qui abbiamo valori continui (reali) per magnitudo e fase. Per costruire una rappresentazione vettoriale sintetica, dobbiamo **quantizzare** queste misure.

Esistono diversi approcci per costruire istogrammi basati su queste informazioni spaziali:

1. **Histogram of Oriented Gradients (HOG)**: Si discretizza lo spazio degli angoli θ (es. 8 bin da 0° a 180°). Ogni pixel "vota" per il bin corrispondente alla sua direzione θ , e il peso del voto è proporzionale alla sua intensità $|\nabla|$. Questo crea un istogramma che descrive in quali direzioni si concentrano i bordi o i pattern della texture.
2. **Matrice di Co-occorrenza (GLCM)**: Cattura la relazione spaziale esatta tra coppie di pixel. Si fissa una distanza d e un angolo α , e si conta quante volte un pixel con intensità di grigio i è adiacente a un pixel di intensità j lungo quella direzione. È concettualmente un istogramma multidimensionale molto potente per texture complesse, ma computazionalmente oneroso.

Nota sull'efficienza: mentre l'istogramma dei colori si calcola con un singolo passaggio lineare $O(N)$, l'estrazione delle feature di texture richiede operazioni di convoluzione e spesso l'analisi di vicinati complessi, portando a costi computazionali significativamente maggiori.

3.8 Segmentazione e Riconoscimento delle Forme

Estrazione dei Bordi (Edge Detection) I bordi rappresentano i confini fisici o semantici tra oggetti diversi o tra un oggetto e lo sfondo. Matematicamente, un bordo corrisponde a una regione in cui il gradiente dell'immagine (calcolato ad esempio con Sobel o Canny) presenta un picco o un forte contrasto locale.

Una volta individuati i pixel appartenenti ai bordi, l'obiettivo è aggregarli in strutture coerenti (forme, linee, segmenti) da analizzare e confrontare. Possiamo procedere secondo due filosofie:

- **Metodo Analitico (Parametrico)**: Ricerca di forme geometriche note (rette, cerchi, ellissi) sfruttando le loro equazioni matematiche.
- **Metodo Descrittivo (Non Parametrico)**: Rappresentazione della forma esatta del contorno così come si presenta, indipendentemente dalla sua regolarità.

Trasformata di Hough (Hough Transform) La Trasformata di Hough è l'algoritmo classico per il **metodo analitico**. Dato un insieme di pixel sparsi che descrivono un potenziale bordo, l'obiettivo è trovare la forma geometrica (es. la retta) che passa per il maggior numero di essi, mitigando rumore e interruzioni del tratto.

Data l'equazione della retta $y = mx + q$, un singolo pixel di coordinate note (x_i, y_i) vincola i parametri m e q secondo la relazione:

$$q = y_i - x_i m$$

Questo significa che un *punto* nello spazio dell'immagine (x, y) corrisponde a una *retta* nello spazio dei parametri (m, q) . Per trovare la retta migliore, si utilizza uno **spazio di accumulazione** (una matrice di voto):

1. Per ogni punto del bordo (x_i, y_i) , si calcolano tutti i possibili valori di q al variare di m .
2. Si incrementa di 1 la cella della matrice corrispondente alla coppia (m, q) .
3. Il picco massimo nella matrice (la cella con più voti) indica i parametri (m, q) della retta che interpola il maggior numero di punti nell'immagine.

Problema di cardinalità: La pendenza m può tendere a infinito (per rette verticali), rendendo impossibile creare una matrice finita. Per questo, si utilizza la **forma polare** della retta:

$$\rho = x \cos \theta + y \sin \theta$$

dove ρ è la distanza della retta dall'origine e θ è l'angolo del vettore normale. Poiché $\theta \in [0, 180^\circ]$ e $\rho \in [-D, D]$ (con D diagonale dell'immagine), lo spazio dei parametri è chiuso e limitato, rendendo la matrice di voto computabile.

3.8.1 Segmentazione basata su Cluster (K-Means)

La segmentazione divide l'immagine in regioni internamente coerenti. L'approccio per clustering considera ogni pixel come un punto in uno spazio multi-dimensionale (es. uno spazio a 5 dimensioni composto da: coordinata X , coordinata Y , e i 3 canali colore R, G, B). Applicando algoritmi come il K-Means, i pixel vengono raggruppati in base alla loro vicinanza spaziale e cromatica simultaneamente, generando segmenti compatti e dal colore omogeneo.

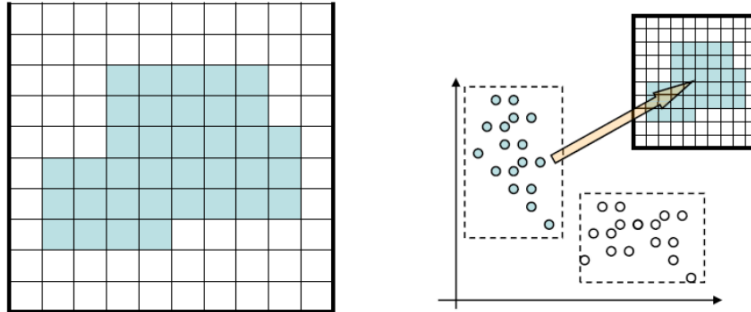


Figura 33: Esempio di segmentazione basata su clustering, dove pixel spazialmente vicini e cromaticamente simili vengono raggruppati.

3.8.2 Region-Growing

È un approccio *bottom-up*. Si parte dalla scelta di uno o più pixel "seme" (seed points). L'algoritmo esamina i pixel adiacenti al seme: se soddisfano un determinato **criterio di omogeneità** (es. la differenza di colore o di grigio è inferiore a una soglia ϵ), vengono annessi alla regione. Il processo continua iterativamente e la regione si "espande" finché non si incontrano pixel che violano il criterio (es. bordi netti).

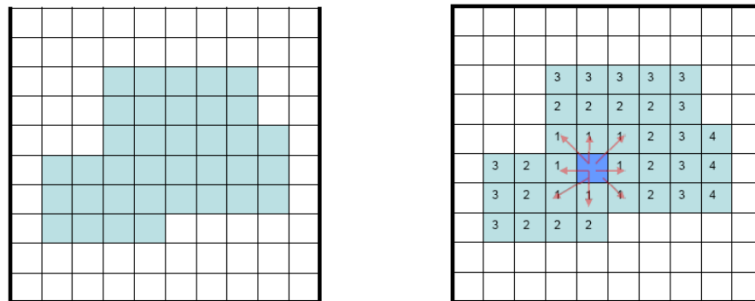


Figura 34: Esempio di segmentazione basata sul region-growing a partire da pixel seme.

3.8.3 Watershed (Spartiacque)

Il metodo Watershed tratta l'immagine dei **gradienti** come se fosse una mappa topografica tridimensionale. In questa analogia geometrica:

- **Valli (bacini):** Regioni in cui il gradiente è vicino a zero, corrispondenti alle zone piatte e omogenee dell'immagine originale.
- **Rilievi (montagne):** Pixel con gradiente elevato, corrispondenti ai bordi tra gli oggetti.

Si immagina di "allagare" questa topografia facendo sgorgare acqua dai punti più bassi (i minimi locali del gradiente). Man mano che l'acqua sale, i bacini adiacenti rischiano di fondersi. Per impedirlo, si costruiscono delle "dighe" lungo i crinali montuosi. Al termine dell'allagamento, le dighe create rappresentano i confini esatti dei segmenti (le linee di spartiacque).

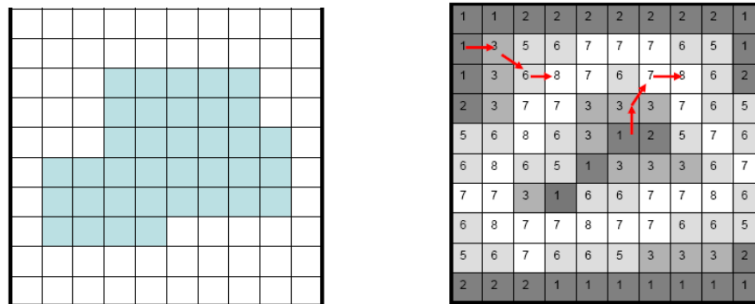


Figura 35: Esempio di segmentazione basata su watershed, con individuazione delle linee di spartiacque sui bordi.

3.9 Rappresentazione e Descrittori di Forma

Una volta isolata una forma (un segmento o un oggetto), dobbiamo tradurla in una feature vettoriale per poterla confrontare con altre forme nel database.

3.9.1 Rappresentazione Bitmap-based (Rigida)

La forma è rappresentata mediante una matrice binaria delle stesse dimensioni dell'immagine originale, dove i pixel appartenenti all'oggetto valgono 1 e quelli dello sfondo valgono 0. Il confronto tra due forme avviene misurando l'area di sovrapposizione (es. tramite un'operazione logica XOR).

Limiti: È una rappresentazione estremamente fragile.

- **Rotazione:** Fortemente sensibile. Ruotare l'oggetto genera una matrice completamente diversa.
- **Traslazione:** Fortemente sensibile. Spostare l'oggetto cambia le coordinate degli 1.
- **Scalatura:** Fortemente sensibile. Ingrandire l'oggetto altera il conteggio e la posizione dei pixel attivi.

0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	1	1	0	1	1	0	0
0	0	1	1	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	0	1
1	1	0	0	0	0	1	1	1	0
0	1	1	1	1	1	1	0	0	0
0	0	1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Figura 36: Esempio di segmentazione e mascheratura basata su bitmap binaria.

3.9.2 Chain-Code (Codice a Catena di Freeman)

È un descrittore basato esclusivamente sul **contorno** dell'oggetto. A partire da un pixel di inizio (spesso scelto in modo arbitrario), si percorre il perimetro dell'oggetto registrando ad ogni passo la **direzione** intrapresa per raggiungere il pixel di contorno successivo. Le direzioni sono quantizzate in un intorno discreto (tipicamente a 4 o 8 connessioni, con valori da 0 a 7). La forma viene così compressa in una **stringa numerica** monodimensionale (es. 0007766544...).

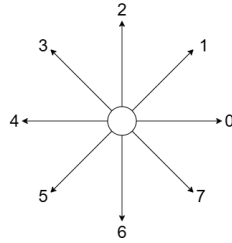


Figura 37: Codifica a 8 direzioni per la generazione del Chain-code.

Proprietà di invarianza:

- **Traslazione: Invariante.** Poiché si registrano solo gli spostamenti *relativi* da un pixel all'altro e non le coordinate spaziali assolute, traslare l'oggetto non altera in alcun modo la stringa risultante.
- **Scalatura: Sensibile.** Un oggetto più grande genererà un contorno con più pixel e quindi una stringa più lunga (es. un segmento orizzontale lungo 2 pixel è codificato come "00", uno lungo 4 pixel come "0000"). Può essere reso invariante tramite tecniche di ricampionamento spaziale del contorno a lunghezza fissa.
- **Rotazione: Sensibile** nella sua forma base, poiché ruotando l'oggetto cambiano le direzioni assolute. Tuttavia, calcolando la **derivata prima** del chain-code (ovvero registrando le variazioni di direzione tra un passo e il successivo, anziché la direzione assoluta) e allineando la stringa in modo che formi il numero intero di valore minimo, la rappresentazione diventa invariante alla rotazione per step discreti (es. multipli di 45°).

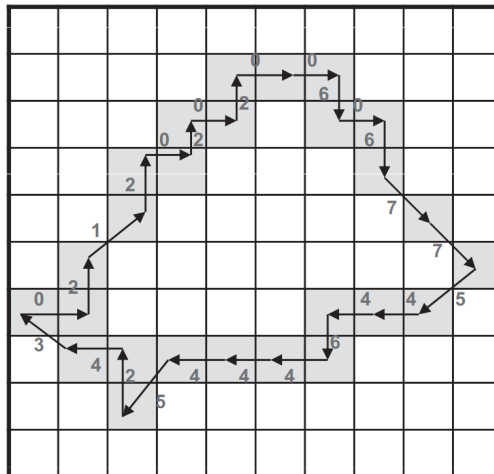


Figura 38: Esempio di estrazione del contorno e rappresentazione tramite chain-code.

3.9.3 Approssimazione Poligonale (Rappresentazione vettoriale)

Una versione alternativa per descrivere i contorni si basa sull'idea di approssimare la forma vedendola come una concatenazione di segmenti di retta (poligonale). L'algoritmo valuta la distanza tra il contorno reale e il segmento che ne unisce gli estremi: se per tutti i pixel questa distanza (l'errore) è inferiore a una soglia prestabilita, la forma viene approssimata con una singola linea retta. Se l'errore supera la soglia, si inserisce un nuovo punto di controllo intermedio (vertice) e si divide l'approssimazione in due o più segmenti (tecnica di *split-and-merge*).

Proprietà di invarianza:

- **Traslazione: Invariante.** La posizione assoluta dei vertici non è vincolante, conta solo la loro disposizione relativa (lunghezze e angoli interni).
- **Rotazione e Scalatura:** Di base **sensibili** se guardiamo le coordinate, ma le variazioni sono facilmente **calcolabili e normalizzabili**. Se in due forme i segmenti mantengono le stesse proporzioni

relative di lunghezza e gli stessi angoli interni, possiamo dedurre matematicamente che si tratta di versioni ruotate o scalate dello stesso oggetto.

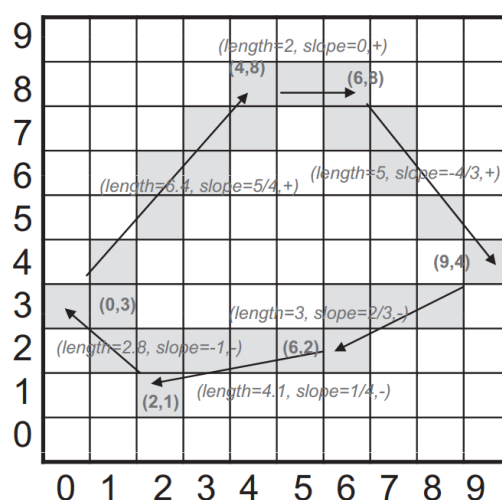


Figura 39: Esempio di approssimazione del contorno tramite vettori poligonal.

3.9.4 Time-Series (Firma Centroidale / Centroid Distance Signature)

Questo metodo riduce drasticamente la dimensionalità trasformando il contorno bidimensionale di una forma in un segnale monodimensionale periodico (una serie temporale).

Il processo funziona così:

1. Si calcola il baricentro spaziale (centroide) dell'oggetto, che avrà coordinate (x_c, y_c) .
2. Si fa "spazzare" un raggio dal centroide verso il contorno per un angolo θ che va da 0 a 360° (2π).
3. Per ogni angolo θ , si misura la distanza euclidea $d(\theta)$ tra il centroide e il punto di intersezione sul contorno.
4. Si riporta questa misurazione su un grafico cartesiano, dove l'asse X è l'angolo θ e l'asse Y è la distanza d .

Il risultato è un'onda continua che costituisce una vera e propria **impronta digitale** del contorno dell'oggetto.

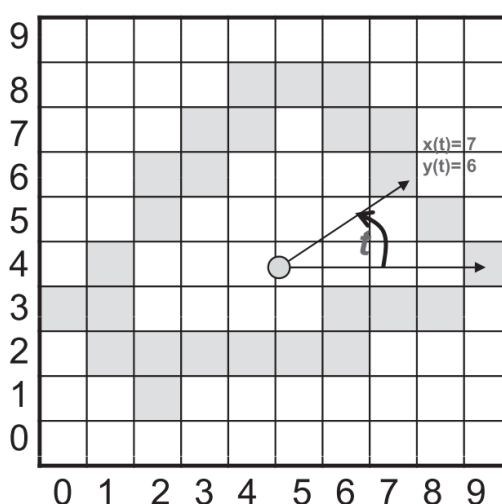


Figura 40: Esempio di "srotolamento" di un contorno in una time-series: asse X l'angolo, asse Y la distanza dal centroide.

Proprietà geometriche:

- **Traslazione: Invariante.** Poiché il sistema di riferimento è ancorato al centroide (che è una proprietà intrinseca della forma) e non al centro dell'immagine, spostare l'oggetto non altera le distanze misurate dal suo baricentro.
- **Scalatura: Sensibile.** Ingrandendo l'immagine, la distanza $d(\theta)$ aumenta, alterando l'ampiezza dell'onda. Si rende invariante normalizzando tutte le distanze (es. dividendo per la distanza massima rilevata).
- **Rotazione: Sensibile.** Ruotando l'oggetto, l'onda trasla orizzontalmente sull'asse X, poiché cambia il punto in cui $\theta = 0$. Si risolve allineando l'angolo 0 all'asse maggiore dell'oggetto, oppure analizzando il segnale nel dominio delle frequenze (Descrittori di Fourier).

3.10 Rappresentazione delle forme

3.10.1 Local Binary Patterns (LBP)

È un descrittore di texture molto efficiente. L'idea alla base è descrivere ogni singolo pixel analizzando il contesto locale che lo circonda. Si prende un pixel centrale e si esamina la matrice 3×3 di pixel adiacenti. Il valore di intensità del pixel centrale funge da **soglia**. Per ognuno degli 8 pixel vicini, si confronta il suo valore con quello centrale: se è maggiore o uguale si assegna il bit '1', altrimenti '0'. Leggendo questi 8 bit in senso orario (o antiorario) si ottiene una stringa binaria che viene convertita in un numero decimale (da 0 a 255), il quale andrà a sostituire il pixel centrale originale.

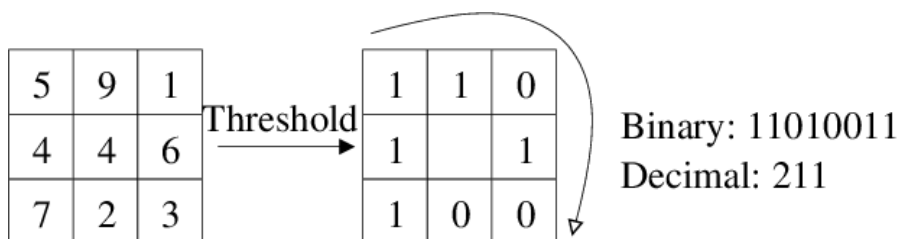


Figura 41: Esempio di calcolo del Local Binary Pattern base su intorno 3x3.

Questo approccio può essere generalizzato definendo intorni circolari di raggio R con un numero P di pixel campionati, permettendo di analizzare texture a diverse scale.

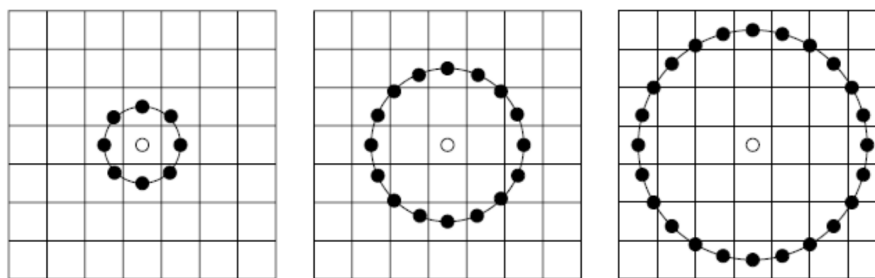


Figura 42: Esempio di LBP esteso ad intorni circolari più grandi.

3.10.2 Histogram of Oriented Gradients (HOG)

Il descrittore HOG cattura la distribuzione delle direzioni dei gradienti (ovvero i bordi e le forme locali) all'interno di un'immagine. L'immagine viene suddivisa in piccole porzioni connesse chiamate **celle** (es. griglie di pixel 8×8). Per ogni pixel all'interno della cella si calcola l'ampiezza (magnitudo) e la direzione del gradiente. I gradienti quantizzati vengono poi raggruppati in un istogramma (solitamente a 9 bin direzionali, da 0 a 180 gradi). Ogni pixel "vota" per una specifica direzione, e il peso del suo voto è proporzionale all'ampiezza del suo gradiente. Infine, per garantire robustezza ai cambiamenti di illuminazione, le celle adiacenti vengono raggruppate in **blocchi** più grandi per normalizzare i valori degli istogrammi.

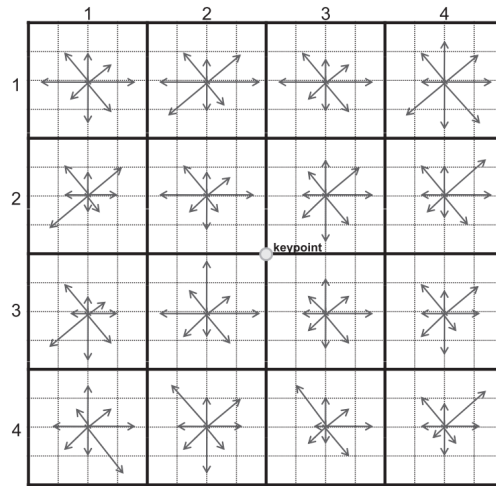


Figura 43: Estrazione delle feature tramite Histogram of Oriented Gradients.

3.11 Features locali

Le feature locali consistono nel descrivere un'immagine non nella sua totalità, ma isolando una collezione ristretta di punti "chiave" altamente informativi e distintivi (es. spigoli, angoli o pattern unici).

L'obiettivo principale di un buon estrattore di feature locali (come SIFT o SURF) è individuare punti che siano **robusti**, ovvero identificabili in modo affidabile anche in presenza di:

- Variazioni di illuminazione e contrasto;
- Cambiamenti di scala (zoom);
- Rotazioni dell'immagine;
- Alterazioni del punto di vista (prospettiva affine);
- Presenza di rumore o occlusione parziale.

3.11.1 SIFT (Scale-Invariant Feature Transform)

L'algoritmo SIFT è uno strumento fondamentale per la *computer vision*, poiché permette di estrarre caratteristiche (feature) locali da un'immagine, in modo da poter riconoscere oggetti o parti comuni in fotografie diverse, indipendentemente da scala, rotazione o illuminazione. A differenza delle feature globali (come l'istogramma dei colori), SIFT si concentra su punti di interesse specifici e altamente riconoscibili, detti *keypoint*. Ad esempio, se abbiamo due foto della stessa casa scattate da distanze diverse, i colori globali potrebbero variare molto, ma un *keypoint* corrispondente alla punta del tetto rimarrà riconoscibile.

L'algoritmo si struttura in quattro passi principali:

1. **Scale-space extrema detection:** ricerca dei punti di interesse lavorando a diverse scale di sfocatura.
2. **Keypoint localization:** filtraggio dei punti individuati per mantenere solo i più robusti, eliminando il rumore.
3. **Orientation assignment:** normalizzazione dell'orientamento per garantire l'invarianza alla rotazione.
4. **Keypoint descriptor computation:** creazione dei vettori descrittivi per l'identificazione univoca dei *keypoint*.

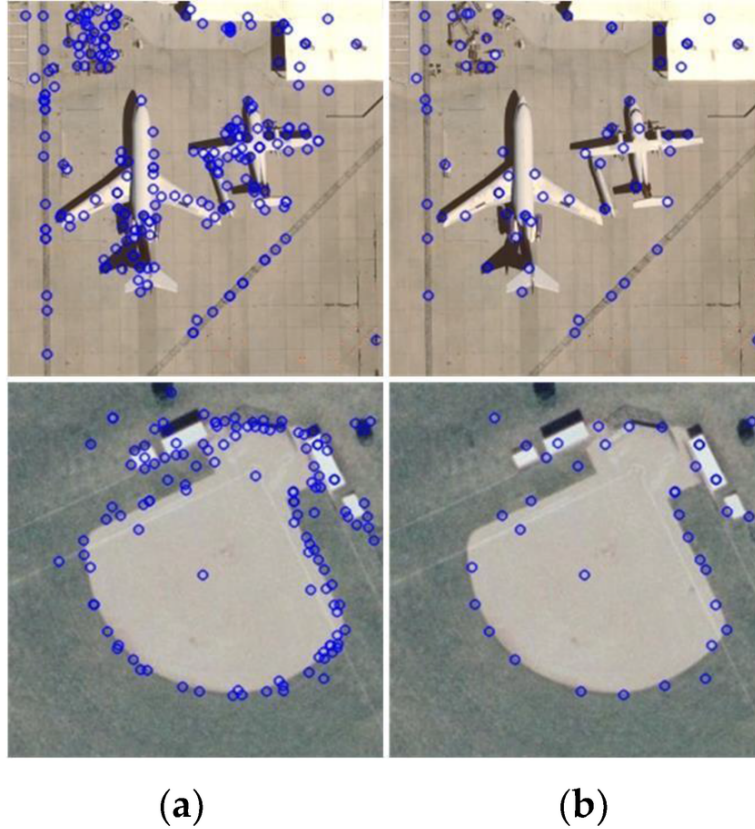


Figura 44: (a) Feature originali SIFT (Scale-Invariant Feature Transform) estratte dalle immagini (b) risultati della selezione dei keypoint

1. Creazione dello spazio scalato (Scale-space extrema detection) Il nostro primo obiettivo è riconoscere le stesse forme a prescindere dalla loro grandezza originaria. Per farlo, l'immagine originale viene progressivamente sfocata (operazione detta *blur* o *smoothing*) utilizzando un filtro Gaussiano.

Definiamo il filtro Gaussiano come:

$$G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{\sigma^2}}$$

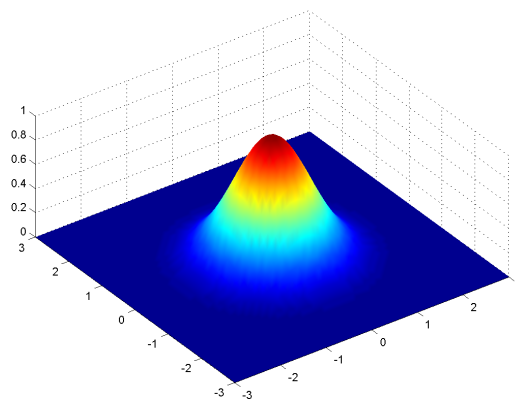


Figura 45: Rappresentazione della campana bidimensionale del filtro Gaussiano

Come funziona in pratica? Immagina questa funzione come una "campana" tridimensionale centrata sul pixel in esame. Il filtro calcola una media pesata dei pixel circostanti: i pixel più vicini al centro hanno un peso molto alto, mentre quelli più distanti (o ai margini della campana) pesano via via di meno. Il parametro σ (sigma) determina la larghezza di questa campana: aumentandolo, la sfocatura diventa più intensa. L'applicazione di questo filtro avviene tramite convoluzione su tutti i pixel:

$$L(x, y, \sigma) = G(x, y, \sigma) \otimes I(x, y)$$

Questo processo genera una lista di immagini sempre più sfocate (a passi $k\sigma$). Per trovare i potenziali punti salienti, si calcola la differenza tra coppie di sfocature adiacenti (metodo noto come *Difference of Gaussians* o DoG):

$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma)$$

Analizzando queste immagini "differenza", ogni pixel viene confrontato con i suoi 26 vicini: 8 pixel adiacenti nella stessa scala, 9 nella scala superiore e 9 nella scala inferiore. Se il pixel risulta essere un massimo o un minimo locale assoluto rispetto a tutti i 26 vicini, denota un forte cambio di gradiente e diventa un potenziale *keypoint*.

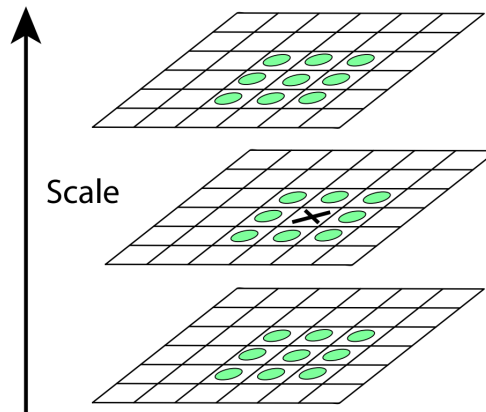


Figura 46: Confronto con i 26 vicini per identificare i potenziali *keypoint*

2. Localizzazione dei punti chiave (Keypoint localization) In questa fase si "pulisce" l'insieme dei candidati, scartando quelli poco informativi. Vengono eliminati due tipi di punti:

- **Punti a basso contrasto:** considerati come semplice rumore dell'immagine.
- **Punti mal localizzati (es. sui bordi):** un punto situato nel mezzo di una linea retta o di un bordo uniforme non porta informazioni distintive, poiché tutti i punti vicini avranno una variazione di gradiente identica e allineata. Al contrario, si conservano i punti vicini a spigoli o vertici (come gli estremi di una retta), dove il gradiente cambia in più direzioni e il punto diventa univocamente rintracciabile.

3. Assegnazione dell'orientamento (Orientation assignment) Per far sì che l'algoritmo riconosca la nostra "casetta" anche se la foto è capovolta, dobbiamo rendere i *keypoint* invarianti alla rotazione. Per ogni *keypoint* isolato, si calcola un istogramma delle direzioni dei gradienti nella regione adiacente, utilizzando la scala σ in cui il punto è stato rilevato. Anche qui si applica una finestra Gaussiana (con parametro 1.5σ) per far sì che i gradienti dei pixel più vicini al *keypoint* abbiano un "peso" maggiore nel calcolo.

La direzione che rappresenta il picco massimo dell'istogramma diventa l'orientamento ufficiale di quel *keypoint*. Se ci sono più picchi rilevanti, il *keypoint* viene duplicato: si mantengono le stesse coordinate, ma si assegnano le diverse orientazioni trovate, per non perdere nessuna potenziale corrispondenza.

4. Calcolo del descrittore (Keypoint Descriptor Computation) L'ultimo passaggio serve a trasformare le informazioni visive in dati numerici (array) facilmente confrontabili dal computer. Si seleziona una regione quadrata di pixel attorno al *keypoint* (tipicamente 16×16) e la si allinea seguendo l'orientamento assegnato al passo 3, "raddrizzando" di fatto l'area analizzata.

Quest'area viene suddivisa in sotto-griglie (generalmente 4×4). Per ciascuna di queste piccole zone, si raggruppano i vettori dei gradienti calcolando un piccolo istogramma delle direzioni. L'insieme di tutti questi istogrammi forma il **descrittore del keypoint**. Ad esempio, partendo da un'area 8×8 , potremmo ottenere un descrittore riassunto in una matrice 2×2 di gradienti. Grazie a questo array, se in un'altra

immagine si rileva uno spigolo simile, il computer confronterà i due descrittori matematicamente per confermare che si tratta dello stesso punto.

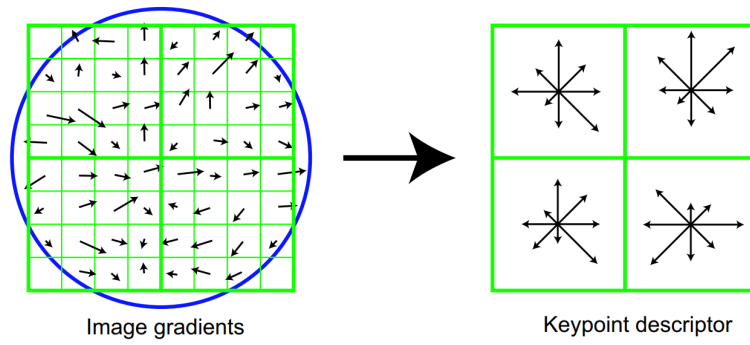


Figura 47: Generazione del descrittore per un keypoint

4 Riduzione della Dimensionalità

Dato un database $D = \{o_1, \dots, o_n\}$ di n oggetti, originariamente descritti in un insieme di m features $F = \{f_1, \dots, f_m\}$, l'obiettivo della riduzione della dimensionalità è identificare un nuovo set di **feature latenti** $Z = \{z_1, \dots, z_k\}$, tali che $k \ll m$.

Vogliamo ottenere un database trasformato $D' = \{o'_1, \dots, o'_n\}$ che mantenga il più possibile le informazioni originali, tale per cui la struttura spaziale e le distanze tra gli oggetti non vengano stravolte:

$$D \approx D'$$

I vantaggi principali di questa operazione sono:

- **Efficienza computazionale:** algoritmi più veloci e minor occupazione di memoria.
- **Riduzione del rumore:** eliminando le features meno rilevanti, si migliora spesso la qualità dei dati.
- **Visualizzazione:** se $k \leq 3$, è possibile visualizzare i dati per analisi esplorative.

4.1 Criteri di Trasformazione e Scelta

Per passare dallo spazio originale a quello ridotto, possiamo adottare diversi approcci:

1. Selezione delle feature (Feature Selection) Invece di creare nuove feature, ci limitiamo a scegliere un sottoinsieme di quelle originali, scartando quelle ridondanti o irrilevanti.

$$Z \subseteq F$$

2. Rappresentazione basata su oggetti prototipo (Landmarking) In questo approccio, usiamo alcuni oggetti rappresentativi del database stesso per descrivere tutti gli altri.

$$Z \subseteq D$$

Ad esempio, supponiamo di dover descrivere vari agrumi. Invece di usare le proprietà fisiche (peso, acidità, spessore della buccia), scelgo dei "prototipi" come il limone, l'arancio e il cedro. A questo punto, descrivo un *limone Meyer* non tramite le sue feature chimiche, ma come "incrocio" (o distanza/similarità) tra un limone standard e un arancio. Allo stesso modo in pasticceria: posso descrivere la torta mimosa come una combinazione vettoriale delle sue basi (pan di spagna e crema chantilly). Le "features" diventano quindi le similarità rispetto a questi oggetti di riferimento.

3. Trasformazioni Orto-normali (Feature Extraction lineare) Invece di scartare feature, le combiniamo linearmente. Troviamo una trasformazione θ che, applicata alle features di partenza, restituisce nuove feature (feature latenti) che sono tra loro **orto-normali** (cioè vettori ortogonali tra loro e di lunghezza unitaria). Queste nuove direzioni vengono derivate analizzando la distribuzione dei dati nel database, in modo da catturare la massima varianza.

4. Trasformazioni Arbitrarie Comprendono tecniche non lineari (come t-SNE, UMAP o Autoencoder neurali) che mappano i dati in uno spazio a dimensionalità ridotta distorcendo lo spazio in modo complesso, utili soprattutto quando le relazioni tra le feature originali non sono lineari.

4.2 Trasformazioni Orto-normali e Proiezioni

L'idea alla base delle trasformazioni orto-normali (come la PCA - Principal Component Analysis) è trovare un nuovo sistema di assi cartesiani. Questi assi:

- Sono orientati verso le direzioni in cui i dati "variano" di più (la varianza corrisponde al potere discriminante).

- Sono ortogonali tra loro, il che garantisce che le nuove feature siano linearmente indipendenti (nessuna ridondanza).

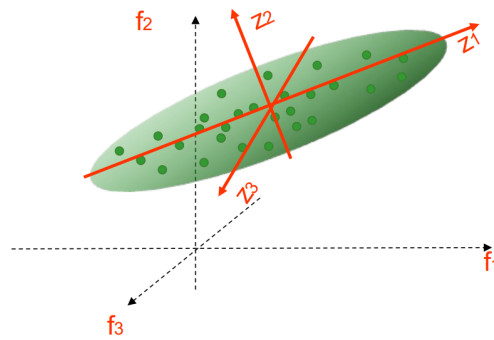


Figura 48: Esempio di trasformazione orto-normale

Fase 1: Trasformazione (Rotazione)

Se calcoliamo tutti gli m nuovi assi e "ruotiamo" il sistema di riferimento, stiamo solo descrivendo gli stessi oggetti da un altro punto di vista. Non abbiamo perso alcuna informazione, abbiamo solo cambiato le coordinate. In questo caso, le distanze tra gli oggetti e gli angoli rimangono **esattamente invariati**.

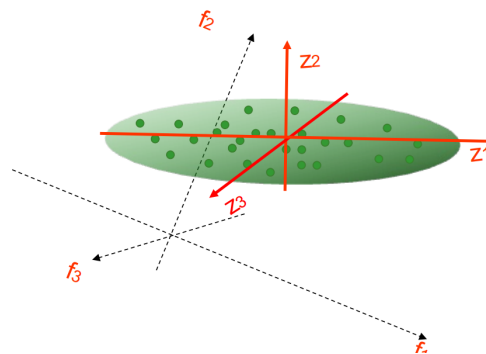


Figura 49: Esempio di rotazione del sistema di riferimento

Fase 2: Proiezione (Riduzione)

A questo punto, ordiniamo i nuovi assi dal più discriminante (maggiore varianza) al meno discriminante. Se scartiamo gli assi meno importanti (passando da m a k assi), effettuiamo una **proiezione**.

La proiezione comporta una perdita di informazione e, geometricamente, **riduce o mantiene inalterate le distanze** tra i punti, ma non le aumenta mai.

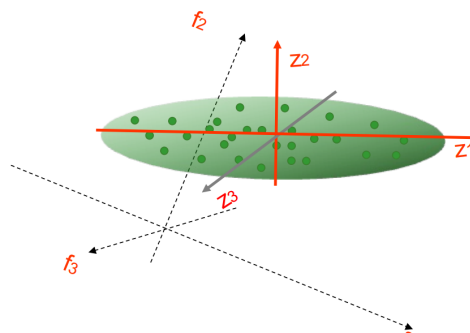


Figura 50: Esempio di riduzione della dimensionalità

4.2.1 Impatto sulle query (False Hit e Miss)

Se usiamo il database ridotto D' per fare delle ricerche (es. trovare tutti gli oggetti entro una distanza R da una query), il rimpicciolimento delle distanze dovuto alla proiezione genera due scenari rispetto a ciò che otterremmo nello spazio originale:

- **False hit (Falsi positivi):** Oggetti che nello spazio originale erano distanti (fuori dal raggio R), ma che nello spazio ridotto sembrano vicini e vengono restituiti dalla query.
- **Miss (Falsi negativi):** Oggetti che nello spazio originale erano vicini (dentro il raggio R), ma che nello spazio ridotto risultano distanti e non vengono restituiti.

Conclusione fondamentale per i database: Le proiezioni ortogonali **non introducono mai Miss**. Poiché la proiezione può solo rimpicciolire le distanze, due punti che erano vicini originariamente lo saranno ancora di più (o in modo uguale) nello spazio ridotto.

Avere dei *false hit* è tollerabile: basta fare un post-filtraggio calcolando la distanza reale (nello spazio originale) solo per gli oggetti restituiti, scartando i falsi allarmi. Se avessimo dei *miss*, perderemmo risultati corretti senza saperlo, vanificando l'affidabilità del sistema.

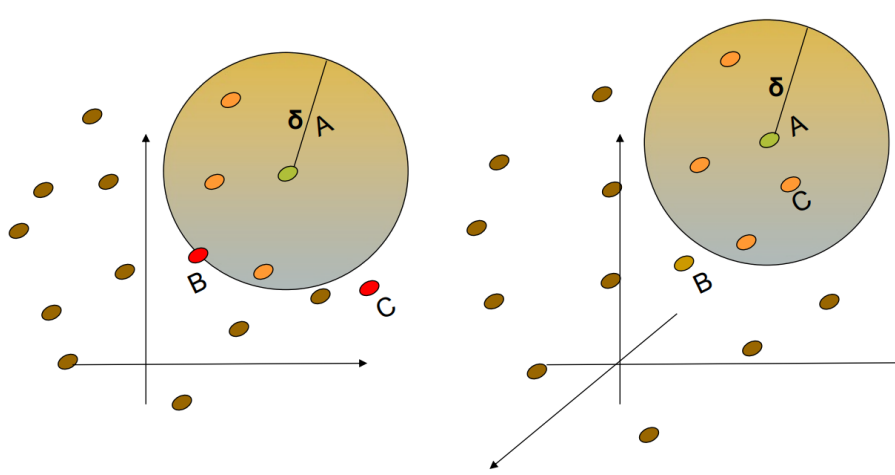


Figura 51: Esempio di query nello spazio ridotto con false hit e miss

4.3 Singular Value Decomposition (SVD)

Nota di correzione rispetto alla notazione standard: Assumiamo che il database sia rappresentato da una matrice D di dimensioni $n \times m$, dove le n righe sono gli oggetti e le m colonne sono le features originali.

A partire da D , possiamo definire due matrici di similarità cruciali tramite il prodotto scalare (dot product):

1.

$$D \times D^T$$

Matrice di dimensione $n \times n$, simmetrica. Rappresenta la **similarità tra gli oggetti** in base ai valori delle loro features.

2.

$$D^T \times D$$

Matrice di dimensione $m \times m$, simmetrica. Rappresenta la **similarità tra le features** (correlazione) rispetto a come si manifestano negli oggetti.

La decomposizione SVD afferma che qualsiasi matrice D può essere fattorizzata nel prodotto di tre matrici:

$$D = U \times S \times V^T$$

Il significato delle tre matrici:

- **U (Vettori singolari sinistri):** È una matrice $n \times k$. Le sue colonne sono gli **autovettori** della matrice DD^T . Rappresentano i concetti latenti rispetto agli oggetti. Essendo DD^T simmetrica e quadrata, l'esistenza di autovettori ortogonali è garantita.
- **V^T (Vettori singolari destri):** È una matrice $k \times m$ (le righe di V^T corrispondono alle colonne di V). Le sue righe sono gli **autovettori** della matrice $D^T D$. Rappresentano le combinazioni lineari delle feature originali che compongono le nuove direzioni latenti.
- **S (Matrice dei valori singolari - Core Matrix):** È una matrice diagonale $k \times k$. Sulla sua diagonale principale si trovano i **valori singolari** ordinati in modo decrescente ($s_1 \geq s_2 \geq \dots \geq s_k \geq 0$). Questi valori sono le radici quadrate degli **autovalori** (sia di DD^T che di $D^T D$, poiché condividono gli stessi autovalori non nulli).

Cos'è un autovettore e un autovalore? Data una matrice quadrata A , un vettore non nullo $\vec{x}$ è un **autovettore** se l'applicazione della matrice A ad $\vec{x}$ non ne cambia la direzione, ma ne scala solo la lunghezza. Formalmente:

$$A \times \vec{x} = \lambda \times \vec{x}$$

Dove λ è una costante chiamata **autovalore**. L'autovalore λ quantifica l'"importanza" o la "forza" di quella specifica direzione all'interno del dataset.

4.3.1 SVD Troncata e Approssimazione del Database

Il valore alla riga j , colonna j della matrice diagonale S ci dice esattamente qual è il **contributo energetico (o potere espressivo)** della j -esima feature latente alla ricostruzione del database originale.

Poiché i valori in S sono ordinati in modo decrescente, le prime colonne catturano la stragrande maggioranza dell'informazione, mentre le ultime spesso catturano solo rumore di fondo.

Se decidiamo di conservare solo le prime k' direzioni (con $k' \ll k \leq \min(n, m)$), possiamo "tagliare" le matrici scartando le ultime righe e colonne. Otteniamo così le matrici ridotte U' , S' e V'^T .

Ricostruendo il database con questi dati ridotti, otteniamo la migliore approssimazione possibile di grado k' :

$$D' = U' \times S' \times V'^T$$

In questo modo abbiamo ottenuto la riduzione della dimensionalità ricercata ($D' \approx D$), pur conservando la maggior parte del potere discriminante originario.

4.4 Principal Component Analysis (PCA)

La PCA (Analisi delle Componenti Principali) ha come obiettivo l'individuazione di **semantiche latenti** (componenti principali) che massimizzino la varianza dei dati, garantendo così il miglior criterio di differenziazione tra gli oggetti. Ci interessa, in sintesi, riconoscere e ordinare le nuove feature in base al loro **potere discriminante**.

Il processo parte dalla matrice di covarianza C , una matrice quadrata e simmetrica di dimensioni $m \times m$, che quantifica la covarianza tra tutte le coppie di feature originali.

Poiché C è simmetrica e definita semi-positiva, può essere decomposta tramite diagonalizzazione:

$$C = U \times S \times U^T$$

Dove:

- U è una matrice ortogonale $m \times m$ le cui colonne sono gli **autovettori** di C . Questa matrice definisce la trasformazione dalle m feature originali alle nuove feature latenti (le direzioni di massima varianza).

- S è una matrice diagonale $m \times m$ i cui elementi sulla diagonale principale sono gli **autovalori** λ_i ordinati in senso decrescente ($\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_m \geq 0$). L'autovalore λ_i quantifica l'**importanza** (la varianza spiegata) della i -esima feature latente.
- $U^\top$ è la trasposta di U , che permette la trasformazione inversa, descrivendo le componenti latenti in termini delle feature originali.

Possiamo ridurre la dimensionalità del dataset troncando le matrici, ovvero rimuovendo le componenti latenti meno importanti (quelle associate agli autovalori più piccoli, situate verso la fine della diagonale di S). Mantenendo solo i primi k autovettori, otteniamo un'approssimazione della matrice di covarianza originaria:

$$C' = U' \times S' \times (U')^\top$$

$$C' \approx C$$

4.4.1 Criteri per la scelta del numero di autovettori (k)

La scelta di quanti autovettori mantenere è cruciale per bilanciare la compressione dei dati e la perdita di informazioni. Esistono diversi metodi euristici e statistici:

1. Autovalore medio (Average Eigenvalue) Si calcola la media di tutti gli autovalori estratti. Vengono mantenuti solo gli autovettori il cui autovalore associato è maggiore o uguale a tale media. L'assunto è che una componente debba spiegare almeno la varianza "media" per essere considerata utile.

2. Regola di Kaiser (Kaiser Criterion) Si mantengono esclusivamente gli autovettori con un autovalore maggiore di 1 ($\lambda > 1$). Questo criterio si applica tipicamente quando la PCA viene calcolata sulla matrice di correlazione (dati standardizzati). La logica è che una componente latente, per essere conservata, deve spiegare almeno la stessa quantità di varianza di una singola feature originale.

3. Analisi Parallela (Parallel Analysis) È uno dei metodi più robusti. Si generano dataset casuali con la stessa dimensionalità (stesso numero di oggetti e feature) del dataset originale. Si calcolano le matrici di covarianza (o correlazione) di questi dati casuali e se ne estraggono gli autovalori. Infine, si confrontano gli autovalori reali con la media degli autovalori casuali. Si mantengono solo le componenti originali il cui autovalore supera quello corrispondente generato dal caso puro.

4. Scree Test È un approccio grafico. Si traccia un grafico (Scree Plot) in cui l'asse X rappresenta l'indice della componente (da 1 a m) e l'asse Y il valore del rispettivo autovalore. Si cerca visivamente un punto di flesso o di discontinuità evidente (il cosiddetto "gomito"). Le componenti a sinistra del gomito vengono mantenute, in quanto catturano la struttura primaria dei dati, mentre quelle a destra (che si appiattiscono) vengono scartate poiché rappresentano prevalentemente rumore.

5. Varianza spiegata cumulativa (Cumulative Variance Explained) Si fissa a priori una soglia percentuale di varianza totale che si desidera preservare (ad esempio, 80%, 90% o 95%). Si sommano gli autovalori in ordine decrescente fino a quando il loro rapporto rispetto alla somma totale degli autovalori raggiunge la percentuale desiderata.

4.5 Multi-Dimensional Scaling (MDS)

A differenza della PCA, che opera su oggetti già descritti da feature esplicite in uno spazio vettoriale, il Multi-Dimensional Scaling (MDS) interviene quando l'unica informazione disponibile è una matrice di **dissimilarità** o **distanza reciproca** tra gli oggetti (es. differenze comportamentali degli utenti, giudizi di similarità). Queste "distanze" potrebbero non essere nemmeno metriche rigorose (potrebbero non rispettare la disuguaglianza triangolare).

L'obiettivo del MDS è identificare un nuovo spazio vettoriale a bassa dimensionalità (k) in cui ogni oggetto sia rappresentato da un vettore di coordinate, in modo tale che le distanze euclidee calcolate in questo nuovo spazio siano **il più possibile proporzionali o simili** alle distanze (o dissimilarità) originarie. Il processo è intrinsecamente un problema di ottimizzazione iterativa.

La Funzione di Stress Per quantificare quanto la configurazione nello spazio ridotto approssimi bene le distanze originali, si utilizza una funzione di costo chiamata **Stress** (tipicamente la formula di Kruskal). Siano d_{ij} le distanze originali tra l'oggetto i e l'oggetto j , e siano d'_{ij} le distanze euclidee calcolate tra i rispettivi punti nel nuovo spazio vettoriale:

$$\text{Stress} = \sqrt{\frac{\sum_{i < j} (d_{ij} - d'_{ij})^2}{\sum_{i < j} d_{ij}^2}}$$

Il termine al numeratore calcola l'errore quadratico totale tra le distanze originali e quelle ricostruite. **Il fattore di scalatura al denominatore ($\sum d_{ij}^2$) funge da normalizzatore.** Questo passaggio è fondamentale per rendere lo Stress una metrica adimensionale e invariante rispetto alla scala complessiva delle distanze, restituendo un valore (spesso interpretabile in percentuale) che indica la bontà dell'adattamento (più lo stress è vicino a 0, migliore è la rappresentazione).

4.5.1 Il Processo di Ottimizzazione

L'algoritmo procede per tentativi, partendo tipicamente da una dimensionalità bassa per poi aumentarla se necessario. L'obiettivo è minimizzare la funzione di Stress.

Iterazione Interna (Ottimizzazione delle coordinate per una data dimensione k) Fissata una dimensionalità (es. $k = 1$ o $k = 2$), si assegnano coordinate casuali agli oggetti. Si calcola lo Stress iniziale e, tramite tecniche di discesa del gradiente, si spostano iterativamente i punti nello spazio per ridurre l'errore tra d_{ij} e d'_{ij} . Il ciclo si interrompe quando la funzione di Stress converge, ovvero quando ulteriori spostamenti dei punti producono miglioramenti marginali o nulli.

Iterazione Esterna (Aumento della dimensionalità) Se il valore di Stress finale ottenuto dall'iterazione interna è ancora troppo alto (superiore alla tolleranza desiderata), la dimensionalità dello spazio corrente non è sufficiente per mappare accuratamente la complessità delle distanze originali. L'algoritmo incrementa quindi la dimensionalità (passando a $k + 1$).

Quando si aumenta la dimensionalità, ci sono due strategie per inizializzare le nuove coordinate:

1. **Cold Start (Riavvio casuale):** Si generano coordinate completamente nuove e casuali per tutte le $k + 1$ dimensioni e si ripete l'ottimizzazione da zero. È utile se l'ottimizzazione precedente era rimasta bloccata in un minimo locale particolarmente sfavorevole.
2. **Warm Start (Mantenimento del progresso):** Si conservano le k coordinate già ottimizzate e si aggiunge solo una $(k + 1)$ -esima coordinata inizializzata casualmente (o a zero). Questa strategia accelera notevolmente la convergenza, capitalizzando il lavoro già fatto, a patto che la configurazione precedente costituisca una buona base di partenza.

Il processo globale termina quando si raggiunge uno Stress accettabile o quando l'aggiunta di ulteriori dimensioni non produce una diminuzione significativa dello Stress.

5 Tensori

Per modellare relazioni complesse, la rappresentazione vettoriale e matriciale (limitata a due dimensioni, come le classiche tabelle righe-colonne) non è sempre sufficiente.

Un **tensore** è la generalizzazione multidimensionale di scalari, vettori e matrici. Formalmente, uno scalare è un tensore di ordine 0, un vettore è un tensore di ordine 1, una matrice è un tensore di ordine 2 e un tensore vero e proprio (o *higher-order tensor*) ha ordine $N \geq 3$.

I tensori sono fondamentali per rappresentare dati multi-relazionali o multidimensionali, come ad esempio le interazioni nel tempo (es. Utente $\times$ Oggetto $\times$ Contesto Temporale) o triplette di un knowledge graph (Soggetto $\times$ Predicato $\times$ Oggetto).

5.1 Nomenclatura e Struttura

Ordine o Modo (Mode) L'ordine di un tensore è il numero delle sue dimensioni, spesso chiamate **modi**. Ad esempio, in un database di recensioni, il tensore $\mathcal{X}$ potrebbe avere tre modi: Utenti (Modo 1), Prodotti (Modo 2) e Tempo (Modo 3).

Fibre (Fibers) Le fibre sono l'equivalente tensoriale di righe e colonne in una matrice. Una fibra è un vettore unidimensionale (sotto-array) estratto da un tensore mantenendo fissi tutti gli indici tranne uno.

In un tensore a 3 dimensioni $\mathcal{X}$, abbiamo:

- **Fibre colonna (Modo-1):** Si ottengono fissando la seconda e la terza dimensione (variabile i , fissi j, k).
- **Fibre riga (Modo-2):** Si ottengono fissando la prima e la terza dimensione (fissi i, k , variabile j).
- **Fibre tubo (Modo-3):** Si ottengono fissando la prima e la seconda dimensione (fissi i, j , variabile k).

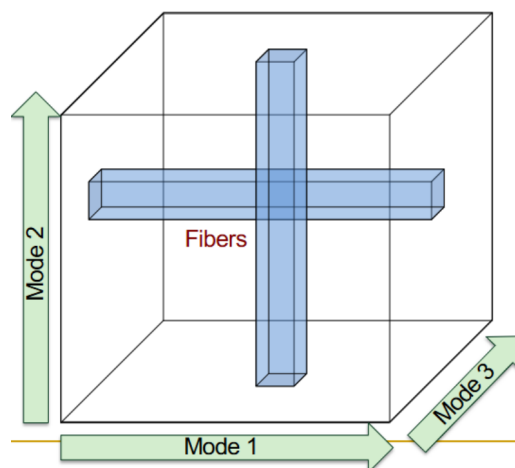


Figura 52: Esempio di fibre in un tensore a 3 modi

Fette (Slices) Le fette sono matrici bidimensionali (sotto-array) estratte da un tensore mantenendo fissi tutti gli indici tranne due.

In un tensore a 3 dimensioni, possiamo estrarre fette in tre direzioni:

- **Orizzontali:** Variano le dimensioni 1 e 2 (fisso il modo 3).
- **Laterali:** Variano le dimensioni 2 e 3 (fisso il modo 1).
- **Frontali:** Variano le dimensioni 1 e 3 (fisso il modo 2).

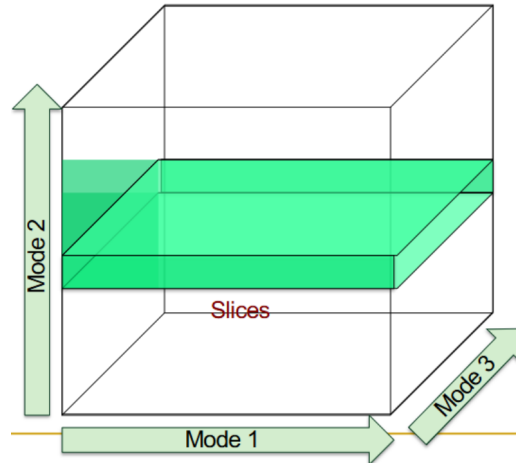


Figura 53: Esempio di fette in un tensore a 3 modi

Matricizzazione (Unfolding / Flattening) La matricizzazione è l'operazione che "appiattisce" un tensore trasformandolo in una matrice bidimensionale. Questo passaggio è cruciale per poter applicare gli algoritmi standard di algebra lineare sviluppati per le matrici (come l'SVD) anche ai tensori.

L'*unfolding* lungo il modo n , indicato con $X_{(n)}$, consiste nel prendere tutte le fibre del modo n e affiancarle per farle diventare le colonne della matrice risultante.

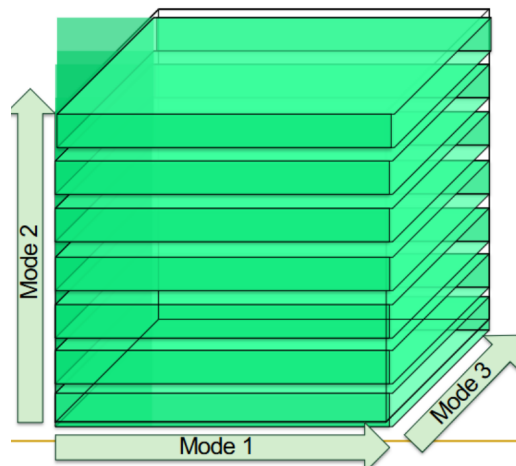


Figura 54: Esempio di matricizzazione in un tensore a 3 modi

5.2 CP Decomposition (CANDECOMP / PARAFAC)

La decomposizione CP fattorizza un tensore in una somma di tensori di **rango 1**. Un tensore a 3 modi di rango 1 è il prodotto esterno ($\circ$) di tre vettori.

Data una matrice a tre modi $\mathcal{X}$ di dimensioni $I_1 \times I_2 \times I_3$, la decomposizione CP mira ad approssimare $\mathcal{X}$ con R componenti latenti:

$$\mathcal{X} \approx \sum_{r=1}^R \lambda_r (\vec{a}_r \circ \vec{b}_r \circ \vec{c}_r)$$

Dove:

- R è il numero di componenti (o semantiche latenti) scelte.
- $\vec{a}_r, \vec{b}_r, \vec{c}_r$ sono i vettori colonna delle matrici dei fattori (chiamate A, B, C), corrispondenti a ciascuna modalità.
- λ_r è un peso che indica l'**importanza** della componente r -esima.

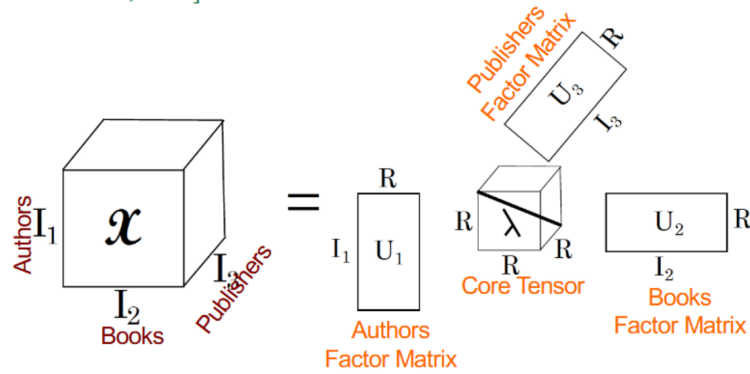


Figura 55: Esempio di decomposizione CP in un tensore a 3 modi

Il concetto del Core Tensor Super-Diagonale La decomposizione CP può essere vista come una forma speciale in cui esiste un **core tensor** (un piccolo tensore centrale di dimensioni $R \times R \times R$) strettamente diagonale. I valori sulla "super-diagonale" di questo cubetto corrispondono ai pesi λ_r . Come nella SVD, questi valori indicano quanto ciascun *cluster* o semantica latente sia significativo per la ricostruzione del tensore originale. Ordinando i valori in modo decrescente (da $(1, 1, 1)$ a (R, R, R)), si nota che le prime componenti catturano le relazioni più forti.

Proprietà e Ottimizzazione L'obiettivo pratico è capire come ciascun elemento delle entità di partenza (es. uno specifico utente o un particolare tag) partecipa a queste relazioni latenti (o cluster), analizzando i valori all'interno delle matrici fattorizzate A , B e C .

A differenza del rango di una matrice, calcolare il **rango esatto** di un tensore (il minimo numero R di componenti per una ricostruzione esatta) è un problema NP-hard.

Di conseguenza:

- R diventa un **parametro di input** (iperparametro) fornito dal ricercatore.
- La decomposizione avviene tramite metodi iterativi di ottimizzazione, come l'**Alternating Least Squares (ALS)**, in cui si fissa temporaneamente la stima di due matrici dei fattori per risolvere e ottimizzare la terza, ciclando finché l'errore di ricostruzione non converge al minimo.

Una cosa certa è che R (il numero di componenti latenti o rango) deve essere scelto in modo accurato. In genere, si imposta $R \ll \min(I_1, I_2, I_3)$ per ottenere una rappresentazione compressa del tensore originale, evitare l'overfitting e garantire una buona capacità di generalizzazione del modello.

Attraverso la decomposizione, il sistema apprende le semantiche latenti e fornisce una stima del tensore originale $\hat{\chi}$. Un aspetto cruciale è che, laddove nel tensore originale vi era un valore nullo o mancante (indicante una relazione non osservata), nel tensore ricostruito avremo un valore stimato. Questo valore rappresenta la forza predetta della relazione latente tra quelle entità. L'intero processo è alla base del funzionamento dei moderni sistemi di raccomandazione collaborativi multidimensionali.

5.3 Tucker Decomposition

Un'altra tecnica fondamentale è la **Tucker Decomposition**. La differenza principale rispetto alla decomposizione CP risiede nella flessibilità e nella struttura del *core tensor*. Nella CP, si impone lo stesso numero di componenti latenti R per ciascun modo (costruendo lo stesso numero di "cluster" per utenti, oggetti e contesti), e il core tensor risultante è strettamente diagonale (un cubo di dimensioni $R \times R \times R$).

Nella decomposizione di Tucker, invece, è possibile specificare un numero diverso di componenti per ciascuna modalità. Definiamo quindi i ranghi R_1, R_2 , e R_3 . Il core tensor $\mathcal{G}$ non è più un tensore diagonale e sparso, ma diventa un tensore **denso** di dimensioni $R_1 \times R_2 \times R_3$. I valori all'interno di questo core tensor (i "pesi") indicano il grado di interazione o l'importanza di ciascuna possibile combinazione di cluster tra le diverse modalità.

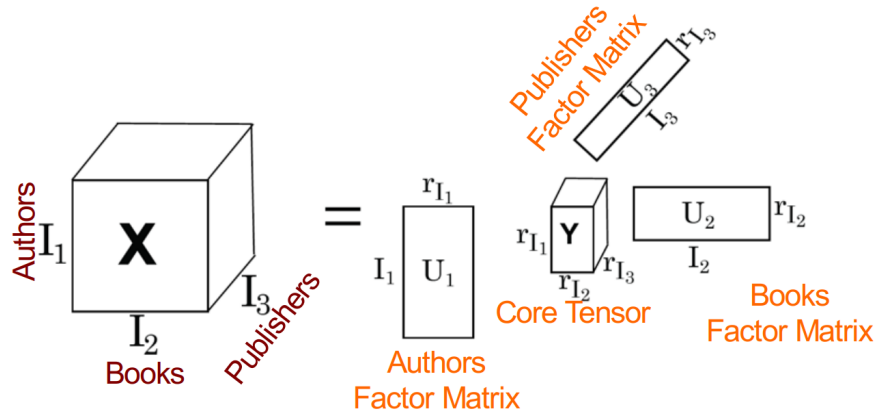


Figura 56: Esempio di decomposizione Tucker in un tensore a 3 modi

Matematicamente, la decomposizione approssimata di Tucker per un tensore a 3 modi si esprime come:

$$\mathcal{X} \approx \mathcal{G} \times_1 A \times_2 B \times_3 C$$

Dove $\times_n$ indica il prodotto tensore-matrice (n-mode product), $\mathcal{G}$ è il core tensor e A, B, C sono le matrici dei fattori (factor matrices). Essendo una tecnica di riduzione della dimensionalità, la ricostruzione non è una trasformazione esatta, ma un'approssimazione ottimizzata.

5.3.1 L'approccio Alternating Least Squares (ALS)

L'ALS (Alternating Least Squares) è un algoritmo iterativo il cui obiettivo è individuare le matrici dei fattori e il core tensor che minimizzano l'errore di ricostruzione del tensore. Poiché ottimizzare simultaneamente tutti i parametri (il core tensor e tutte le matrici dei fattori) è un problema matematicamente intrattabile (non convesso), l'ALS adotta una strategia "alternata".

Il funzionamento si articola nei seguenti passaggi logici:

- **Inizializzazione:** Si creano randomicamente (o tramite SVD) le matrici dei fattori A (di dimensioni $I_1 \times R_1$), B ($I_2 \times R_2$) e C ($I_3 \times R_3$).
- **Fissaggio e Aggiornamento:** Si mantengono fisse due matrici (es. B e C) e si aggiorna la terza (A) risolvendo un problema classico di minimizzazione dei minimi quadrati (Least Squares), che in questo caso diventa lineare e calcolabile.
- **Iterazione:** Si procede a rotazione, aggiornando B mantenendo fisse A e C , e poi C mantenendo fisse A e B . Una volta aggiornate le matrici, si ricalcola il core tensor ottimale.
- **Convergenza:** Il ciclo si ripete finché la differenza di errore tra un'iterazione e l'altra (δ) scende sotto una soglia di tolleranza prefissata (tol), oppure fino al raggiungimento di un numero massimo di iterazioni.

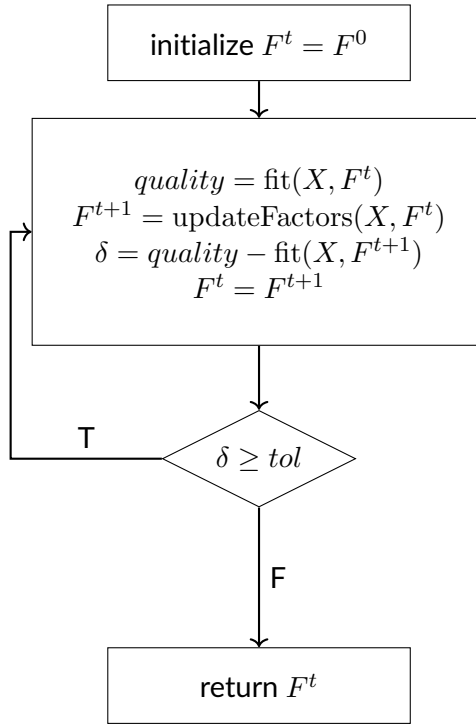


Figura 57: Iterative optimization procedure

5.3.2 Tucker - HOSVD (Higher-Order SVD)

L'HOSVD è una specifica estensione della SVD (Singular Value Decomposition) applicata ai tensori multidimensionali. Serve per calcolare una decomposizione di Tucker in modo diretto.

Il procedimento si scompone in:

1. **Matricizzazione (Unfolding):** A partire dal tensore $\mathcal{X}$, si generano tre matrici bidimensionali $X_{(1)}$, $X_{(2)}$ e $X_{(3)}$ srotolando il tensore lungo le sue tre modalità (I_1, I_2, I_3).
2. **Calcolo dei Fattori tramite SVD:** Su ciascuna matrice srotolata $X_{(n)}$ si applica la SVD classica ($X_{(n)} = U_n \Sigma_n V_n^\top$). Ci concentriamo esclusivamente sulle matrici dei vettori singolari sinistri, ovvero U_1, U_2, U_3 . Queste diventano le nostre matrici dei fattori principali (corrispondenti ad A, B, C).
3. **Derivazione del Core Tensor:** Essendo le matrici U ortogonali, la loro matrice inversa coincide con la loro matrice trasposta ($U^{-1} = U^\top$). Si ottiene quindi il core tensor $\mathcal{G}$ proiettando il tensore originale sulle matrici dei fattori trovate, moltiplicando $\mathcal{X}$ per le trasposte $U_1^\top, U_2^\top$ e $U_3^\top$.

Se si mantengono tutte le componenti, l'HOSVD produce una scomposizione esatta di $\mathcal{X}$. Tuttavia, solitamente si troncano le matrici U ai primi R_n vettori singolari per ottenere un'approssimazione compressa.

5.3.3 Tucker - HOOI (Higher-Order Orthogonal Iteration / ALS for Tucker)

Quando si troncano le componenti per ottenere un'approssimazione, la semplice HOSVD non garantisce l'errore di ricostruzione minimo assoluto. Per questo si utilizza una variante dell'ALS specifica per Tucker, nota anche come HOOI.

L'algoritmo minimizza la seguente funzione di costo:

$$\min_{\hat{\mathcal{X}}} \|\mathcal{X} - \hat{\mathcal{X}}\| \quad \text{con} \quad \hat{\mathcal{X}} = \mathcal{G} \times_1 A \times_2 B \times_3 C$$

Si procede così:

1. Si parte con matrici dei fattori A, B, C inizializzate (spesso usando proprio i risultati della HOSVD come punto di partenza).

2. Si iterano gli aggiornamenti: per rivedere ad esempio A , si proietta il tensore srotolato rispetto a I_1 sulle altre matrici attuali (B e C) e si calcola la SVD del risultato per trovare la nuova A ortogonale.
3. Si ripete l'aggiornamento per B e per C .
4. Al termine del ciclo, si ricalcola il core tensor $\mathcal{G}$.
5. L'iterazione si ferma quando la variazione dell'errore di ricostruzione $\|\mathcal{X} - \hat{\mathcal{X}}\|$ scende sotto la soglia di tolleranza o gli incrementi di accuratezza risultano troppo piccoli.

Osservazioni sulle Performance e Limiti L'analisi tensoriale presenta importanti sfide computazionali:

- **Costo Computazionale:** Cresce in modo esponenziale rispetto alla dimensione dei tensori (modi), quindi si cerca di limitarne drasticamente il numero in fase di progettazione.
- **Intermediary Data Blow-up:** Queste operazioni richiedono risorse di memoria estremamente elevate, ottimizzate solo se i dati riescono a risiedere completamente in RAM. A partire da tensori originariamente sparsi (con molti zeri), i dati intermedi generati durante le moltiplicazioni matriciali non lo sono affatto: diventano densi e subiscono un effetto "esplosione", saturando velocemente la memoria disponibile del sistema.

6 Modelli Probabilistici

La caratteristica fondamentale dei modelli probabilistici è l'assunzione che i dati osservati (es. parole in un testo, interazioni utente-oggetto) siano stati generati da un processo governato da distribuzioni di probabilità sottostanti. Il nostro obiettivo è stimare i parametri di queste distribuzioni affinché spieghino al meglio i dati reali che abbiamo a disposizione.

6.1 Aspect Model (Modello ad Aspetti)

L'Aspect Model formalizza questa intuizione. Supponiamo di avere:

- Un insieme di n oggetti osservabili: $D = \{o_1, \dots, o_n\}$
- Un insieme di m feature: $F = \{f_1, \dots, f_m\}$

Il nostro scopo è modellare la matrice delle co-occorrenze, le cui entrate $P(o, f)$ denotano la **probabilità congiunta** di osservare contemporaneamente un certo oggetto o e una certa feature f nel nostro dataset.

L'assunzione chiave dell'Aspect Model è l'esistenza di variabili **non osservate** (o latenti), denotate con $z \in Z = \{z_1, \dots, z_k\}$. Queste variabili rappresentano le "semantiche" o i "concetti" nascosti che legano gli oggetti alle feature.

Il processo generativo (ovvero come immaginiamo siano stati creati i dati) avviene in tre fasi:

1. Viene selezionato un oggetto o con una probabilità a priori $P(o)$.
2. Dato l'oggetto o , viene selezionata una classe latente z con probabilità condizionata $P(z | o)$.
3. Data la classe latente z , viene generata una feature f con probabilità condizionata $P(f | z)$.

L'ipotesi di indipendenza condizionale afferma che f e o sono indipendenti l'uno dall'altro se conosciamo z .

6.2 Probabilistic LSA (PLSA)

La Probabilistic Latent Semantic Analysis (PLSA) è l'applicazione diretta dell'Aspect Model. Sfruttando la regola delle probabilità totali e le indipendenze descritte, la probabilità congiunta $P(o, f)$ si può esprimere marginalizzando (sommando) su tutti i possibili stati latenti z :

$$P(o, f) = P(o) \sum_{z \in Z} P(f | z) P(z | o)$$

Applicando il Teorema di Bayes sulla prima parte ($P(o)P(z | o) = P(z)P(o | z)$), possiamo riscrivere l'equazione in una forma completamente simmetrica:

$$P(o, f) = \sum_{z \in Z} P(z) P(o | z) P(f | z)$$

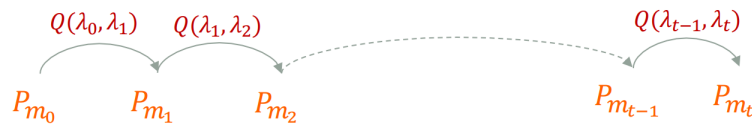


Figura 58: Rappresentazione grafica del modello Probabilistic Latent Semantic Analysis

Questa formulazione è il parallelo probabilistico della scomposizione matriciale che troviamo nella classica SVD ($P = U\Sigma V^T$). In PLSA, la matrice originaria $n \times m$ viene scomposta in probabilità:

- La matrice U ($n \times k$) corrisponde alle probabilità $P(o_i | z_l)$.
- La matrice diagonale Σ ($k \times k$) corrisponde alle probabilità a priori degli argomenti $P(z_l)$.
- La matrice V^T ($k \times m$) corrisponde alle probabilità $P(f_j | z_l)$.

6.2.1 L'algoritmo Expectation-Maximization (EM)

Nella scomposizione classica miriamo a minimizzare l'errore quadratico (norma L2). Nei modelli probabilistici, invece, stiamo confrontando due distribuzioni di probabilità:

- $P_d = P_{data}(o, f)$: la distribuzione empirica (i dati reali osservati).
- $P_m = P_{model}(o, f)$: la distribuzione descritta dal nostro modello parametrizzato.

Per misurare la "distanza" tra due distribuzioni si usa la divergenza di Kullback-Leibler (KL-divergence), $\Delta_{KL}(P_d, P_m)$. Minimizzare la KL-divergence equivale matematicamente a massimizzare la **log-likelihood** (la log-verosimiglianza) dei parametri del modello rispetto ai dati.

Poiché le variabili latenti z non sono direttamente osservabili, non possiamo calcolare la soluzione ottima in un solo passaggio. Dobbiamo usare un approccio iterativo: l'algoritmo Expectation-Maximization (EM).

L'intuizione dell'EM è semplice: se conoscessimo i concetti nascosti z , potremmo calcolare facilmente le probabilità del modello; d'altra parte, se conoscessimo i parametri esatti del modello, potremmo dedurre a quale concetto z appartiene ogni osservazione. L'EM risolve questo problema alternando due fasi finché i valori non si stabilizzano (convergono).

1. E-Step (Expectation) In questa fase calcoliamo il valore atteso (Expectation). Usiamo i parametri **attuali** del modello (che all'inizio vengono inizializzati in modo casuale) per "indovinare" i dati nascosti.

Specificamente, per ogni coppia osservata (o, f) , calcoliamo la probabilità a posteriori che lo stato latente sia z , usando il Teorema di Bayes:

$$P(z | o, f) = \frac{P(z)P(o | z)P(f | z)}{\sum_{z' \in Z} P(z')P(o | z')P(f | z')}$$

Questo valore rappresenta quanto "peso" o responsabilità diamo al concetto latente z per aver generato l'osservazione (o, f) .

2. M-Step (Maximization) In questa fase assumiamo che le probabilità stimate nell'E-Step siano "vere". Usiamo questi pesi per massimizzare la log-likelihood attesa aggiornando i parametri del modello. Le equazioni di aggiornamento per PLSA diventano proporzionali alle stime calcolate:

- **Aggiornamento di $P(f | z)$:** Quante volte la feature f è stata generata da z ?

$$P(f | z) \propto \sum_{o \in D} P_{data}(o, f)P(z | o, f)$$

- **Aggiornamento di $P(o | z)$:** Quante volte l'oggetto o è stato associato a z ?

$$P(o | z) \propto \sum_{f \in F} P_{data}(o, f)P(z | o, f)$$

- **Aggiornamento di $P(z)$:** Qual è la probabilità complessiva dell'argomento z ?

$$P(z) \propto \sum_{o \in D} \sum_{f \in F} P_{data}(o, f)P(z | o, f)$$

(Nota: Il simbolo $\propto$ indica che al termine di ogni calcolo è necessario normalizzare i risultati dividendo per la loro somma totale, così da garantire che si tratti sempre di distribuzioni di probabilità valide che sommano a 1).

Il ciclo si ripete: i nuovi parametri appena calcolati nell'M-Step vengono reinseriti nell'E-Step per ottenere stime migliori per z , in un processo di "hill climbing" che garantisce ad ogni iterazione di aumentare la bontà di adattamento del modello fino al raggiungimento di un massimo locale.

6.3 Latent Dirichlet Allocation (LDA)

La Latent Dirichlet Allocation (LDA) rappresenta un'evoluzione fondamentale rispetto ai precedenti modelli probabilistici. L'obiettivo rimane lo stesso: definire un modello generativo che spieghi come un database di documenti testuali sia stato creato a partire da un insieme di argomenti (topics) latenti.

A differenza dei modelli precedenti, LDA introduce l'uso di **distribuzioni a priori di Dirichlet**, che permettono al modello di generalizzare molto meglio, rendendolo un modello probabilistico generativo completo.

Formalizziamo il nostro scenario:

- **Dati osservati:** Un corpus $D = \{o_1, \dots, o_n\}$ di n documenti (oggetti).
- **Dati osservati:** Un vocabolario $F = \{f_1, \dots, f_m\}$ di m feature (parole).
- **Dati osservati:** Ogni documento o_i ha una lunghezza N_i (il numero totale di parole in esso contenute).
- **Dati non osservati (latenti):** Un insieme di topic $z \in Z = \{z_1, \dots, z_k\}$.

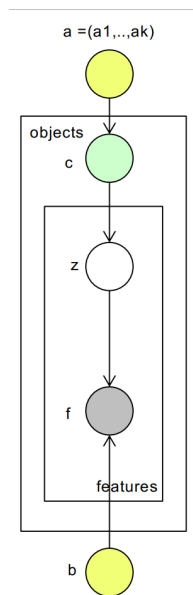


Figura 59: Rappresentazione grafica del modello Latent Dirichlet Allocation

6.3.1 Il Processo Generativo di LDA

LDA assume che ogni documento sia stato generato seguendo un processo probabilistico ben preciso. Per generare un singolo documento o_i , la "macchina probabilistica" esegue i seguenti passi:

1. **Scelta della lunghezza:** Si estrae la lunghezza del documento N_i da una distribuzione di Poisson con parametro ξ :

$$N_i \sim \text{Poisson}(\xi)$$

2. **Scelta della distribuzione dei topic:** Si estrae una distribuzione di topic θ_i specifica per quel documento da una distribuzione di Dirichlet con parametro α :

$$\theta_i \sim \text{Dirichlet}(\alpha)$$

(Intuitivamente: θ_i ci dice in che percentuale il documento o_i parla di politica, sport, economia, ecc.)

3. **Generazione delle singole parole:** Per ciascuna delle N_i feature (parole) da inserire nel documento:

- Si sceglie uno specifico topic $z_{i,j}$ basandosi sulla distribuzione θ_i del documento:

$$z_{i,j} \sim \text{Multinomiale}(\theta_i)$$

- Si sceglie la parola effettiva $f_{i,j}$ basandosi sulla distribuzione delle parole associata a quel topic scelto. Questa distribuzione, che chiamiamo ϕ_z , è a sua volta estratta da un'altra Dirichlet con parametro β :

$$f_{i,j} \sim \text{Multinomiale}(\phi_{z_{i,j}})$$

L'intuizione chiave di LDA è che i documenti mostrano **sparsità**: un documento tipicamente tratta solo un piccolo numero di argomenti (non tutti i k topics), e un argomento usa frequentemente solo un piccolo sottoinsieme di parole del vocabolario. Le distribuzioni di Dirichlet (α e β) sono cruciali proprio perché permettono di modellare e forzare matematicamente questa sparsità.

Poiché calcolare l'inferenza esatta per questo modello è un problema NP-hard, nella pratica si utilizzano algoritmi di approssimazione per stimare i parametri, come il **Gibbs Sampling** o la **Variational Inference**.

$$p(f | a, b) = \int p(c | a) \left(\prod_{i=1}^N \sum_{j=1}^k p(f_i | z_j, b) p(z_j | c) \right) dc$$

6.4 Confronto tra SVD, PLSA e LDA

Questi tre metodi affrontano il problema della riduzione della dimensionalità e dell'estrazione di semantiche latenti, ma partono da presupposti matematici profondamente diversi.

Singular Value Decomposition (SVD) / LSA

- **Natura:** È un metodo puramente algebrico di scomposizione matriciale.
- **Funzione di costo:** Minimizza l'errore quadratico medio (Norma Frobenius / L2). Assume implicitamente che il rumore nei dati segua una distribuzione Gaussiana.
- **Limiti:** Produce valori negativi all'interno delle matrici fattorizzate. In contesti testuali (frequenze di parole) o di raccomandazione, un valore negativo non ha un significato semantico interpretabile, né può essere letto come una probabilità.

Probabilistic LSA (PLSA)

- **Natura:** È un modello probabilistico basato sull'Aspect Model.
- **Funzione di costo:** Minimizza la divergenza di Kullback-Leibler (KL-divergence) tramite Expectation-Maximization. Rispetta la natura di conteggio dei dati testuali (distribuzione multinomiale).
- **Vantaggi:** Sostituisce i valori negativi con vere e proprie probabilità (valori tra 0 e 1 che sommano a 1), rendendo i risultati altamente interpretabili.
- **Limiti:** Soffre di **overfitting**. Il numero di parametri da stimare per le distribuzioni dei documenti ($P(z | o_i)$) cresce linearmente al crescere del numero di documenti nel training set ($k \times n$). Inoltre, non essendo un modello generativo completo, non definisce una probabilità per documenti *nuovi* e mai visti in fase di addestramento.

Latent Dirichlet Allocation (LDA)

- **Natura:** È un modello probabilistico generativo pienamente bayesiano.
- **Vantaggi rispetto a PLSA:** Risolve il problema dell'overfitting trattando le distribuzioni dei topic nei documenti (θ) non come parametri fissi da stimare per ogni documento, ma come variabili aleatorie estratte da una distribuzione a priori globale (Dirichlet).

- **Scalabilità:** Il numero di parametri associati ai documenti non cresce linearmente con le dimensioni del corpus di addestramento. Di conseguenza, LDA generalizza facilmente permettendo di calcolare in modo robusto la distribuzione degli argomenti anche su documenti completamente nuovi.

7 Indicizzazione Spaziale e Riduzione della Dimensionalità

Nota: Per l'introduzione generale e il contesto di riferimento, vedere gli appunti MAADB.

Nei sistemi di gestione di basi di dati (DBMS), strutture dati classiche come i B-Tree sono progettate per indicizzare dati monodimensionali. Quando gestiamo dati spaziali o multidimensionali, emerge la necessità di mappare il nostro spazio n -dimensionale in uno spazio monodimensionale (1D).

Questa trasformazione comporta inevitabilmente una perdita di informazione topologica, ma il nostro obiettivo primario è preservare il più possibile la **località spaziale**. Vorremmo un mapping (una funzione di linearizzazione) che sia consistente con le distanze dello spazio di partenza: *se due punti sono vicini nello spazio multidimensionale originale, le loro proiezioni monodimensionali dovrebbero rimanere vicine nel nuovo ordinamento*.

Una volta ottenute queste proiezioni monodimensionali, possiamo usarle come chiavi primarie per ordinare i punti e inserirli in un indice standard.

7.1 Curve che riempiono lo spazio (Space-Filling Curves)

Per effettuare questa mappatura si utilizzano le **Space-Filling Curves** (curve di riempimento dello spazio). Si tratta di percorsi continui che visitano sequenzialmente tutti i punti di un determinato spazio multidimensionale (griglia), passando per ogni cella **esattamente una sola volta**.

La qualità di una curva di riempimento si valuta in base alla sua capacità di mantenere la località e di evitare "salti" (discontinuità) improvvisi tra punti che, seppur adiacenti nella curva, risultano distanti nello spazio originale.

7.1.1 Ordine per Righe e per Colonne (Row-Major / Column-Major)

È l'approccio più banale. Prevede la lettura sequenziale di tutta la prima riga da sinistra a destra, poi tutta la seconda riga, e così via (o in verticale per l'ordine per colonne). Ad ogni punto viene associato il numero progressivo della lettura.

Svantaggio critico: Si ha un'ottima fedeltà spaziale finché ci si muove sulla stessa riga. Tuttavia, al termine di una riga, si verifica un "salto" improvviso (discontinuità): il punto finale di una riga e il punto iniziale della riga successiva sono vicinissimi nell'ordinamento 1D, ma si trovano ai capi opposti dello spazio multidimensionale di partenza.

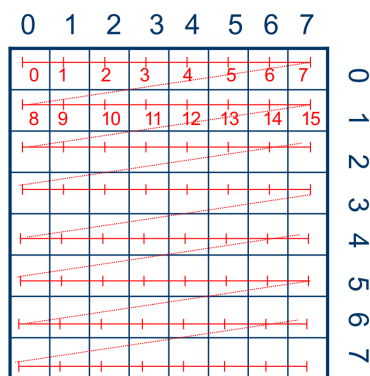


Figura 60: Esempio di ordine per righe e ordine per colonne

7.1.2 Ordine a Serpente (Righe Prime / Boustrophedon)

Per mitigare il problema del salto a fine riga dell'ordine Row-Major, si inverte la direzione di lettura ad ogni riga: la prima riga viene letta da sinistra a destra, la seconda da destra a sinistra, la terza da sinistra a destra, e così via.

Svantaggio critico: Sebbene risolva la discontinuità sui bordi laterali, il problema si ripresenta in forma diversa. Ad esempio, il primo elemento della prima riga e il primo elemento della terza riga sono vicini nello spazio 2D, ma separati da intere righe di scansione nell'ordinamento 1D. I salti diventano meno frequenti ma potenzialmente più penalizzanti in termini di clustering spaziale.

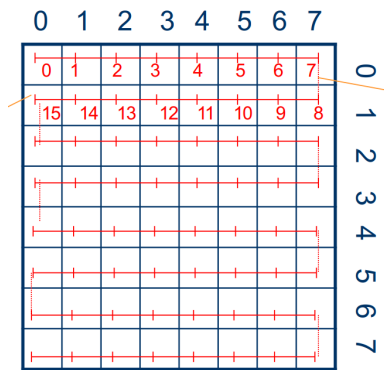


Figura 61: Esempio di ordine per righe prime (a serpente)

7.1.3 Ordine Diagonale (Cantor)

Seguendo la funzione di accoppiamento di Cantor, lo spazio viene scandito lungo le diagonal.

Svantaggio critico: Man mano che la scansione si sposta verso le diagonal centrali, la lunghezza della diagonale aumenta. Di conseguenza, il passaggio dalla fine di una lunga diagonale all'inizio di quella successiva genera salti enormi, rendendo questa curva pessima per il mantenimento della località spaziale nei database.

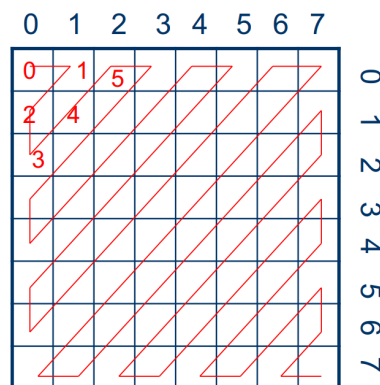


Figura 62: Esempio di ordine diagonale di Cantor

7.1.4 Curva Z-order (Codifica di Morton)

La curva Z-order è il **miglior compromesso** in ambito informatico tra conservazione della località spaziale ed efficienza computazionale. La curva visita lo spazio formando uno schema a forma di "Z" ricorsiva: divide lo spazio in 4 quadranti per formare una Z grande, e all'interno di ogni quadrante applica lo stesso schema a Z su scala minore (approccio frattale associato ai Quad-Tree).

Il vantaggio principale della curva Z-order è la sua **estrema facilità di calcolo**. Per trovare l'indice 1D di un punto, basta prendere le sue coordinate binarie e **intercalare (intermescolare) i loro bit**.

Ad esempio, se abbiamo un punto di coordinate $X = 1$ e $Y = 2$:

- $X = 1 \implies 001_2$
- $Y = 2 \implies 010_2$

Intercalando i bit (prendendo alternativamente un bit da Y e uno da X):

$$Z_{val} = y_2x_2y_1x_1y_0x_0 \implies 001001_2 = 9_{10}$$

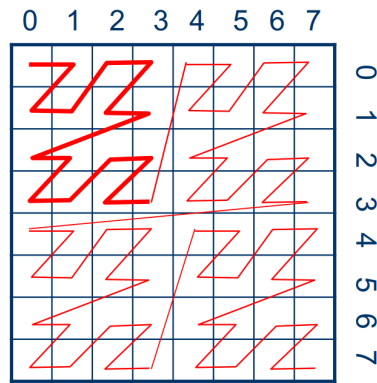


Figura 63: Esempio di curva Z-order e della sua struttura frattale

Oltre all'efficienza computazionale, la codifica di Morton descrive molto bene i **range spaziali** giocando sui prefissi binari. Ad esempio, tutti i punti che condividono lo stesso prefisso binario (es. 0110XX) apparterranno fisicamente allo stesso quadrante logico, permettendo al DBMS di effettuare query di range filtrando interi blocchi di spazio tramite semplici operazioni bitwise.

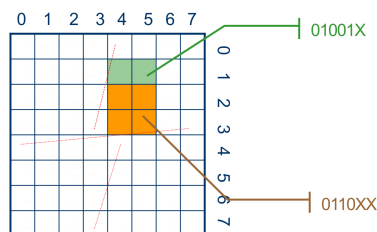


Figura 64: Esempio di raggruppamento di punti vicini (range) tramite prefissi binari nella curva Z-order

7.1.5 Curva di Peano-Hilbert

La curva di Hilbert garantisce la **migliore approssimazione spaziale media** in assoluto. Evita i "salti" diagonali tipici della curva Z-order poiché il passaggio da un quadrante al successivo avviene sempre tra celle rigorosamente adiacenti. Anche in questo caso si tratta di un frattale che si riavvolge su se stesso.

Svantaggio critico: La logica di costruzione è molto complessa. A differenza del semplice bit-interleaving della Z-order, per calcolare il valore di Hilbert di un punto sono necessarie rotazioni e riflessioni binarie dipendenti dallo stato precedente (spesso implementate tramite macchine a stati finiti). Per questo motivo, nei database di grandi dimensioni si preferisce solitamente la Z-order, barattando una leggera perdita di località spaziale in cambio di un'elevata velocità di inserimento e calcolo.

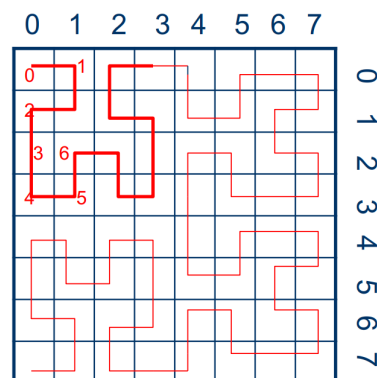


Figura 65: Esempio di curva di Peano-Hilbert e dei suoi passaggi adiacenti

7.2 Come indicizzare sfruttando la multidimensionalità

Definiamo delle strutture. Dati:

- Uno spazio a $N > 1$ dimensioni
- M punti
- una distanza tra i punti

Possiamo usare strutture a indici multidimensionali, ad esempio:

- k-d tree
- point quadrees
- MX quadtree
- R-tree
- TV-tree
- X-tree

7.2.1 File a Griglia (Grid File)

L'obiettivo primario di un Grid File, analogamente a qualsiasi altra struttura di indicizzazione fisica, è l'organizzazione strategica dei dati all'interno delle pagine su disco al fine di minimizzare gli accessi (I/O) necessari per risolvere una specifica query.

Il Grid File opera concettualmente come l'indice di un libro suddiviso in capitoli: lo spazio multidimensionale di ricerca viene scisso tramite una **griglia ortogonale** in regioni (chiamate *celle*), che possono avere dimensioni irregolari a seconda della densità dei dati.

Struttura e Mappatura A ciascuna cella della griglia corrisponde una specifica pagina in memoria (disco), la quale memorizza un certo numero limitato di record vettoriali (o surrogati). La **capacità della cella** è strettamente legata all'hardware: è uguale alla capacità fisica della pagina divisa per la dimensione in byte necessaria a memorizzare un singolo punto. Per gestire questo collegamento, il DBMS mantiene una struttura dati aggiuntiva, detta **Grid Directory**, che mappa le coordinate logiche di ogni cella della griglia al corrispondente indirizzo fisico della pagina su disco.

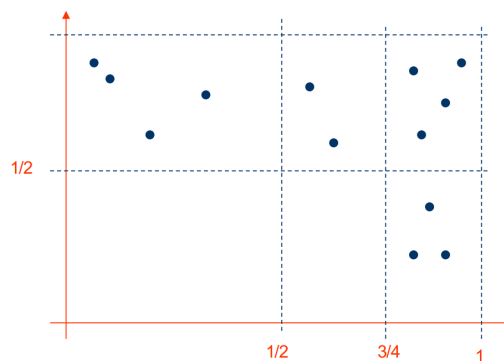


Figura 66: Esempio di file a griglia

Inserimento e Partizionamento Dinamico (Splitting) Il Grid File si adatta dinamicamente all'inserimento dei dati:

1. **Fase Iniziale:** Tutti i punti inseriti convergono in un'unica pagina fisica, finché lo spazio lo consente.
2. **Primo Split:** Quando la pagina raggiunge il limite di saturazione (overflow), lo spazio di ricerca viene diviso tracciando un iper-piano (ad esempio, una linea verticale in $2D$). I punti vengono letti e redistribuiti: tutti gli elementi a sinistra della linea di separazione rimangono nella pagina originale, mentre quelli a destra vengono spostati in una pagina appena allocata.

3. **Split Successivi:** Questo meccanismo fa sì che all'interno della stessa pagina fisica vengano raggruppati punti spazialmente vicini tra loro. Se, in seguito a nuovi inserimenti, una delle pagine dovesse riempirsi nuovamente, il sistema effettuerà uno split lungo una direzione perpendicolare a quella precedente (es. un asse orizzontale), frammentando ulteriormente lo spazio per mantenere la regolarità della griglia vettoriale.

Vantaggi e Svantaggi Questa tipologia di suddivisione a griglia è concettualmente semplice e permette una **facile ed efficiente risoluzione delle query** (in particolare le query puntuali e di range), in quanto la posizione di un punto nella griglia può essere calcolata matematicamente accedendo direttamente alla pagina di interesse.

Tuttavia, il Grid File soffre di alcune **criticità strutturali importanti**:

- **Saturazione non controllabile:** Le pagine in memoria hanno un numero limitato di slot per gli elementi. Il tasso di riempimento di una cella è guidato unicamente dalla similarità e dalla distribuzione reale dei punti, rendendo il controllo del volume dei dati per singola cella difficilmente prevedibile.
- **Spreco di memoria (Data Skew):** Il difetto più grave emerge quando i dati sono distribuiti in modo **fortemente non uniforme** (clusterizzati). Poiché lo split di una cella in overflow traccia un iper-piano che attraversa l'intero dominio dello spazio vettoriale, un taglio reso necessario in una zona densissima dei dati andrà a suddividere inutilmente anche le zone vuote adiacenti. Questo fenomeno causa un'esplosione della dimensione della Grid Directory e genera un enorme numero di celle logiche vuote associate a pagine su disco quasi inutilizzate, portando a uno spreco massiccio di memoria.

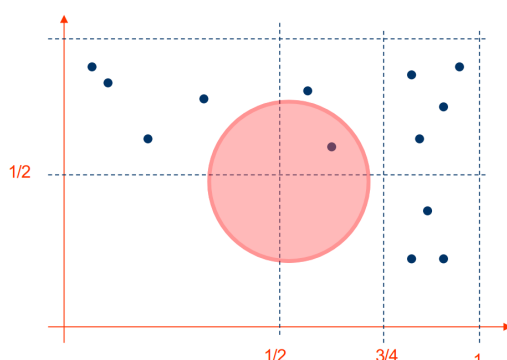


Figura 67: Esempio di file a griglia con accesso a pagine vuote

7.3 Alberi di punti (Point Trees)

Le strutture ad albero per punti sono progettate per indicizzare dati multidimensionali, ovvero vettori nello spazio. L'obiettivo è definire criteri di partizionamento dello spazio che siano computazionalmente efficienti da mantenere, evitando che i costi di riorganizzazione strutturale superino i benefici della ricerca indicizzata su disco.

7.3.1 Point Quadtree (Quadtree di punti)

Il Point Quadtree è un albero multidimensionale in cui, operando in uno spazio bidimensionale ($2D$), ogni nodo interno possiede al massimo 4 figli (fanout pari a 4).

Ogni nodo dell'albero è etichettato da uno dei punti del dataset da indicizzare e, contemporaneamente, funge da fulcro per il partizionamento. La radice divide implicitamente l'intero spazio in 4 quadranti (sottospazi) disgiunti, generati tracciando un asse verticale e uno orizzontale passanti esattamente per le coordinate del punto stesso.

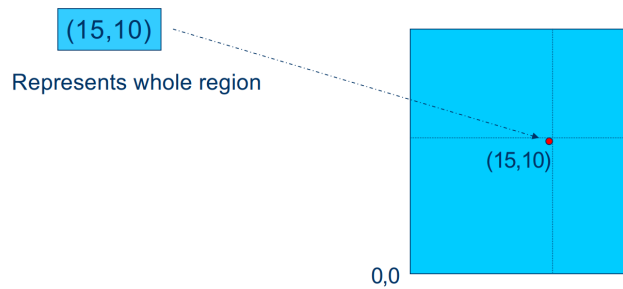


Figura 68: Esempio di quadtree di punti: partizionamento iniziale dalla radice

L'inserimento procede attraversando l'albero: le coordinate di ogni nuovo punto vengono confrontate con quelle del nodo corrente, in modo da assegnarlo al quadrante logico di pertinenza (Nord-Ovest, Nord-Est, Sud-Ovest, Sud-Est). Se il quadrante è vuoto, il punto viene inserito come nodo foglia e, a sua volta, fungerà da centro per partizionare ulteriormente quel sottomondo; se è già occupato, la ricerca scende ricorsivamente lungo il ramo.

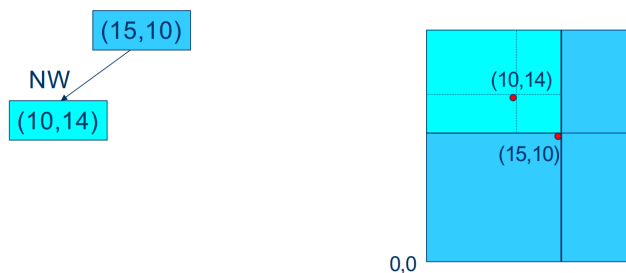


Figura 69: Esempio di quadtree di punti: inserimento di un nuovo punto

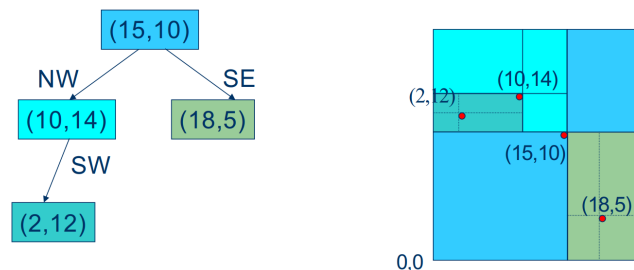


Figura 70: Esempio di quadtree di punti: fase finale

Gestione dei bordi: Al fine di evitare ambiguità algoritmiche, se un punto ricade esattamente su uno degli assi di partizionamento creati da un nodo genitore, è necessario decidere a priori una convenzione di inclusione (es. assegnare sempre all'area di "destra" o "sopra").

A differenza del *Grid File*, in cui la partizione è regolare, la costruzione di un Point Quadtree è **guidata dai dati (data-driven)**. La conformazione dell'albero, pertanto, risente drasticamente dell'ordine di inserimento dei record.

Caratteristiche di Complessità A differenza del *Grid File*, ogni pagina su disco può ospitare più nodi logici. Assumendo di avere N nodi totali e M foglie:

- **Altezza dell'albero:** $O(\log_4 M)$ nel caso ottimo (albero bilanciato); $O(N)$ nel caso pessimo (inserimento di punti già ordinati monotonicamente).
- **Tempo di inserimento:** $O(\log_4 M)$ nel caso ottimo; $O(N)$ nel caso pessimo.

Cancellazione L'operazione di cancellazione in un Point Quadtree è **estremamente complessa**. Eliminando un nodo interno, si distrugge il fulcro di divisione dello spazio per l'intero sottoalbero discendente, costringendo a una costosa e complicata riallocazione strutturale. Per mitigare il problema,

tipicamente si adotta un approccio di **cancellazione logica** (o virtuale): i nodi vengono marcati con un flag come cancellati (tombstone). Quando lo spreco di risorse dovuto ai nodi "fantasma" supera una certa tolleranza, l'intero albero viene ricostruito da zero (*rebuild*).

Ricerca del Vicino Più Prossimo (1-NN Search) L'obiettivo è identificare il punto nel dataset più vicino a una specifica query Q . La ricerca attraversa l'albero procedendo verso i rami più promettenti, ignorando intere regioni spaziali tramite il **pruning** (potatura).

A tal fine è necessaria una metrica spaziale formale. Definiamo la distanza $d : S \times S \rightarrow \mathbb{R}$ che rispetta:

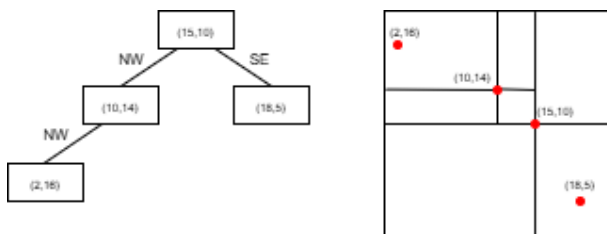
- $d(x, x) = 0$ (Identità)
- $d(x, y) = d(y, x)$ (Simmetria)
- $d(x, y) + d(y, z) \geq d(x, z)$ (Disuguaglianza triangolare)

Estendiamo poi il concetto di distanza dal rapporto "punto-punto" al rapporto "punto-regione" ($S \times S^2 \rightarrow \mathbb{R}$):

$$d(x, R) = \min\{d(x, y) \mid y \in R\}$$

L'algoritmo di visita standard per 1-NN è:

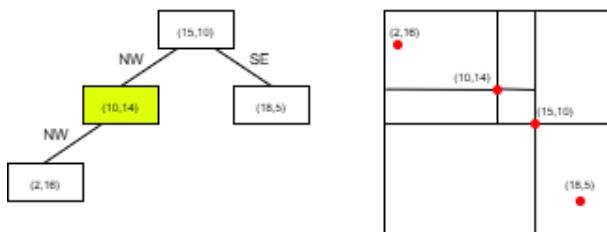
1. Si mantengono in memoria *distanzaMigliore* (inizializzata a ∞) e *soluzioneMigliore* (inizializzata a NIL).
2. Si visita l'albero partendo dalla radice.
3. Calcolata la distanza $d(Q, N.\text{punto})$, se questa è strettamente minore di *distanzaMigliore*, le variabili di tracciamento vengono aggiornate.
4. **Pruning:** Un sottoalbero radicato nel nodo N viene visitato **solo se** la distanza tra la query e la regione rettangolare rappresentata dal nodo è promettente, ovvero se $d(Q, N.\text{regione}) < \text{distanzaMigliore}$. Altrimenti il ramo viene potato.



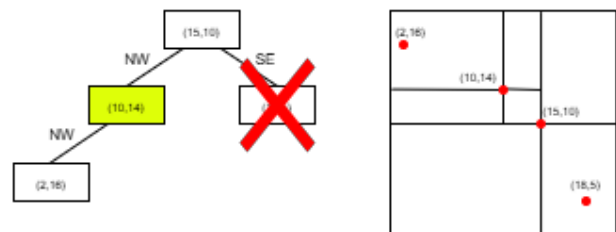
$Q = (9, 14)$ $\text{bestdist} = \infty$, $\text{bestSOL} = \text{NIL}$



$Q = (9, 14)$ Visit root because $d(Q, \text{Root.reg}) = 0 < \text{bestDist}$
Set $\text{bestdist} = \text{SQRT}(52)$, $\text{bestSOL} = (15, 10)$



$Q = (9, 14)$, $d(Q, \text{Root.NW.Reg}) = 0 < \text{SQRT}(52)$



$Q = (9, 14)$, $d(Q, \text{Root.NW.Reg}) = 0 < \text{SQRT}(52)$
 $d(Q, \text{Root.SE.Reg})$ is not less than $\text{SQRT}(52)$ - so pruned

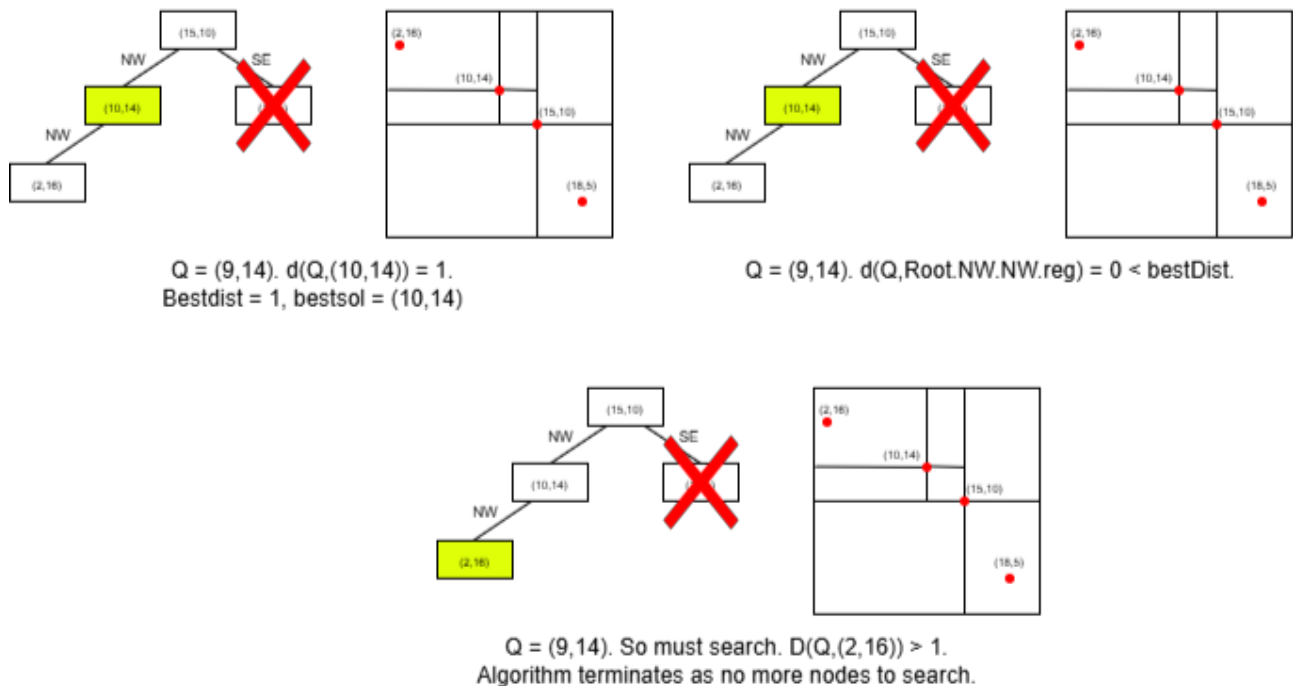


Figura 71: Esempio di ricerca NN nello spazio partizionato

Ricerca dei K Vicini Più Prossimi (K-NN Search) L'algoritmo viene facilmente esteso per le query K-NN:

- Si alloca un array $SOL[1..K]$ inizializzato a NIL, tenuto ordinato per vicinanza (dove $SOL[1]$ è il miglior candidato e $SOL[K]$ l'ultimo utile).
- Si traccia il parametro *bestdist*, che rappresenta costantemente la distanza tra Q e l'elemento in $SOL[K]$.
- **Pruning:** Una regione N .regione viene potata senza essere esplorata se $d(Q, N.regione) \geq \text{bestdist}$.

Query di Range Data una query spaziale centrata in Q con raggio R , si vogliono trovare tutti i punti ricadenti all'interno del cerchio topologico $C(Q, R)$.

- **Condizione di Pruning Rigorosa:** Si ignora il ramo discendente dal nodo N se il rettangolo di pertinenza N .regione non genera alcuna intersezione formale con l'area circolare $C(Q, R)$.
- **Condizione di Pruning Rilassata (Bounding Box):** Al fine di abbassare il costo computazionale, il pruning può essere applicato tramite l'operatore **BB (Bounding Box)**. La regione N viene ignorata solo se non interseca $bb(C(Q, R))$, ovvero il più piccolo rettangolo in grado di racchiudere il cerchio di query. Questa condizione, pur garantendo correttezza algoritmica, offre un pruning inferiore (è meno selettiva) rispetto al calcolo circolare diretto.

Criticità e Limiti dei Point Quadtree Nonostante la concezione intuitiva, la struttura dei Point Quadtree soffre di gravi svantaggi:

- **Decadimento Strutturale:** Essendo data-driven senza bilanciamento automatico, gli alberi tendono a diventare estremamente asimmetrici e profondi.
- **Spreco di Memoria (Frammentazione):** I nodi periferici che non coprono la totalità del fanout presenteranno puntatori a NIL. Essendo la struttura immagazzinata in pagine su disco di dimensione fissa, questi puntatori vuoti occupano spazio reale, diminuendo drasticamente il numero netto di record ospitabili in RAM per pagina (aumentando la I/O disk latency).
- **Gestione dello Storico Difficoltosa:** Il meccanismo di lazy deletion (cancellazione virtuale) costringe al salvataggio in memoria di molti nodi obsoleti.

- **Imprevedibilità Prestazionale:** Come corollario dei punti precedenti, i tempi di latenza necessari a concludere un'operazione K-NN o Range Query diventano scarsamente deterministici nel tempo.

7.4 K-D Trees (K-Dimensional Trees)

I **K-D trees** sono una variante e un'evoluzione dei point quadrees, ideati per mitigare i problemi di sbilanciamento e frammentazione spaziale. A differenza dei quadtree (in cui il fanout dipende dal numero di dimensioni, es. 2^k), i K-D trees sono rigorosamente **alberi binari** per un qualsiasi numero di dimensioni k . Ogni nodo ha quindi al massimo 2 figli.

Come nei point quadrees, ogni nodo rappresenta una regione spaziale e la radice rappresenta l'intero dominio di ricerca. La struttura dell'albero è guidata dai dati (data-driven).

La particolarità del K-D tree risiede nella tecnica di partizionamento: l'algoritmo suddivide lo spazio **lungo una singola dimensione per volta**, alternando ciclicamente la dimensione di split ad ogni livello dell'albero. Assumendo che la radice sia a livello 0, la dimensione D lungo la quale il livello i effettua lo split è data dall'operatore modulo:

$$D = i \bmod k$$

Ad esempio, in uno spazio 2D ($k = 2$), i nodi di livello pari (0, 2, 4...) splittano lungo l'asse x , mentre i nodi di livello dispari (1, 3, 5...) splittano lungo l'asse y . Questa operazione taglia l'iper-piano in due semi-spazi (sinistro/destro o inferiore/superiore).

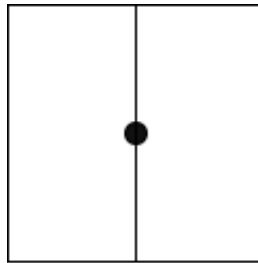


Figura 72: Esempio di KD tree: partizionamento iniziale lungo l'asse x

L'inserimento di un nuovo punto segue la logica di ricerca binaria: si confronta la coordinata del punto da inserire (solo quella relativa alla dimensione di split del livello corrente) con quella del nodo visitato. Se è minore si scende a sinistra, altrimenti a destra.

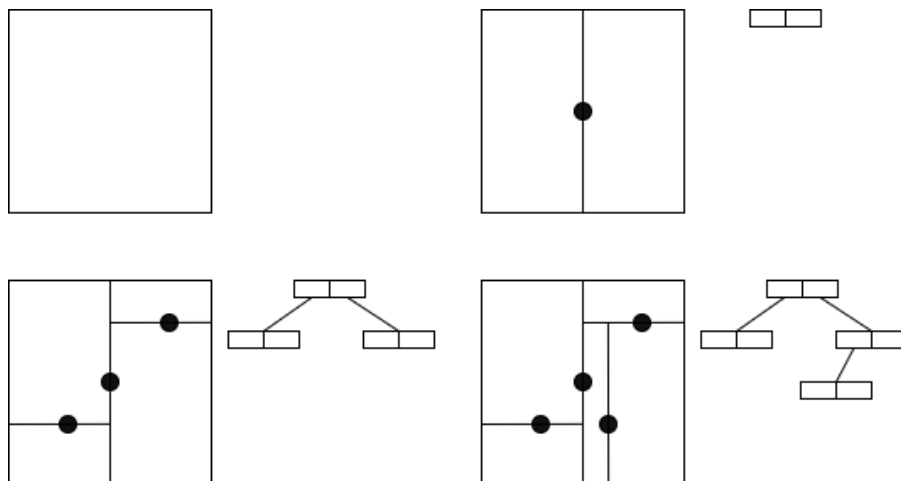


Figura 73: Esempio di inserimento e suddivisione alternata in un KD tree

7.4.1 Estensione a dimensioni $k > 2$

Il meccanismo scala naturalmente per spazi a k dimensioni (es. $3D$, $4D$). Se $k = 3$ (coordinate x, y, z), i livelli splittano in sequenza lungo x , poi y , poi z , per tornare a x al quarto livello. Ogni split non crea più delle semplici linee, ma **iper-piani** di dimensione $k-1$ (es. un piano 2D taglia uno spazio 3D). Il vantaggio

fondamentale è che lo spazio sprecato dai puntatori vuoti (nodi NIL) è drasticamente inferiore rispetto ai quadrees, garantendo una maggiore efficienza di memoria.

7.5 MX Quadrees (Matrix Quadrees)

Se nei point quadrees lo splitting è determinato dalle coordinate esatte dei dati inseriti, portando a partizioni eterogenee, nei **MX quadrees** la regione è splittata in modo geometricamente perfetto e indipendente dai dati.

L'intero spazio è modellato come una matrice discreta di dimensione $2^n \times 2^n$. Ad ogni livello, la regione viene suddivisa tracciando linee verticali e orizzontali passanti esattamente per il **centro geometrico** della regione stessa, creando 4 quadranti identici, finché non si raggiunge la massima risoluzione consentita (pixel o cella di base). I dati reali sono memorizzati unicamente al livello delle foglie.

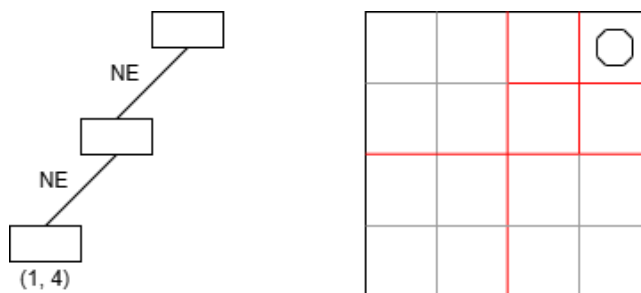


Figura 74: Esempio di inserimento del primo elemento nel centro geometrico in un MX quadtree

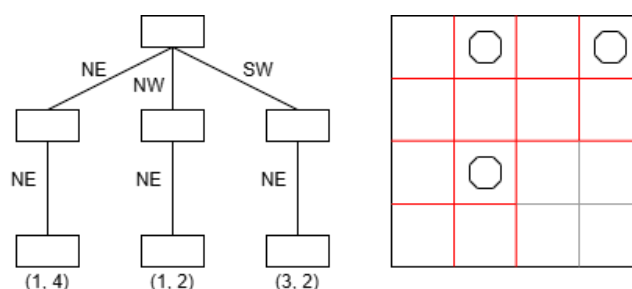


Figura 75: Esempio di inserimento finale: la griglia è regolare

Ricerca e Inserimento L'inserimento guida l'attraversamento in base al quadrante geometrico in cui ricade il punto, forzando la creazione di sottonodi fino alla profondità necessaria per isolare il punto nella matrice. Il costo di inserimento, nel caso pessimo, è proporzionale all'altezza dell'albero, ovvero $O(n)$, dove n è la granularità (risoluzione) dello spazio. La ricerca si avvale delle coordinate di *bounding* del quadrante per decidere se potare (pruning) o esplorare un ramo.

Cancellazione La cancellazione in un MX quadtree è estremamente banale rispetto ai point quadtree. Poiché la geometria dell'albero non dipende dai dati, la rimozione di un punto consiste semplicemente nel marcare la sua foglia come vuota. Successivamente, se tutti i 4 figli di un nodo padre diventano vuoti, l'albero esegue un "merge" risalendo verso l'alto e compattando i nodi inutili (collassamento dell'albero).

7.6 R-Trees (Region/Rectangle Trees)

Gli **R-trees** sono strutture ad albero concepite per indicizzare dati spaziali estesi (regioni, poligoni) tramite approssimazioni rettangolari. Costituiscono lo standard industriale per l'indicizzazione geospaziale nei DBMS (utilizzati in sistemi come PostGIS o, storicamente, da Google Maps).

Sono la generalizzazione spaziale dei B-Tree: alberi bilanciati in cui sia i nodi interni che le foglie possiedono limiti di occupazione. Ogni nodo, grande quanto una pagina di memoria, può contenere fino a N record (Minimum Bounding Rectangles - MBR) e, per garantire il bilanciamento, deve contenerne

almeno $\frac{N}{2}$ (tranne la radice). Il fanout N dipende unicamente dalla dimensione della pagina diviso per la grandezza in byte necessaria a descrivere le coordinate di un MBR.

Rappresentazione delle Coordinate: Per rappresentare un Minimum Bounding Rectangle in k dimensioni, non servono matrici complesse, ma è sufficiente memorizzare gli estremi per ogni asse. In $2D$, servono 4 valori (2 punti cardinali): il vertice in basso a sinistra (x_{min}, y_{min}) e il vertice in alto a destra (x_{max}, y_{max}). In generale, per k dimensioni sono necessarie $2k$ coordinate per definire in modo univoco l'iper-rettangolo:

$$MBR = [x_{1_{min}}, x_{1_{max}}] \times [x_{2_{min}}, x_{2_{max}}] \times \cdots \times [x_{k_{min}}, x_{k_{max}}]$$

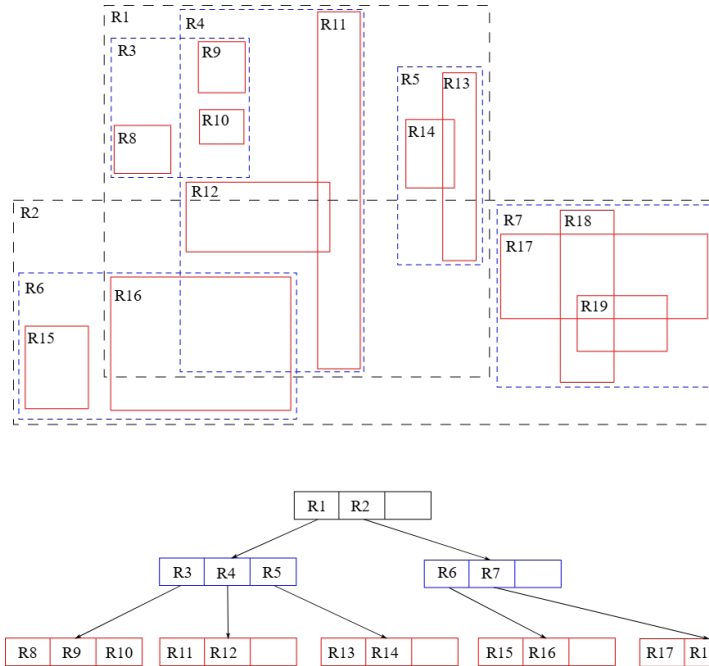


Figura 76: Esempio di R-tree: gerarchia dei Minimum Bounding Rectangles

Ogni nodo interno rappresenta il *bounding box* che ingloba tutti i rettangoli figli. A differenza dei quadtree classici, negli R-trees le regioni appartenenti allo stesso livello **possono sovrapporsi (overlapping)**.

7.6.1 Criteri di Splitting in fase di Inserimento

Quando un nodo supera la capacità massima N (overflow), deve essere splittato in due nodi distinti. L'efficienza delle query future dipende quasi interamente da come questi rettangoli vengono partizionati. Esistono due parametri in conflitto da ottimizzare:

Caso 1: Minimizzazione dello Spazio Morto (Dead Space) Lo spazio morto è l'area del MBR genitore che non contiene dati effettivi (vuoto). Una politica di split potrebbe concentrarsi nel creare due nuovi MBR con un volume totale minimo. Il difetto di questa strategia è che può tollerare e generare un forte *overlapping* tra le nuove regioni.

Caso 2: Minimizzazione dell'Overlapping Se due MBR si sovrappongono parzialmente, una query di range che cade nell'area di intersezione ci costringerà a caricare e visitare **entrambe** le pagine da disco, aumentando il costo di I/O. Questa politica aggrega i rettangoli (es. due molto allungati verticalmente da una parte, due orizzontalmente dall'altra) con l'obiettivo primario di mantenere l'intersezione tra le loro aree il più vicino possibile allo zero. Tuttavia, questo spesso causa un forte aumento dello spazio morto globale.

7.6.2 R+ Trees

Per risolvere radicalmente il problema delle query multiple causate dall'overlapping, gli **R+ trees** vietano categoricamente la sovrapposizione tra nodi dello stesso livello. Se l'inserimento di un MBR causa overlapping tra due regioni genitrici, il rettangolo inserito viene fisicamente "tagliato" (clipped) e diviso in due sotto-rettangoli indipendenti, ciascuno assegnato alla propria regione di pertinenza (mantenendo un identificatore comune).

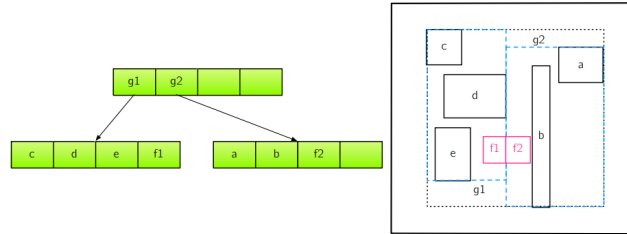


Figura 77: Esempio di R+ tree con oggetti clippati per evitare sovrapposizioni

Questa garanzia velocizza enormemente le query di ricerca puntuale (Point Query), poiché garantisce l'esplorazione di un singolo cammino dalla radice alla foglia. Lo svantaggio è l'esplosione dello spazio su disco e l'overhead gestionale (se una query di range colpisce il pezzo f_1 , l'algoritmo deve assicurarsi di ricostruire e restituire l'oggetto originario aggregando anche f_2).

7.6.3 Ricerca Nearest Neighbor (NN) e le Metriche di Pruning

Nelle query Top-K (o NN), non avendo un raggio R iniziale esplicito, dobbiamo stimarlo dinamicamente in modo da potare l'albero (branch-and-bound) garantendo di non perdere risultati corretti. L'obiettivo è esplorare l'albero abbassando costantemente il raggio di ricerca man mano che troviamo candidati migliori.

Per decidere se un nodo (MBR) debba essere esplorato o scartato, usiamo due metriche di distanza fondamentali calcolate rispetto alla query q :

1. minDist (Ottimismo) $\text{minDist}(q, M)$ è la distanza minima assoluta tra la query q e il bordo della Minimum Bounding Region M . È il limite inferiore: se la regione M contenesse un punto proprio sul suo bordo più vicino a q , questa sarebbe la sua distanza.

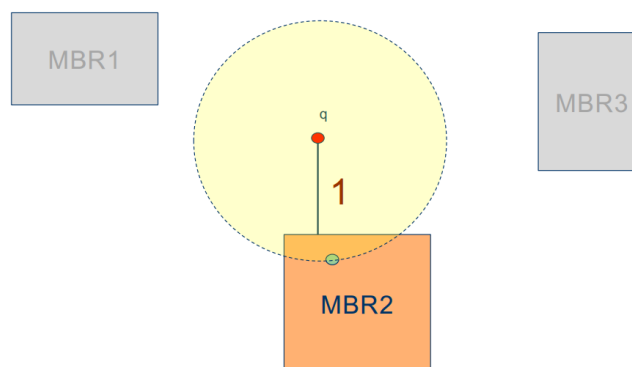


Figura 78: Esempio di calcolo minDist

2. minMaxDist (Pessimismo con Garanzia) $\text{minMaxDist}(q, M)$ fornisce una garanzia cruciale. Per ogni dimensione, considera l'iper-piano dell'MBR più vicino a q , trova in esso il punto geometricamente più lontano da q , e poi prende la minima tra queste distanze massime. Il teorema fondamentale ci assicura che **esiste almeno un oggetto all'interno di M la cui distanza da q è minore o uguale a $\text{minMaxDist}(q, M)$ **.

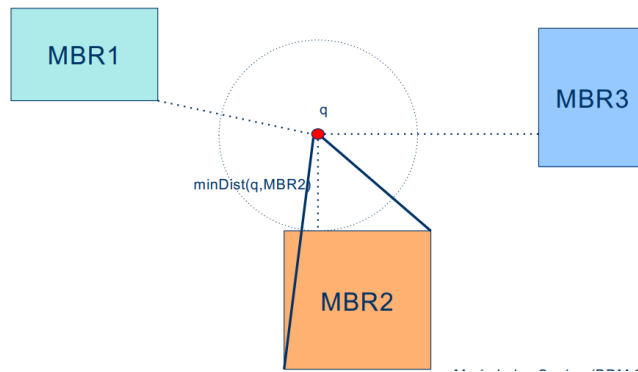


Figura 79: Esempio di minMaxDist fase iniziale

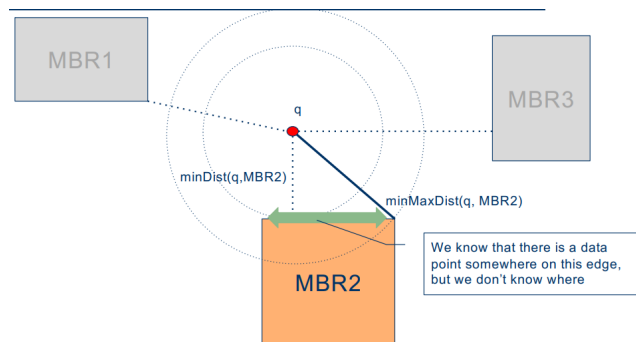


Figura 80: Esempio di minMaxDist fase finale

Sia r il raggio corrente (la distanza del miglior candidato trovato finora). Le regole di pruning algoritmico diventano:

- **Pruning 1:** Se $\minDist(q, M_1) > \minMaxDist(q, M_2)$ per due regioni qualsiasi, la regione M_1 viene totalmente scartata. Poiché in M_2 ci sarà *certamente* qualcosa di più vicino rispetto all'ingresso più ottimistico di M_1 .
- **Pruning 2 (Aggiornamento raggio):** Se troviamo una regione M in cui $\minMaxDist(q, M) < r$, possiamo tranquillamente aggiornare il nostro raggio di ricerca $r \leftarrow \minMaxDist(q, M)$ perché abbiamo la garanzia di trovare un vicino quantomeno a quella distanza.
- **Pruning 3 (Esclusione standard):** Se $\minDist(q, M) > r$, la regione M è scartata perché, nel migliore dei casi, i suoi punti sarebbero comunque più lontani del miglior candidato già in mano.
- **Esplorazione Obbligatoria:** Se $\minDist(q, M) \leq r \leq \minMaxDist(q, M)$, la regione deve essere esplorata, poiché non possiamo confermare né escludere la presenza del punto ottimo al suo interno.

7.7 X-Trees (eXtended Node Trees)

Gli **X-trees** affrontano il noto degrado delle prestazioni degli R-trees in spazi ad alta dimensionalità (maledizione della dimensionalità). All'aumentare delle dimensioni, l'overlapping tra i volumi dei nodi diventa inevitabilmente estremo. Se la logica standard dell'R-tree dovesse forzare uno split che genera un overlapping troppo elevato (secondo una soglia limite), l'X-tree **evita lo split**. Invece di dividere il nodo, ne crea uno a dimensione variabile chiamato **Supernodo**. Questo supernodo alloca più pagine fisiche contigue su disco. Pertanto, l'albero predilige occasionalmente la scansione sequenziale di un supernodo piuttosto che attraversare directory ad albero con intersezioni massicce, ottimizzando nettamente il disk I/O.

7.8 Pyramid Trees

Il **Pyramid tree** è una tecnica di indicizzazione che non mappa rettangoli, ma converte uno spazio multidimensionale uniforme in un valore monodimensionale (1D), permettendo di memorizzare i vettori in un B+ Tree convenzionale.

Il centro esatto dell'iper-cubo dati funge da apice. Lo spazio k -dimensionale viene partizionato in $2k$ **piramidi** indipendenti, le cui basi coincidono con le facce dell'iper-cubo.

Ogni punto vettoriale viene così assegnato univocamente a una specifica piramide. All'interno della piramide, il punto viene mappato calcolando la sua **distanza ortogonale** h dal centro dello spazio verso la base della piramide.

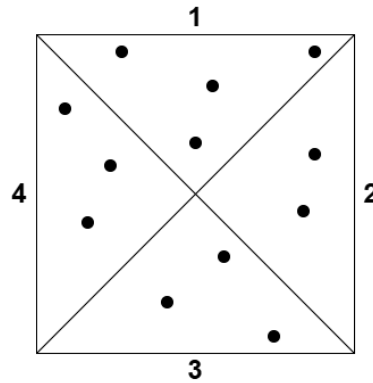


Figura 81: Suddivisione dello spazio in piramidi dal centro

L'identificatore 1D finale (Pyramid Value) di un vettore v si calcola tipicamente come $P_{val} = i + h$, dove i è l'indice numerico identificativo della piramide e h è l'altezza scalata sulla profondità. Tutti i punti presenti in una determinata "fetta" trasversale della piramide godranno di prossimità nel B+ Tree.

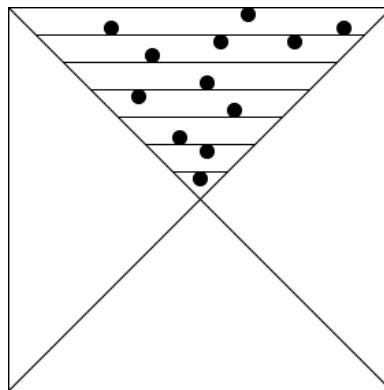


Figura 82: Affettamento trasversale (slicing) delle piramidi per mappare i valori in 1D

Vantaggi e Svantaggi Il pregio principale risiede nell'immunità al partizionamento overlapping (essendo disgiunto a monte) e nella capacità di sfruttare indici scalari estremamente consolidati (B+ Tree). Lo **svantaggio critico** è l'effetto "bordo". I punti vicino al vertice della piramide (il centro geometrico) sono spazialmente molto vicini, ma lungo le ampie basi, la distanza tra punti aumenta. Peggio ancora, due punti vicinissimi ma posizionati a cavallo dello spigolo di divisione tra due piramidi riceveranno P_{val} distantissimi (es. $i = 1$ e $i = 2$), distruggendo la località spaziale durante le query che intersecano i confini piramidali.

7.9 TV Trees (Telescopic Vector Trees)

I **TV Trees** introducono il concetto di riduzione della dimensionalità attiva. In scenari con centinaia di features in cui un R-Tree fallirebbe completamente, l'idea è organizzare ed estrarre le feature ordinandole in base alla loro importanza (es. tramite PCA o varianza intrinseca). Le feature più discriminanti sono poste all'inizio del vettore.

Il termine "telescopico" significa che il vettore logico che descrive la regione viene dinamicamente allungato o accorciato lungo l'albero. Nei nodi più alti (vicino alla radice), l'albero raggruppa gli oggetti usando solo le prime α feature vitali, ignorando totalmente il resto del vettore. Questo consente un fanout elevato (più entry per pagina) e riduce l'overlapping al vertice.

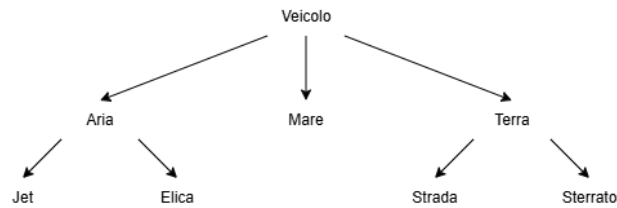


Figura 83: Esempio di discriminazione telescopica progressiva delle feature

Ogni nodo mantiene un **centro** spaziale esatto rispetto ad α feature, e un **raggio di tolleranza**. Solo se lo spazio nella pagina finisce (causa conflitti di dati con le medesime prime feature), l'algoritmo estende telescopicamente la dimensionalità locale a $\alpha + 1$ (o più), includendo la feature successiva per riuscire a splittare e discriminare correttamente i dati, restringendo via via il focus verso le foglie.

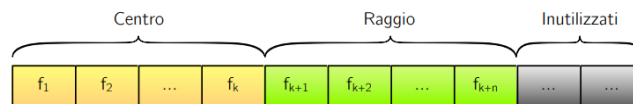


Figura 84: Estensione progressiva del vettore rispetto all'ordine di importanza

Svantaggi Operativi Sebbene garantisca un'indicizzazione altissima, la struttura risulta fragile in contesti dinamici. Richiede che tutte le features siano valutate, pesate e ordinate a monte della costruzione dell'albero. Se nuovi inserimenti modificano statisticamente la varianza di una feature, l'intero ordinamento telescopico diviene sub-ottimale, obbligando di fatto a una complessa ed esplosiva ristrutturazione dell'albero.

8 Hashing per Nearest Neighbor Search

L'obiettivo è definire tecniche di hashing che siano **locality sensitive** (sensibili alla località). A differenza dell'hashing tradizionale, progettato per evitare collisioni, qui vogliamo che la funzione di hash associ allo stesso bucket (ovvero alla stessa pagina su disco) oggetti multimediali che risultano vicini nello spazio di ricerca. In questo modo, la ricerca di oggetti simili si traduce in una o poche letture sequenziali su disco.

8.1 Locality Sensitive Hashing (LSH)

Le tecniche di partizionamento spaziale deterministico (come celle del grid file o space-filling curves) tentano già di preservare la località spaziale. L'approccio **Locality Sensitive Hashing (LSH)** affronta invece il problema tramite una formulazione **randomizzata e probabilistica**.

Nell'hashing standard, se due oggetti sono identici collidono nello stesso bucket, ma nulla si può dire per oggetti vicini ma distinti. Con LSH, il principio fondamentale è che *al crescere della somiglianza tra due oggetti, deve crescere proporzionalmente la loro probabilità di collisione*. L'algoritmo si basa sulla generazione casuale dei bucket: ripetendo il processo di hashing si otterranno configurazioni differenti, ma statisticamente prevedibili.

Sia $\text{sim}(o_1, o_2)$ una funzione di similarità definita nello spazio di ricerca. Una funzione di hash $h(\cdot)$ è sensibile alla località rispetto a $\text{sim}(\cdot)$ se la probabilità di collisione equivale alla loro similarità:

$$\mathbb{P}[h(o_1) = h(o_2)] = \text{sim}(o_1, o_2)$$

La sfida ingegneristica risiede nel trovare famiglie di funzioni H appropriate per specifiche metriche di distanza.

Definizione formale di LSH Una famiglia di funzioni di hash H si definisce (r, cr, P_1, P_2) -sensitive se, per ogni coppia di oggetti o_i e o_j e per una funzione $h \in H$ estratta uniformemente a caso, valgono le seguenti proprietà:

- Se la distanza $d(o_i, o_j) \leq r$, allora $\mathbb{P}[h(o_i) = h(o_j)] \geq P_1$ (alta probabilità di collisione per oggetti vicini).
- Se la distanza $d(o_i, o_j) \geq cr$, allora $\mathbb{P}[h(o_i) = h(o_j)] \leq P_2$ (bassa probabilità di collisione per oggetti lontani).
- $P_1 > P_2$ e $c > 1$.

8.1.1 La Griglia Randomizzata

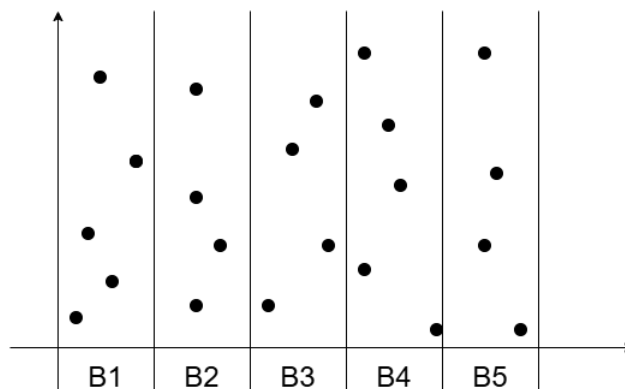


Figura 85: Esempio di partizionamento a griglia

L'utilizzo di una singola funzione di hash per partizionare lo spazio presenta problematiche intrinseche:

- **Falsi positivi (False Hits):** Oggetti distanti possono cadere nello stesso bucket molto ampio.

- **Falsi negativi (Misses):** Oggetti vicini ma situati sui bordi della partizione possono essere assegnati a bucket adiacenti ma separati.

Congiunzione di funzioni (Riduzione dei False Hits) Per abbattere la probabilità che punti lontani collidano, si utilizza l'**amplificazione per congiunzione (AND)**. Si definiscono casualmente nuovi assi e si compone una super-funzione di hash combinando k funzioni indipendenti:

$$g_j(o) = (h_{1,j}(o), \dots, h_{k,j}(o))$$

dove $h_{i,j} \in H$. Affinché due punti collidano rispetto a g_j , devono collidere per *tutte* le k funzioni contemporaneamente.

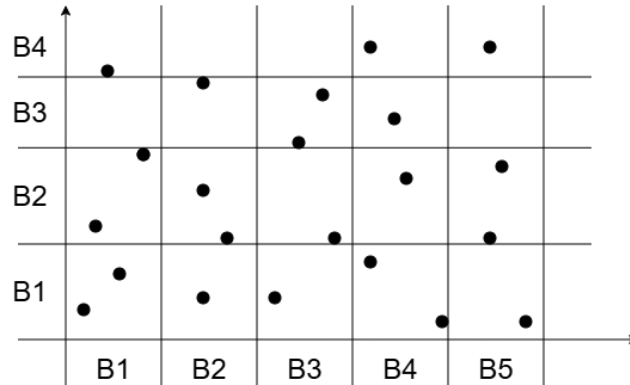


Figura 86: Spazio partizionato tramite congiunzione di funzioni LSH

L'incremento di k riduce drasticamente i falsi positivi, ma sfortunatamente riduce anche la probabilità di successo per punti realmente vicini, aumentando in modo inaccettabile i **missing hits**.

Disgiunzione tramite Livelli (Riduzione dei Misses) Per recuperare i missing hits causati dall'alta restrittività della congiunzione, si adotta l'**amplificazione per disgiunzione (OR)**, costruendo L livelli paralleli di tabelle hash (o foreste di alberi).

Si generano L super-funzioni g :

$$g_1(o) = (h_{1,1}(o), \dots, h_{k,1}(o))$$

$$\vdots$$

$$g_L(o) = (h_{1,L}(o), \dots, h_{k,L}(o))$$

Due oggetti collidono globalmente se condividono il bucket per **almeno una** delle L super-funzioni.

Dimostrazione teorica del numero di livelli L Vogliamo garantire una probabilità molto alta, $1 - \delta$, di recuperare oggetti la cui distanza è $\leq r$ (ovvero, tolleriamo un tasso di miss massimo pari a δ).

Consideriamo o_1, o_2 tali che $d(o_1, o_2) \leq r$.

1. La probabilità di collisione su una singola funzione base è $\mathbb{P}[h(o_1) = h(o_2)] \geq P_1$.
2. La probabilità di collisione su una super-funzione (congiunzione di k funzioni) è P_1^k .
3. La probabilità di **non** collidere su quella super-funzione è $1 - P_1^k$.
4. Essendo i livelli indipendenti, la probabilità di **non collidere in nessuno degli L livelli** è $(1 - P_1^k)^L$.

Poiché vogliamo che questa probabilità di miss sia esattamente uguale alla soglia tollerata δ , imponiamo:

$$(1 - P_1^k)^L = \delta$$

Passando ai logaritmi in base $(1 - P_1^k)$:

$$L = \log_{1-P_1^k} \delta = \frac{\ln(\delta)}{\ln(1 - P_1^k)}$$

Questa formula fornisce l'esatto numero di tabelle hash necessarie per garantire l'efficacia del sistema.

8.1.2 Costruzione delle funzioni LSH per specifiche metriche

Spazio binario: Distanza di Hamming Dati vettori binari d -dimensionali $\{0, 1\}^d$, la distanza di Hamming quantifica il numero di bit divergenti. La famiglia LSH H è composta dalle proiezioni su una coordinata estratta a caso: $h_i(x) = x_i$ con $i \in \{1, \dots, d\}$. La probabilità di collisione $\mathbb{P}[h(p) = h(q)]$ equivale alla frazione di bit identici tra p e q . Questa famiglia è (r, cr, P_1, P_2) -sensitive ponendo $P_1 = 1 - \frac{r}{d}$ e $P_2 = 1 - c\frac{r}{d}$ (con $c > 1 \implies P_1 > P_2$).

Spazio continuo: Distanza di Manhattan (L_1) Per vettori reali operanti sotto la metrica della distanza L_1 , si fissa un'ampiezza di partizione $w \gg r$. Si sovrappone una griglia traslata casualmente nello spazio in ogni dimensione:

- Si generano displacement randomici per gli assi $s_1, \dots, s_d$ con distribuzione uniforme in $[0, w)$.
- La funzione valuta in quale cella spaziale ricade il vettore tramite troncamento:

$$h_{s_1, \dots, s_d}(\mathbf{x}) = \left(\left\lfloor \frac{x_1 - s_1}{w} \right\rfloor, \dots, \left\lfloor \frac{x_d - s_d}{w} \right\rfloor \right)$$

Distanza Euclidea (L_2) Con l'approccio di proiezione casuale (Random Projections), fissiamo un'ampiezza di quantizzazione $w \gg r$. Definiamo un vettore di direzione $\mathbf{p}$ estratto casualmente e uno shift scalare b estratto in $[0, w)$. La funzione LSH proietta il punto $\mathbf{x}$ lungo la retta $\mathbf{p}$ e lo inserisce in segmenti larghi w :

$$h_{\mathbf{p}, b}(\mathbf{x}) = \left\lfloor \frac{\mathbf{p} \cdot \mathbf{x} + b}{w} \right\rfloor$$

Gestione dinamica del raggio di ricerca (Multiscala) LSH fornisce garanzie statistiche fissato uno specifico raggio di soglia r . Se le query del sistema richiedono raggi di ricerca r ampiamente variabili, il database necessiterebbe teoricamente di un indice distinto per ogni possibile raggio. Per limitare l'esplosione dello spazio su disco si adottano euristiche sub-ottimali:

- **Ispezione multiprobe (Adiacenza logica):** A partire dai bucket individuati dalla funzione LSH, il sistema sonda anche i bucket adiacenti nella griglia vettoriale quantizzata, emulando dinamicamente un raggio r (o ampiezza w) maggiore.
- **Rilassamento delle congiunzioni:** Per esplorare un raggio più ampio utilizzando lo stesso indice, si può temporaneamente abbassare il parametro k , richiedendo collisioni solo su un sottoinsieme parziale delle funzioni generate per una specifica g_j .
- **Stima frequenziale:** Si profila il carico applicativo del DBMS e si istanziano indici LSH calcolati esclusivamente sui valori di r statisticamente più rilevanti.

9 Clustering

A differenza delle tecniche di indicizzazione classiche, il clustering non si basa sulla partizione rigida di uno spazio di ricerca vettoriale, ma raggruppa gli oggetti multimediali in base alla loro reciproca affinità. Questa caratteristica lo rende estremamente flessibile: può essere applicato sia quando gli oggetti sono definiti in uno spazio vettoriale, sia quando non lo sono.

Il requisito fondamentale per applicare il clustering è disporre di una misura che indichi quanto gli oggetti siano simili o vicini tra loro:

- **In uno spazio vettoriale:** la somiglianza si calcola tramite una metrica di distanza prestabilita (es. distanza euclidea).
- **Fuori da uno spazio vettoriale:** è sufficiente conoscere la distanza relativa tra gli oggetti, derivata ad esempio da valutazioni umane o da conoscenze a priori fornite dal dominio applicativo.

Non avendo confini geometrici ben definiti per i cluster (come invece accade per le regioni negli indici ad albero), la descrizione del gruppo si riduce all'individuazione di un **rappresentante**. Questo elemento funge da "riassunto" del cluster e può essere:

- **Rappresentante reale (Medoide):** uno degli oggetti effettivamente presenti nel gruppo, scelto perché posizionato in un'area centrale.
- **Rappresentante virtuale (Centroide):** un oggetto artificiale costruito calcolando, per ogni feature, la media dei valori assunti da tutti gli oggetti appartenenti a quel cluster.

Durante l'esecuzione di una query, il sistema confronta la query stessa con i rappresentanti dei vari cluster per decidere se esplorarne o meno il contenuto. Se la query è molto distante dal rappresentante, il cluster viene ignorato. L'obiettivo primario è scegliere un rappresentante che descriva il gruppo nel modo più fedele possibile, riducendo al minimo il rischio di potare (scartare) cluster che in realtà contengono risultati validi.

Alla base di molti algoritmi di clustering vi è la **matrice di somiglianza**. Si tratta di una matrice quadrata in cui righe e colonne rappresentano gli oggetti; il valore della cella i, j esprime il grado di somiglianza tra l'oggetto i e l'oggetto j . Questa struttura può essere interpretata come un **grafo completo**, in cui gli oggetti sono i nodi e il peso di ciascun arco corrisponde alla somiglianza tra i due nodi collegati.

9.1 Potatura dello spazio di ricerca

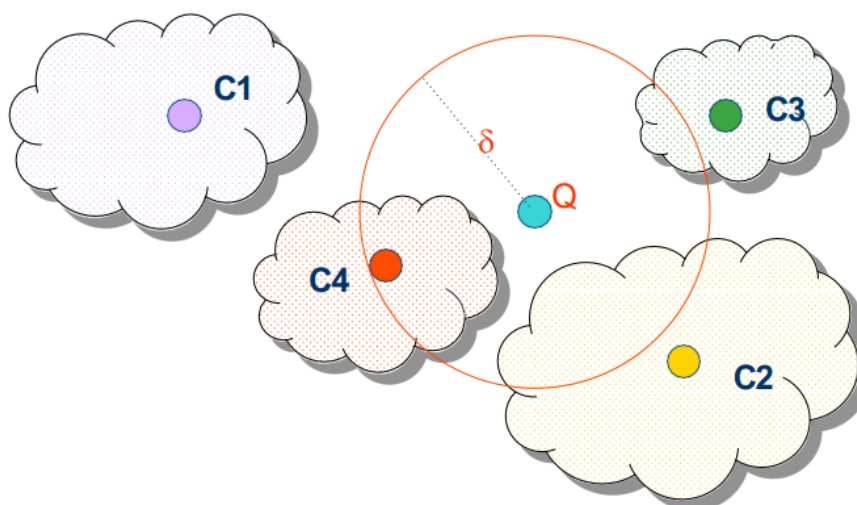


Figura 87: Query di range in uno spazio con cluster

Quando viene sottomessa una richiesta, ad esempio una query di range con raggio δ , il sistema confronta la query Q con i rappresentanti dei cluster disponibili. Verranno analizzati solo gli oggetti contenuti

nei cluster il cui rappresentante interseca l'area della query. Nell'esempio illustrato in figura, l'unico cluster il cui rappresentante ricade all'interno del range della query è C_4 .

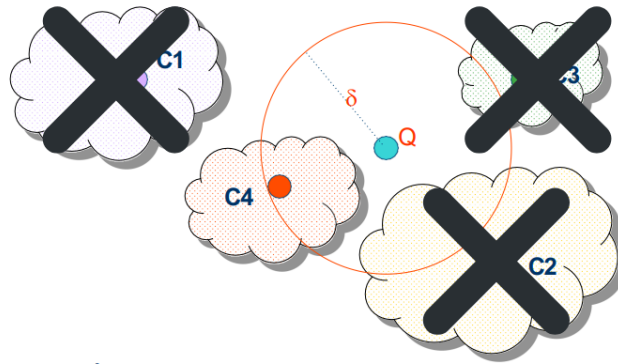


Figura 88: Falsi negativi (Misses) in una query di range

Questa strategia, pur accelerando la ricerca, non è esente da errori. Come si nota nell'esempio, i cluster C_2 e C_3 vengono scartati perché i loro rappresentanti non soddisfano il criterio della query; tuttavia, essi contengono effettivamente alcuni oggetti che si trovano all'interno del raggio δ . Questa situazione genera dei **misses** (o falsi negativi), riducendo la *recall* del sistema.

9.2 Metodi sound (non iterativi)

Gli algoritmi di clustering si dividono principalmente in due grandi categorie: metodi non iterativi (spesso basati su grafi) e metodi iterativi.

Questi approcci si basano sull'utilizzo di un grafo di somiglianza, derivato dalla matrice di somiglianza descritta in precedenza. Il processo richiede la definizione di una soglia di tolleranza T :

- Se si lavora con **misure di somiglianza**, si collegano con un arco solo le coppie di oggetti tali per cui $\text{sim}(o_1, o_2) \geq T$ (soglia minima di somiglianza tollerata).
- Se si lavora con **misure di distanza**, si collegano le coppie tali per cui $\text{dist}(o_1, o_2) \leq T$ (distanza massima tollerata per appartenere allo stesso gruppo).

La soglia T può essere fornita come parametro in input dall'utente oppure calcolata in modo adattivo dall'algoritmo.

9.2.1 Componenti connesse (Single-Linkage)

Un primo approccio consiste nel definire i cluster come le **componenti connesse** del grafo. Se esiste un percorso (anche composto da più archi) che collega due nodi, essi apparterranno allo stesso cluster. Il vantaggio principale è la semplicità computazionale. Il difetto critico è l'estrema sensibilità alla scelta della soglia T :

- Una soglia troppo permissiva genera un unico gigantesco cluster.
- Una soglia troppo stringente produce molti cluster formati da un singolo oggetto.
- Si verifica frequentemente l'**effetto catena** (chaining): oggetti molto distanti tra loro finiscono nello stesso cluster solo perché uniti da una lunga sequenza di nodi intermedi connessi a due a due. All'aumentare delle dimensioni del dataset, la probabilità di questo fenomeno cresce, rendendo il metodo poco robusto.

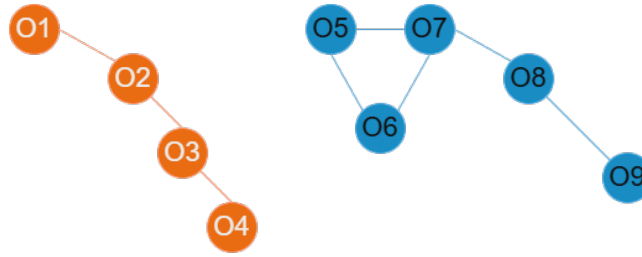


Figura 89: Esempio di clusterizzazione con componenti connesse (Effetto catena)

9.2.2 Clique (Complete-Linkage)

Per ovviare all'effetto catena, si può utilizzare l'approccio basato sulle **clique** (o componenti fortemente connesse). In questo caso, affinché un gruppo di oggetti formi un cluster, ogni oggetto deve essere connesso direttamente con *tutti* gli altri oggetti del gruppo. Questo garantisce che all'interno di un cluster gli oggetti siano tutti strettamente simili tra loro. Tuttavia, presenta due svantaggi significativi:

1. La ricerca delle clique massimali in un grafo è un problema **NP-completo**, rendendo il calcolo molto oneroso su dataset di grandi dimensioni.
2. Le clique non sono necessariamente disgiunte, portando alla formazione di **cluster sovrapposti**. Di conseguenza, durante una query, uno stesso oggetto potrebbe essere esplorato più volte.

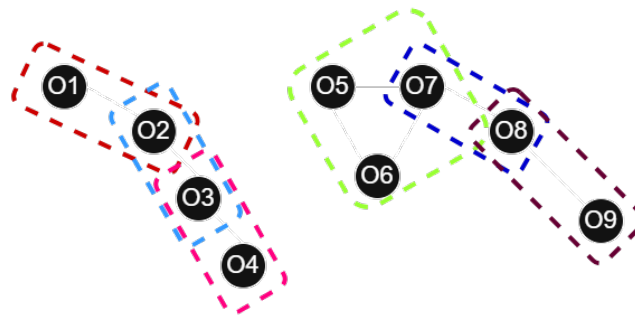


Figura 90: Esempio di clusterizzazione con clique

9.3 Misure di qualità dei cluster

Per valutare la bontà di un raggruppamento si utilizzano diverse metriche che mirano a misurare la coesione interna dei cluster e la separazione tra cluster diversi.

9.3.1 Diametro

Il diametro di un cluster è definito come la **distanza massima** tra una qualsiasi coppia di oggetti appartenenti ad esso. Un buon algoritmo di clustering dovrebbe produrre cluster con diametri ridotti, sintomo di elevata coesione interna.

9.3.2 Omogeneità e Compattezza

La compattezza misura la similarità media tra gli oggetti all'interno dello stesso cluster. L'obiettivo è **massimizzare** questo valore:

$$\text{compactness}(C_i) = \sum_{o_k \neq o_l; o_k, o_l \in C_i} \text{sim}(o_k, o_l)$$

Se ci si trova in uno spazio vettoriale, un ottimo indicatore di omogeneità è il **Root Mean Square Error (RMSE)**. Questa metrica valuta quanto fedelmente il centroide μ_i rappresenti gli oggetti reali del cluster,

misurando la distanza quadratica media degli oggetti dal centro. Si desidera **minimizzare** questo valore, calcolato mediando su tutti i k cluster:

$$\text{RMSE}(C) = \frac{1}{k} \sum_{1 \leq i \leq k} \sqrt{\frac{1}{|C_i|} \sum_{o_j \in C_i} (|o_j - \mu_i|^2)}$$

9.3.3 Dimensione bilanciata

È auspicabile che i cluster abbiano un numero di elementi bilanciato. Cluster troppo grandi avrebbero rappresentanti poco significativi (troppo generici), mentre cluster troppo piccoli risulterebbero ridondanti e poco utili ai fini dell'indicizzazione. Per quantificare il bilanciamento si utilizza l'**Entropia**, che va **massimizzata**. L'entropia è massima quando tutti i cluster equiprobabili (stessa dimensione), e scende quando si hanno squilibri marcati. Data la probabilità $P(C_k)$ che un oggetto appartenga al cluster C_k :

$$\text{entropy}(C) = \sum_{1 \leq i \leq k} P(C_i) \log \left(\frac{1}{P(C_i)} \right)$$

9.3.4 Separazione

Mentre all'interno di un cluster vogliamo alta omogeneità, tra cluster diversi desideriamo alta separazione (oggetti dissimili). La separazione si può valutare misurando la distanza tra gli oggetti più vicini appartenenti a cluster distinti. L'obiettivo è **massimizzare** questo valore:

$$\text{min_separation}(C_i) = \min_{o_l \in C_i, o_k \notin C_i} (\text{dist}(o_k, o_l))$$

In alternativa, in un approccio a grafo, si può valutare il **Cut**. Il cut conta il numero di coppie di oggetti che risultano simili (hanno un arco che li unisce) ma che sono stati assegnati a cluster diversi dall'algoritmo. L'obiettivo è **minimizzare** questo valore:

$$\text{Total Cut} = \sum_{C_i \neq C_j; C_i, C_j \in C} \text{cut}(C_i, C_j)$$

Dove $\text{cut}(C_i, C_j)$ rappresenta il numero di archi spezzati tra il cluster C_i e il cluster C_j .

L'expansion non tiene conto soltanto di quanti archi vengono tagliati tra due cluster, ma normalizza questo valore rispetto alla loro dimensione (cardinalità). Questo permette di distinguere, ad esempio, il taglio di 2 archi su un totale di 100 rispetto al taglio di 2 archi su 5. L'obiettivo è **minimizzare l'expansion**: un valore alto indica che sono stati recisi troppi archi rispetto alle dimensioni dei cluster, suggerendo che sono stati separati oggetti in realtà molto simili tra loro.

$$\text{expansion}(C_i, C_j) = \frac{\text{cut}(C_i, C_j)}{\min\{|C_i|, |C_j|\}}$$

La separazione di un singolo cluster si valuta confrontandolo con il resto del grafo ("il resto del mondo"). Si cerca di minimizzare l'espansione massima, il che equivale a isolare ogni cluster dal resto della rete effettuando il minor numero di tagli possibile rispetto alla sua grandezza:

$$\max_{C_i \in C} \text{expansion}(C_i, S - C_i)$$

La **Cheeger ratio** raffina ulteriormente il concetto di espansione. Invece di normalizzare per il numero di nodi, normalizza per il *grado* totale dei nodi. Si sceglie il minimo tra la somma dei gradi dei nodi all'interno del cluster e quella al di fuori. Questo è utile perché se un cluster contiene nodi con un grado molto alto (molto connessi), è più probabile che il taglio abbia reciso archi significativi, separando indebitamente oggetti affini. Risulta quindi una metrica più precisa dell'expansion:

$$\text{ratio}_{\text{Cheeger}}(C_i) = \frac{\sum_{C_i \neq C_j; C_i, C_j \in C} \text{cut}(C_i, C_j)}{\min\{\sum_{o_k \in C_i} \text{degree}(o_k), \sum_{o_k \notin C_i} \text{degree}(o_k)\}}$$

9.3.5 Integrità intra-cluster

L'integrità di un cluster è definita come la minima espansione valutata tra tutte le possibili bipartizioni (sotto-cluster) in cui il cluster stesso potrebbe essere diviso. Misura quanto il cluster sia coeso e resistente a ulteriori divisioni.

$$integrity(C_i) = \min_{\langle C_{i,1}, C_{i,2} \rangle \in cuts(C_i)} expansion(C_{i,1}, C_{i,2})$$

9.3.6 Modularità

La modularità è una metrica globale fondamentale per valutare la qualità dell'intero partizionamento. Confronta la densità degli archi interni ai cluster con quella che si avrebbe in una distribuzione casuale. Per ogni cluster, calcola il rapporto tra gli archi interni e gli archi totali del grafo ($|E|$), sottraendo poi il valore atteso di questa frazione se gli archi fossero distribuiti casualmente. L'obiettivo è **massimizzare** la modularità: un valore alto indica una struttura di rete ben definita, in cui le connessioni intra-cluster sono dense e quelle inter-cluster sono rare.

$$modularity(C) = \sum_{C_i \in C} \left[\frac{cut(C_i, C_i)}{|E|} - \left(\frac{\sum_{C_j \in C} cut(C_i, C_j)}{|E|} \right)^2 \right]$$

9.3.7 Clustering spettrale

Il clustering basato sul partizionamento spettrale si fonda sull'analisi della **matrice laplaciana** di un grafo. Per un grafo di n nodi, si costruisce una matrice $n \times n$ dove:

- Sulla **diagonale principale** è riportato il grado del nodo (il numero di archi incidenti).
- Gli **altri elementi** valgono -1 se i due nodi corrispondenti sono collegati da un arco, altrimenti 0 .

$$\begin{bmatrix} 1 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{bmatrix}$$

Questa matrice gode di proprietà algebriche fondamentali per individuare partizionamenti ottimali:

- È simmetrica e definita semipositiva, quindi possiede autovalori reali non negativi.
- Il numero di autovalori pari a 0 corrisponde esattamente al numero di componenti connesse del grafo (esiste sempre almeno un autovalore nullo).

Se il grafo è interamente connesso, si analizza il primo autovalore non nullo (il secondo autovalore più piccolo), che definisce la **connettività algebrica** (o *Fiedler value*). Questo valore indica quanto sia difficile dividere il grafo:

- Se è molto vicino a 0 , il grafo si divide facilmente in due macro-strutture.
- Se è grande, il grafo è fortemente coeso.

Per partizionare i dati, si osserva l'**autovettore associato a questo autovalore** (il vettore di Fiedler). Esso conterrà valori positivi e negativi: i nodi associati a componenti positive formano un cluster, quelli associati a componenti negative ne formano un altro. Questo taglio garantisce un partizionamento che recide il minor numero possibile di archi. Il processo può essere iterato ricorsivamente sui sottografi ottenuti. Il limite principale di questa tecnica è il costo computazionale, che diventa proibitivo su grafi molto grandi a causa del calcolo di autovalori e autovettori.

9.4 Metodi iterativi

9.4.1 Single-pass (leader-based)

L'obiettivo di questi metodi è raggruppare gli oggetti scorrendo il database una singola volta, garantendo grande efficienza computazionale.

Il funzionamento base (algoritmo del leader) procede così: si seleziona il primo oggetto, che diventa il rappresentante (leader) del primo cluster. Quando si esamina l'oggetto successivo, se ne calcola la distanza dai rappresentanti esistenti:

- Se la distanza è **inferiore** a una soglia T , l'oggetto viene assegnato al cluster.
- Se la distanza è **superiore** a T per tutti i cluster attuali, l'oggetto diventa il rappresentante di un nuovo cluster.

Esistono diverse varianti per decidere l'assegnazione quando l'oggetto rispetta la soglia T per più di un cluster:

- **Closest-leader**: si assegna al cluster con il leader più vicino (favorisce la compattezza).
- **Smallest-cluster**: si assegna al cluster con meno elementi (favorisce il bilanciamento).
- **First-match**: si assegna al primo leader trovato che rispetta la soglia (massimizza la velocità).

Anche il ruolo di rappresentante può essere fisso (l'oggetto originario) o adattivo (il centroide del cluster viene ricalcolato a ogni inserimento; più costoso ma produce cluster migliori). In tutti i casi, il risultato finale dipende fortemente dalla scelta della soglia T e dall'ordine di lettura dei dati.

9.4.2 Assenza di soglia

In assenza di una soglia prefissata, si può operare partendo dal grafo delle distanze e costruendo il **Minimum Spanning Tree (MST)**, ovvero l'albero di copertura che connette tutti i nodi minimizzando la somma dei pesi degli archi.

Per creare i cluster, si procede "potando" i rami dell'albero: si eliminano gli archi il cui peso (distanza) risulta anomalo o significativamente superiore rispetto alla media degli archi adiacenti. Rimuovendo i collegamenti più lunghi, l'albero si frammenta in sottografi disgiunti, che costituiscono i cluster finali.

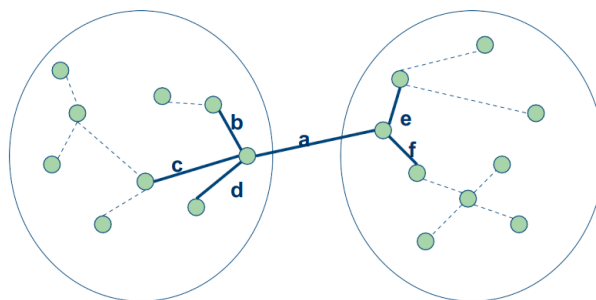


Figura 91: Esempio di clusterizzazione senza soglia (MST pruning). L'arco a rappresenta una distanza anomala e viene reciso per formare i cluster.

9.4.3 Max-a-min

Rinunciando al vincolo di scorrere i dati una sola volta, l'approccio iterativo Max-a-min offre maggiore precisione. Richiede in input non una soglia di distanza, ma il numero predefinito di cluster desiderati, k . Si divide in due fasi logiche:

Massimizzazione (Inizializzazione) L'obiettivo è scegliere k rappresentanti il più distanti possibile tra loro, per coprire bene lo spazio. Si seleziona un oggetto a caso come primo rappresentante. Attraverso un passaggio completo sul dataset, si trova l'oggetto con la massima distanza dal primo, eleggendolo a secondo rappresentante. Il terzo sarà quello che massimizza la minima distanza dai due già scelti, e così via fino a isolare k centroidi.

Minimizzazione (Assegnazione) Si scorre nuovamente il database e ogni elemento viene assegnato definitivamente al rappresentante a esso più vicino, completando la partizione.

K-Means Il **K-Means** è la naturale evoluzione basata sull'ottimizzazione iterativa. Solitamente parte da un'inizializzazione (che può essere randomica o stile Max-a-min) e utilizza una funzione di costo, tipicamente l'errore quadratico medio o RMSE, per valutare la qualità dei cluster. L'algoritmo alterna ripetutamente due passi:

1. Assegna ogni oggetto al cluster il cui centroide è più vicino.
2. Ricalcola il centroide di ogni cluster come punto medio degli oggetti ad esso assegnati.

Il processo termina quando gli oggetti smettono di cambiare cluster, massimizzando la compattezza. La compattezza del singolo cluster è valutata sui residui:

$$RMSE_m = \sqrt{\frac{1}{N} \sum_{i=1}^N [\hat{F}_m(z_i) - F_m(z_i)]^2}$$

La qualità globale è la media dei valori di ogni cluster:

$$RMSE = \frac{1}{M} \sum_{m=1}^M RMSE_m$$

Un *RMSE* troppo vicino a zero o non convergente può indicare l'uso di feature poco discriminanti per lo spazio vettoriale scelto.

9.4.4 Assenza di distanza

Quando non si dispone di metriche esatte per calcolare la distanza geometrica o semantica tra gli oggetti (es. nei sistemi di raccomandazione), la somiglianza viene inferita dinamicamente osservando il **comportamento degli utenti**. Oggetti fruiti o acceduti congiuntamente tendono a essere considerati vicini.

Confidence clustering (Clustering basato su confidenza) Questo algoritmo è particolarmente utile in scenari dinamici in cui non è possibile definire una distanza spaziale o semantica esatta tra gli oggetti, ma si dispone di informazioni comportamentali (ad esempio, il pattern di accesso degli utenti). L'idea alla base è che se due oggetti vengono frequentemente fruiti insieme (co-occorrenza), essi debbano appartenere allo stesso gruppo.

Il metodo richiede di fissare a priori due parametri:

1. Il **numero totale di cluster** K che si desidera ottenere.
2. La **capienza massima** r di ogni cluster, per garantire che i raggruppamenti rimangano dimensionalmente bilanciati e che un cluster non "assorba" l'intero database.

Inizializzazione e concetto di Confidenza L'algoritmo parte assegnando ogni oggetto o_i a un cluster in modo del tutto **casuale**. A ogni assegnazione viene associato un valore di "confidenza" (conf), limitato in un range discreto (ad esempio, tra 1 e 10, dove 1 rappresenta la precarietà assoluta e 10 la massima stabilità). Questo punteggio rappresenta quanto il sistema è "sicuro" che un determinato oggetto si trovi nel cluster corretto. Inizialmente, anche questo valore può essere assegnato casualmente.

Dinamica di aggiornamento (Pattern di Accesso) Il vero cuore dell'algoritmo si attiva dinamicamente quando il sistema registra una "evidenza", ad esempio quando un utente accede simultaneamente agli oggetti o_a e o_b . A questo punto si verifica l'appartenenza dei due oggetti ai cluster e si agisce di conseguenza:

- **Scenario 1: Conferma (Stesso Cluster)**

Se o_a e o_b appartengono già allo **stesso cluster**, l'evidenza empirica corrobora la partizione attuale. Di conseguenza, il sistema "premia" l'assegnazione incrementando la confidenza di entrambi i nodi ($\text{conf}(a)++$, $\text{conf}(b)++$, fino al valore massimo).

- **Scenario 2: Discrepanza (Cluster Diversi)**

Se o_a si trova nel cluster C_i e o_b nel cluster C_j , c'è un conflitto tra l'appartenenza reale e il comportamento dell'utente. Questa anomalia richiede una reazione per tentare di riavvicinare gli oggetti, gestita secondo tre possibili sotto-casi dipendenti dalla forza (confidenza) dei due oggetti:

- **Caso 2a: Scontro tra elementi "forti" ($\text{conf}(a) > 1$ e $\text{conf}(b) > 1$)**

Entrambi gli oggetti sono ben radicati nei rispettivi cluster. Poiché nessuno dei due è un candidato debole facilmente spostabile, l'algoritmo non effettua alcuno spostamento per non compromettere partizioni che storicamente hanno funzionato. Si limita a insinuare il dubbio: decrementa la confidenza di entrambi ($\text{conf}(a)-$, $\text{conf}(b)-$). Se accessi anomali come questo si ripetono, la loro confidenza scenderà gradualmente fino a renderli spostabili.

- **Caso 2b: Incontro tra elementi "deboli" ($\text{conf}(a) == 1$ e $\text{conf}(b) == 1$)**

Entrambi gli oggetti sono in uno stato di estrema precarietà. Si decide di unirli forzatamente spostandone uno (es. muovendo o_b nel cluster C_i di o_a). Tuttavia, per il **vincolo di capienza massima** costante r , l'ingresso di o_b in C_i obbliga a espellere un altro oggetto o_c da C_i , che verrà mandato in C_j per compensare. Si esegue lo scambio e si assegna o si mantiene a 1 la confidenza dei nodi appena spostati.

- **Caso 2c: Attrazione di un forte su un debole ($\text{conf}(a) > 1$ e $\text{conf}(b) == 1$)**

In questa situazione sbilanciata, l'oggetto stabile o_a cerca di "attirare" l'oggetto debole o_b all'interno del proprio cluster C_i . Ancora una volta, per far entrare o_b , serve fare spazio cacciando un altro elemento o_c da C_i . Qui l'algoritmo agisce con cautela:

- * **Se esiste un elemento sacrificabile:** Si esplora il cluster C_i alla ricerca di un nodo o_c con confidenza minima ($\text{conf}(c) == 1$). Se lo si trova, avviene lo scambio: o_b entra in C_i e il debole o_c finisce in C_j .
- * **Se NON esiste un elemento sacrificabile:** Tutti gli elementi in C_i hanno una confidenza alta (> 1). Spostare uno di questi elementi causerebbe un grave danno a un cluster molto coeso. L'algoritmo quindi **annulla lo spostamento** di o_b . Come penalità per l'anomalia registrata, si decrementa la confidenza del nodo forte o_a ($\text{conf}(a)-$). (Non si tocca la confidenza di o_b perché è già al valore minimo e non avrebbe senso portarla a 0 in questo contesto).

Considerazioni finali sul ruolo della soglia massima Il valore limite della confidenza (es. 10) svolge il ruolo di "ammortizzatore" contro la volatilità (rumore). Se un oggetto raggiunge confidenza 10, significa che ha maturato una lunga storia di accessi coerenti all'interno del suo cluster. Per farlo spostare, non basterà un singolo accesso anomalo: serviranno almeno 10 co-occorrenze con oggetti di altri cluster (senza che nel frattempo vi siano accessi conformi a rafforzarlo) prima che la sua confidenza scenda a 1 e diventi "sacrificabile". In questo modo, l'algoritmo ignora i comportamenti sporadici o accidentali, modellandosi solo sulle reali tendenze strutturali degli utenti.

9.4.5 Clustering adattivo

Variante ancora più flessibile in cui non è prefissato nemmeno il numero di cluster. Ogni oggetto viene posizionato inizialmente in modo casuale lungo un asse unidimensionale (la retta dei reali). A ogni co-occorrenza o accesso simultaneo tra un gruppo di oggetti, la loro posizione viene alterata: vengono avvicinati artificialmente verso il loro **centro di massa** condiviso di un fattore parametrico. Per evitare il collasso globale, un numero equivalente di oggetti casuali viene allontanato. In presenza di regolarità comportamentali costanti, gli oggetti strettamente correlati formeranno spontaneamente aggregati densi lungo la retta, determinando i cluster in via del tutto emergente.

10 Classificazione

La classificazione può essere vista concettualmente come una sorta di clustering supervisionato. A differenza del clustering tradizionale, in cui il sistema deve scoprire autonomamente la struttura dei dati, nella classificazione gli oggetti di partenza (che costituiscono il **training set**) sono già etichettati, ovvero associati a una classe nota a priori. L'obiettivo è analizzare questi oggetti per dedurre delle regole di separazione o dei modelli matematici in grado di descrivere le differenze tra le classi. Questi modelli guideranno poi il sistema nella classificazione automatica di nuovi oggetti non etichettati (fase di *testing* o *inferenza*).

10.1 Support Vector Machines (SVM)

Le Support Vector Machines (Macchine a Vettori di Supporto) sono tra i classificatori più robusti e utilizzati, progettati nativamente per risolvere problemi di classificazione **binaria** (due classi).

L'idea alla base delle SVM è quella di verificare se le due classi di oggetti sono linearmente separabili (cioè non mescolate nello spazio vettoriale) e, in tal caso, individuare il miglior confine di separazione. Questo confine geometrico è una retta in uno spazio bidimensionale, un piano in tre dimensioni, e prende il nome generico di **iperpiano di separazione** in uno spazio multidimensionale.

L'equazione dell'iperpiano è:

$$\vec{w} \cdot \vec{x} + b = 0$$

dove $\vec{w}$ è il vettore dei pesi (normale all'iperpiano, che ne definisce l'orientamento) e b è un termine di bias (che trasla l'iperpiano rispetto all'origine).

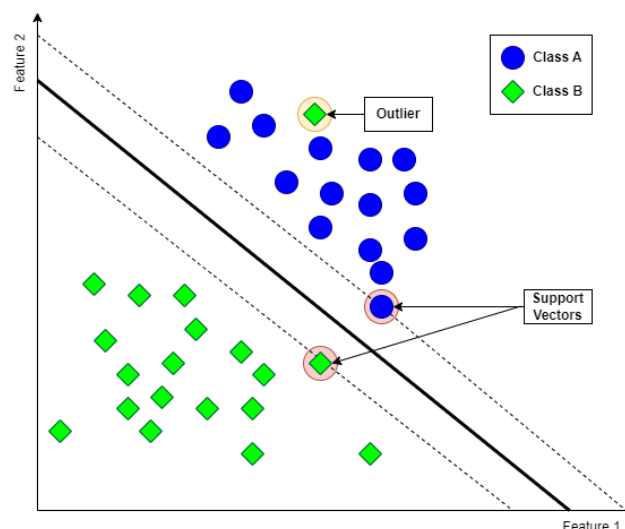


Figura 92: Esempio di Support Vector Machine in uno spazio bidimensionale

Quando un nuovo oggetto $\vec{x}_{new}$ deve essere classificato, lo si valuta rispetto all'iperpiano: se il risultato dell'equazione è positivo, l'oggetto appartiene a una classe; se è negativo, appartiene all'altra.

Tra gli infiniti iperpiani che possono separare due classi, la SVM cerca quello **ottimo**, definito dai seguenti criteri:

- Deve separare correttamente i dati di addestramento.
- Deve trovarsi alla **massima distanza possibile** dai punti più vicini di entrambe le classi.

Questi punti critici, che si trovano esattamente ai bordi della fascia di separazione, sono detti **vettori di supporto** (da cui il nome dell'algoritmo). Sono gli unici punti che influenzano realmente la posizione dell'iperpiano; tutti gli altri punti più lontani potrebbero essere rimossi senza alterare il risultato.

Definiamo i confini di questa fascia di separazione (il "margine") imponendo che passino per i vettori di supporto. Le equazioni di questi iperpiani marginali diventano:

$$\vec{w} \cdot \vec{x} + b = +1 \quad \text{e} \quad \vec{w} \cdot \vec{x} + b = -1$$

La larghezza totale di questa fascia di separazione, detta **margin**, è calcolabile geometricamente come:

$$\frac{2}{\|\vec{w}\|}$$

L'obiettivo matematico della SVM è trovare il vettore $\vec{w}$ e la costante b che massimizzino questo margine (il che equivale a **minimizzare** la norma $\|\vec{w}\|$) rispettando il vincolo che tutti gli elementi di una classe stiano al di sopra del margine positivo ($\vec{w} \cdot \vec{x} + b \geq 1$) e quelli dell'altra classe al di sotto del margine negativo ($\vec{w} \cdot \vec{x} + b \leq -1$). Questo si traduce in un problema di ottimizzazione vincolata.

Soft Margin (Tolleranza all'errore) Nella realtà, i dati raramente sono perfettamente separabili in modo lineare; spesso ci sono sovrapposizioni o *outlier* (dati anomali o etichettati male). Per evitare che l'algoritmo fallisca o crei un margine troppo stretto e instabile per colpa di pochi punti, si introduce il concetto di **Soft Margin**. Si rilassano i vincoli tollerando che un certo numero di oggetti invada il margine o cada nel semispazio della classe sbagliata. Il problema di ottimizzazione viene modificato per minimizzare due quantità contemporaneamente: la norma $\|\vec{w}\|$ (per allargare il margine) e una variabile di costo che penalizza l'errore di classificazione.

Il trucco del Kernel (Kernel Trick) Quando i dati sono intrinsecamente intrecciati e non è possibile separarli con un iperpiano lineare nemmeno usando un soft margin, le SVM applicano una trasformazione matematica. Sfruttando funzioni chiamate **Kernel**, i dati vengono proiettati in uno spazio vettoriale a dimensionalità superiore. In questo nuovo spazio "arricchito", i dati diventano linearmente separabili, permettendo di trovare un iperpiano che, riportato nello spazio originale, corrisponde a un confine di separazione curvo e altamente non lineare.

10.1.1 Classificazione N-aria (Multiclasse)

Poiché le SVM sono per natura classificatori binari, per risolvere problemi con N classi (es. Rosso, Verde, Blu, Giallo) si utilizzano delle strategie che combinano classificatori binari multipli. Le due tecniche principali sono:

One-vs-One (OvO - Uno contro Uno) Si addestra un classificatore binario per ogni possibile coppia di classi. Se abbiamo n classi, addestreremo $\frac{n(n-1)}{2}$ classificatori (es. per 4 classi: RV, RB, RG, VB, VG, BG = 6 classificatori). Quando arriva un nuovo oggetto da classificare, questo viene sottoposto a tutti i modelli. Ogni classificatore emette un voto per una delle due classi che sa distinguere. Alla fine, l'oggetto viene assegnato alla classe che ha ricevuto il maggior numero di voti in totale.

One-vs-Rest (OvR o OvA - Uno contro Tutti) Si addestrano tanti classificatori quante sono le classi (quindi n classificatori). Ogni modello è specializzato nel distinguere *una specifica classe* da *tutte le altre combinate* (es. Rosso contro "Non Rosso", Verde contro "Non Verde", ecc.). In fase di classificazione, l'oggetto viene valutato da tutti gli n classificatori. Viene scelta la classe il cui classificatore restituisce il valore di confidenza più alto (ovvero, l'oggetto risulta essere più "profondamente" e inequivocabilmente all'interno della regione di quella specifica classe rispetto al confine di separazione).

10.2 K-Nearest Neighbors (k-NN)

Uno dei principali vantaggi della rappresentazione degli oggetti in uno spazio vettoriale è la possibilità di risolvere problemi di classificazione sfruttando algoritmi basati sul concetto di distanza geometrica, come il **k-Nearest Neighbors (k-NN)**. Si tratta di un metodo di apprendimento supervisionato di tipo *lazy* (o basato sull'istanza): l'algoritmo non costruisce un modello matematico esplicito durante una fase di addestramento, ma memorizza l'intero dataset e differisce tutto il calcolo al momento dell'inferenza.

10.2.1 Meccanismo di classificazione

Quando si deve classificare un nuovo oggetto (query), l'algoritmo calcola la distanza tra questo e tutti gli oggetti etichettati presenti nel training set. Successivamente, seleziona i k elementi con la distanza minore (i "vicini"). La classificazione avviene tramite un sistema di **votazione a maggioranza** (Majority Voting): ogni vicino esprime un "voto" pari alla propria classe di appartenenza. Il nuovo oggetto viene assegnato alla classe che ha ottenuto il maggior numero di voti tra i k vicini selezionati.

10.2.2 La scelta del parametro k

La determinazione del valore k è cruciale per le prestazioni del sistema e rappresenta un classico problema di compromesso (*bias-variance tradeoff*):

- Se k è **troppo piccolo** (es. $k = 1$), la classificazione risulta estremamente sensibile al rumore locale e agli *outlier*. Il sistema rischia di classificare in modo errato un oggetto solo perché, casualmente, il punto a lui più vicino è un dato anomalo o etichettato male.
- Se k è **troppo grande**, il modello diventa troppo generico e i confini decisionali sfumano. Il rischio principale è che il vicinato includa troppi oggetti appartenenti a classi diverse, "annacquando" l'informazione locale e portando a decisioni errate.

In generale, per problemi di classificazione binaria, è buona prassi scegliere un valore di k **dispari** per evitare situazioni di stallo o pareggio (*tie*) durante la votazione.

Il problema delle classi sbilanciate Una criticità significativa del k -NN emerge quando il dataset presenta classi con cardinalità molto diverse (classi sbilanciate). Se si sceglie un k elevato in presenza di una classe minoritaria, il sistema tenderà a favorire sempre la classe di maggioranza. Ad esempio, supponiamo che la classe corretta per la nostra query sia la più piccola e contenga solo 12 elementi totali, ma noi impostiamo $k = 30$. Anche se l'algoritmo è perfettamente accurato e recupera tutti i 12 oggetti corretti tra i vicini, gli altri 18 elementi necessari per raggiungere il valore di k apparterranno inevitabilmente ad altre classi. Al momento del voto, questi 18 oggetti sovrasteranno i 12 corretti, causando una misclassificazione certa. Per mitigare questo rischio, una regola empirica suggerisce di fissare k come una frazione (o una percentuale) della cardinalità della **classe più piccola** presente nel dataset.

10.2.3 Variante: Weighted k-NN (Voto pesato)

Per migliorare la robustezza dell'algoritmo contro il problema delle classi sbilanciate e attenuare gli effetti di un k possibilmente sovradimensionato, si utilizza spesso una variante nota come **Weighted k-NN**. In questo approccio, i voti dei vicini non hanno tutti lo stesso peso (voto unitario), ma l'influenza di ciascun vicino è modulata in base alla sua vicinanza alla query. Tipicamente, il peso w_i assegnato al vicino i -esimo è inversamente proporzionale alla sua distanza:

$$w_i = \frac{1}{\text{dist}(x_{\text{new}}, x_i)}$$

Grazie a questa correzione:

- I vicini **estremamente prossimi** all'oggetto da classificare hanno un impatto fortissimo sulla decisione finale.
- I vicini **più lontani** (che magari rientrano nei k solo perché si trovano in una regione densa appartenente a una classe di maggioranza) avranno un peso marginale, evitando di inquinare il risultato della votazione.

10.3 Decision tree (Alberi di decisione)

Gli alberi di decisione sono modelli di classificazione di tipo gerarchico. Il processo di apprendimento consiste nel costruire una struttura ad albero partendo da un singolo nodo (la radice) che racchiude l'intero *training set*, per poi suddividere iterativamente i dati scendendo lungo i rami.

L'obiettivo del partizionamento è duplice. Da un lato si cerca di frammentare i dati per ottenere gruppi sempre più specifici, ma dall'altro si devono rispettare due caratteristiche fondamentali:

1. **Omogeneità (o Purezza):** Gli elementi all'interno di ciascun nodo terminale (foglia) dovrebbero, idealmente, appartenere tutti alla stessa classe.
2. **Omogeneità supportata (Generalizzazione):** Ogni nodo deve contenere un numero statisticamente rilevante di elementi. Se la suddivisione viene spinta all'estremo creando nodi con un singolo oggetto, si incorre nel problema dell'**overfitting** (sovradattamento): il modello impara "a memoria" il training set, diventando troppo specifico e perdendo la capacità di generalizzare (ovvero di classificare correttamente nuovi dati mai visti prima).

La costruzione dell'albero procede tipicamente con un approccio *top-down* di tipo *greedy* (ingordo):

1. **Inizializzazione:** Si associano tutte le osservazioni a un unico nodo di partenza (la radice).
2. **Iterazione di split:** Finché esistono nodi da suddividere, si ripete la seguente procedura:
 - (a) Si seleziona un nodo foglia **eterogeneo** n (cioè impuro, contenente elementi misti di classi diverse).
 - (b) Si valuta un criterio matematico per scegliere la feature f che meglio discrimina le osservazioni presenti nel nodo n .
 - (c) Si suddivide il nodo n : per ogni valore (o intervallo di valori) assunto dalla feature f , si crea un nuovo nodo figlio e vi si smistano le osservazioni corrispondenti.
3. **Terminazione:** Il processo si arresta quando tutti i nodi sono diventati omogenei (puri) o quando è stato raggiunto un criterio di arresto prefissato (es. numero minimo di elementi per nodo o profondità massima dell'albero), restituendo la classificazione finale.

10.3.1 Selezione delle feature

Il cuore dell'algoritmo risiede nel passo 2b: come decidere qual è la feature f "migliore" per effettuare lo split? Si utilizzano delle metriche matematiche per valutare il potere discriminante di ogni attributo.

Entropia L'entropia è una misura mutuata dalla teoria dell'informazione che quantifica l'incertezza, o l'**impurità**, di un nodo. Se un nodo contiene oggetti equamente divisi tra tutte le classi, l'entropia è massima; se contiene oggetti di una sola classe, l'entropia è zero (purezza totale). Durante lo split, l'obiettivo è **minimizzare l'entropia** dei nodi figli rispetto a quella del nodo padre. Questa riduzione di incertezza viene chiamata *Information Gain* (Guadagno di Informazione): la feature migliore è quella che garantisce il guadagno di informazione più alto, muovendo il sistema verso la massima prevedibilità.

Fisher's discriminant ratio (FSR) Quando le feature sono valori numerici continui e si sta affrontando un problema binario, si può utilizzare il rapporto di Fisher per valutare quanto una feature riesca a separare due classi. Il calcolo si basa sulle medie (μ) e sulle varianze (σ^2) dei valori della feature f_i per le due classi (classe 1 e classe 2):

$$FSR(f_i) = \frac{(\mu_{i,1} - \mu_{i,2})^2}{\sigma_{i,1}^2 + \sigma_{i,2}^2}$$

Questa formula esprime un rapporto molto intuitivo:

- Il **numeratore** rappresenta la distanza al quadrato tra i baricentri (medie) delle due classi. Vogliamo che sia grande, in modo che i due gruppi siano ben distanziati.
- Il **denominatore** rappresenta la dispersione (varianza) interna alle singole classi. Vogliamo che sia bassa, in modo che gli elementi di ciascuna classe siano compatti e raggruppati vicino alla propria media.

La feature f_i migliore è quella che **massimizza** l'FSR, garantendo cluster coesi internamente ma ben separati globalmente.

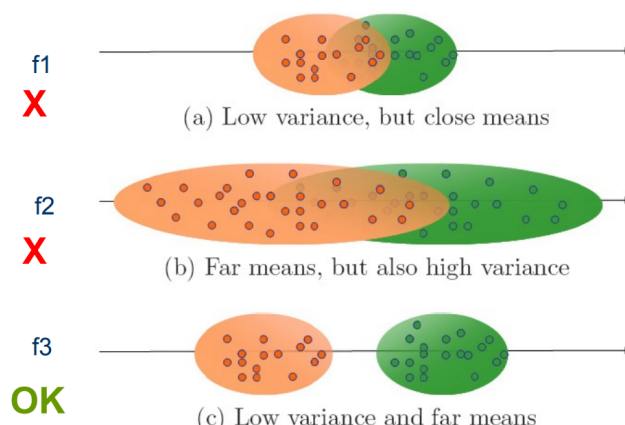


Figura 93: L'FSR premia le feature che presentano una grande distanza tra le medie delle classi (alta separazione) e una bassa varianza (alta compattezza).

10.3.2 Overfitting e Potatura (Pruning)

Continuare la suddivisione dei nodi fino a ottenere foglie perfettamente omogenee (purezza totale) porta quasi inevitabilmente all'**overfitting** (sovradattamento). L'albero diventa troppo profondo e complesso, finendo per "imparare a memoria" anche il rumore statistico e le anomalie presenti nel training set, perdendo così la sua capacità di generalizzare su nuovi dati.

Per mitigare questo problema, si adottano due strategie principali di potatura (*pruning*):

- **Pre-pruning (Potatura preventiva):** Si riduce la richiesta di purezza assoluta, arrestando la crescita dell'albero prima del tempo imponendo dei vincoli (ad esempio, definendo una profondità massima dell'albero o richiedendo un numero minimo di campioni per poter effettuare uno split).
- **Post-pruning (Potatura a posteriori):** Si lascia crescere l'albero fino alla massima espansione e, successivamente, tramite una fase di validazione (*validation set*), si eliminano a ritroso i rami periferici che non contribuiscono a migliorare le capacità di generalizzazione del modello, sostituendoli con foglie unificate.

10.4 Classificazione Ensemble

Spesso un singolo modello di classificazione, come un singolo albero di decisione, può risultare "debole" (bassa accuratezza, instabilità o alta varianza). Le tecniche *Ensemble* si basano sull'idea di addestrare molti modelli semplici (*weak learners*) e combinare le loro previsioni per ottenere un unico meta-classificatore molto più robusto e performante.

Affinché un approccio ensemble sia efficace, è fondamentale che i singoli classificatori siano **indipendenti e diversi tra loro**. In questo modo, gli errori commessi da un singolo modello verranno compensati dalle scelte corrette degli altri durante la votazione finale. Per garantire questa diversità, si introduce una componente di **casualità** (*randomness*) nel processo di addestramento, utilizzando le seguenti strategie:

Bagging (Bootstrap Aggregating) Questa tecnica agisce sui dati di addestramento. Si creano molteplici classificatori indipendenti addestrandoli su sottoinsiemi diversi del training set originale. Questi sottoinsiemi vengono generati tramite campionamento casuale **con reinserimento** (bootstrap). Di conseguenza, all'interno di un singolo sottoinsieme alcuni oggetti appariranno ripetuti, mentre altri verranno ignorati, garantendo una forte variazione strutturale nei modelli risultanti.

Random split selection (Selezione casuale dello split) Questa tecnica agisce sulla topologia del modello al momento della creazione dei nodi (è il principio cardine delle *Random Forest*). Invece di valutare tutte le feature disponibili per trovare lo split assoluto migliore in un nodo, l'algoritmo estrae un **sottoinsieme casuale** di n feature e sceglie la migliore solo all'interno di questo gruppo ristretto. Questo forza l'albero a esplorare percorsi decisionali alternativi.

Random subspace (Metodo dei sottospazi casuali) A differenza della tecnica precedente, che seleziona feature casuali a ogni singolo nodo, questa tecnica agisce a livello globale per ogni classificatore. Si costruiscono diversi modelli, ciascuno addestrato su un **sottoinsieme casuale fisso** dell'intero spazio delle feature. La selezione casuale ammette la sovrapposizione (*overlapping*), ma garantisce che i vari classificatori osservino il dataset da prospettive dimensionali profondamente differenti.

10.5 Classificazione Bayesiana

La classificazione Bayesiana è una tecnica probabilistica che si fonda sul **Teorema di Bayes**. Dato un oggetto O_i (rappresentato come un vettore di feature), l'obiettivo è stimare la probabilità a posteriori che esso appartenga a una determinata classe C_j .

Il Teorema di Bayes definisce questa probabilità come:

$$p(C_j | O_i) = \frac{p(O_i | C_j)p(C_j)}{p(O_i)}$$

Dove:

- $p(C_j | O_i)$ è la **probabilità a posteriori** (la nostra incognita).
- $p(O_i | C_j)$ è la **verosimiglianza** (likelihood), ovvero la probabilità di osservare l'oggetto O_i sapendo che appartiene alla classe C_j .
- $p(C_j)$ è la **probabilità a priori** della classe C_j .
- $p(O_i)$ è il fattore di normalizzazione (l'evidenza globale). Essendo costante per tutte le classi, viene tipicamente ignorato durante la fase di predizione, poiché ci si limita a massimizzare il numeratore.

L'oggetto O_i viene trattato come l'insieme congiunto delle sue caratteristiche empiriche (feature). Calcolare direttamente l'esatta distribuzione congiunta $p(O_i | C_j)$ su molte dimensioni è computazionalmente intrattabile; per questo si adottano delle strategie di approssimazione.

10.5.1 Naive Bayes (Ipotesi di Indipendenza)

L'approccio più semplice è il classificatore **Naive Bayes**. Esso introduce un'assunzione "ingenua": postula che tutte le feature siano **condizionatamente indipendenti** tra loro data la classe. Sotto questa ipotesi, la probabilità di osservare un oggetto intero si riduce al prodotto delle singole probabilità marginali delle sue feature:

$$p(O_i | C_j) = \prod_{k=1}^n p(f_{i,k} | C_j)$$

Questo riduce un problema multidimensionale complesso in un calcolo estremamente rapido e scalabile.

10.5.2 Gestione delle dipendenze e Informazione Mutua

Nella realtà, l'ipotesi di indipendenza forte è quasi sempre violata, poiché le feature sono spesso correlate (dipendenti). Se c'è dipendenza, il teorema del prodotto diretto decade:

$$p(O_i | C_j) \neq \prod_{k=1}^n p(f_{i,k} | C_j)$$

Per tenere conto di queste interazioni, è necessario misurare la "forza" del legame tra le coppie di feature. A tale scopo si utilizza una metrica basata sulla **Divergenza di Kullback-Leibler** (KL-divergence). La divergenza generica tra due distribuzioni di probabilità p_1 e p_2 indica quanta "informazione" si perde approssimando la prima con la seconda:

$$D_{KL}(p_1 \parallel p_2) = \sum_x p_1(x) \log \frac{p_1(x)}{p_2(x)}$$

Applicando questo concetto alle feature, calcoliamo l'**Informazione Mutua** D_{ij} tra le feature f_i e f_j . Si confronta la probabilità congiunta reale delle due feature con quella teorica che si avrebbe se fossero indipendenti:

$$D_{ij} = I(p(f_i \wedge f_j), p(f_i)p(f_j))$$

Se $p(f_i \wedge f_j) = p(f_i)p(f_j)$, le feature sono effettivamente indipendenti e $D_{ij} = 0$. Valori crescenti indicano una dipendenza sempre più marcata.

10.5.3 Grafo di dipendenza (Approssimazione di Chow-Liu)

Calcolata l'informazione mutua per tutte le possibili coppie di feature, il sistema modella queste interazioni costruendo un **grafo di dipendenza** (inizialmente un grafo completo). In questo grafo i nodi sono le singole feature e il peso di ciascun arco rappresenta il grado di dipendenza D_{ij} tra le due feature collegate.

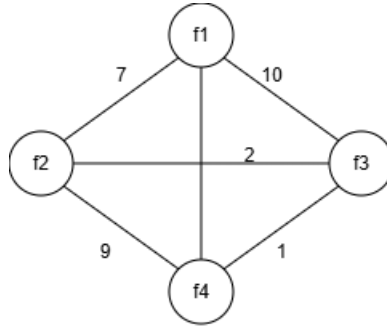


Figura 94: Grafo di dipendenza iniziale (completo)

Per evitare la complessità esponenziale di un grafo completamente connesso, si filtra la struttura calcolando l'**Albero di copertura di peso massimo** (Maximum Spanning Tree). Questo processo (noto come algoritmo di Chow-Liu) rimuove gli archi con i pesi minori, preservando solo le dipendenze più forti del sistema e rompendo eventuali cicli.

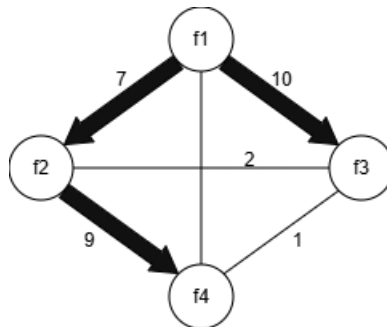


Figura 95: Albero di copertura massima (Maximum Spanning Tree)

La struttura ad albero ottenuta funge da guida per scomporre in modo trattabile la probabilità congiunta dell'oggetto. Fissando arbitrariamente un nodo come radice (es. f_1), la probabilità dell'evento congiunto è data dal prodotto della probabilità marginale della radice per le **probabilità condizionate** di ogni altro nodo rispetto al proprio nodo padre nell'albero. Facendo riferimento all'esempio in figura, l'equazione diventa:

$$p(f_1 \wedge f_2 \wedge f_3 \wedge f_4) = p(f_1)p(f_2 | f_1)p(f_3 | f_1)p(f_4 | f_2)$$

Ogni termine condizionato della formula, come ad esempio $p(f_2 \mid f_1)$, viene calcolato empiricamente dal training set (rispondendo in modo frequenzialista alla domanda: *"quanti oggetti soddisfano la feature f_2 tra tutti quelli che già soddisfano la feature f_1 ?"*). Questa metodologia rappresenta un eccellente compromesso tra la troppa semplicità del Naive Bayes e l'estrema complessità strutturale delle reti bayesiane complete.

11 Classificazione con reti neurali

L'utilizzo di semantiche latenti per la classificazione di immagini rappresenta un approccio fondamentale per estrarre caratteristiche salienti da dati ad alta dimensionalità.

Tecniche classiche di riduzione della dimensionalità come la **PCA (Principal Component Analysis)** e la **SVD (Singular Value Decomposition)** sono essenzialmente delle decomposizioni agli autovalori. Esse si caratterizzano per:

- Effettuare una **trasformazione lineare** dei dati originali.
- Individuare dei **vettori di base ortogonali** tra loro.
- Assegnare a ciascun vettore di base un peso determinato direttamente dai rispettivi **autovalori**, i quali quantificano l'importanza informativa di quella specifica componente.

In contesti avanzati ci si pone una domanda fondamentale: *se possediamo una conoscenza aggiuntiva del dominio (domain knowledge), come possiamo sfruttarla per orientare e migliorare questo processo di estrazione delle feature?* L'utilizzo del Deep Learning e delle Reti Neurali risponde a questa esigenza.

11.1 Decomposizione LR

Sia X una matrice di dimensioni $n \times m$. Dato un parametro dimensionale k (tipicamente $k < \min(n, m)$), cerchiamo due matrici L (di dimensioni $n \times k$) e R (di dimensioni $k \times m$) tali che si verifichi la fattorizzazione:

$$X = LR$$

Poiché una tale decomposizione esatta potrebbe non esistere, ci si accontenta di trovare un'**approssimazione**, formulata come $X = LR + \epsilon$, dove ϵ rappresenta una matrice di errore sufficientemente piccola. Se poniamo $\tilde{X} = LR$ come la nostra ricostruzione dei dati, l'obiettivo matematico diventa la minimizzazione della funzione di costo (Loss):

$$Loss = \Delta(X, \tilde{X}) = \epsilon$$

Per calcolare queste matrici si fa ricorso a un processo di ottimizzazione iterativa noto come **Alternating Least Squares (ALS)**, un algoritmo analogo a quelli utilizzati per la fattorizzazione dei tensori:

1. **Inizializzazione:** Si parte assegnando valori casuali alle due matrici iniziali L_0 e R_0 .
2. **Fase Iterativa:** Per ogni iterazione $i > 0$, e fino al raggiungimento di un criterio di terminazione, si eseguono alternatamente due ottimizzazioni parziali:
 - (a) **Ottimizzazione di L:** Fissata la matrice R_{i-1} , si calcola L_i che minimizza la distanza $\Delta(X, X'_i)$, dove $X'_i = L_i R_{i-1}$.
 - (b) **Ottimizzazione di R:** Fissata la matrice L_i , si risolve per R_i minimizzando la distanza $\Delta(X, X''_i)$, dove $X''_i = L_i R_i$.

11.1.1 Architettura Encoder/Decoder

Le matrici L e R trovate corrispondono concettualmente a un'architettura di tipo **Encoder/Decoder**, dove L funge da *encoder* (codificatore) e R da *decoder* (decodificatore).

Se moltiplichiamo a destra l'equazione $X = LR$ per la matrice inversa R^{-1} , otteniamo:

$$XR^{-1} = L$$

Questa equazione rappresenta la fase di **codifica**: i dati originali nello spazio X (High Dimensional Space) vengono proiettati in un nuovo spazio latente a dimensionalità ridotta k (Low Dimensional Latent Space).

La **decodifica**, per tornare alla matrice $\tilde{X}$ ricostruita, si ottiene moltiplicando le feature latenti L per il decoder R :

$$LR = \tilde{X}$$

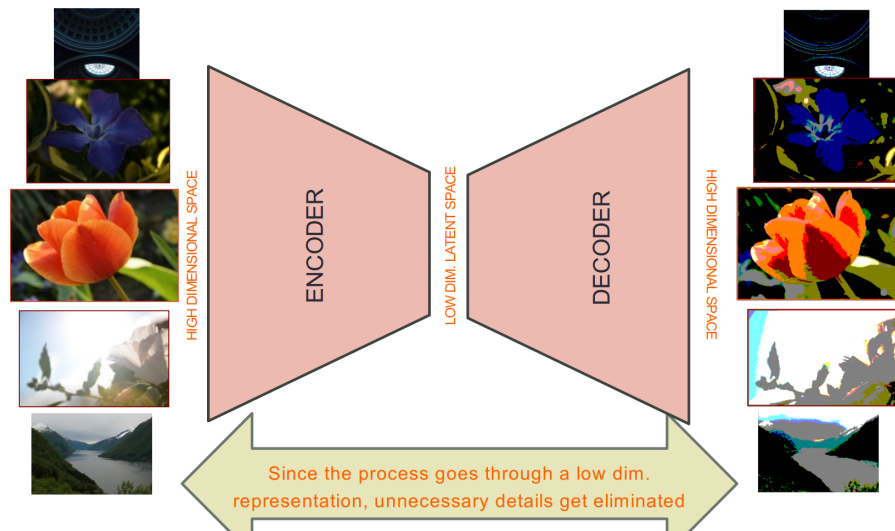


Figura 96: Architettura Encoder/Decoder: compressione nello spazio latente e successiva ricostruzione

Durante la decodifica si verifica una perdita di informazioni (ϵ). Idealmente, questa perdita rappresenta il rumore dei dati originali, permettendo al modello di isolare solo i pattern più salienti.

Regolarizzazione Per integrare la conoscenza del dominio ed evitare l'overfitting, si utilizzano vincoli e tecniche di regolarizzazione:

- **Non-negative Matrix Factorization (NMF):** Impone vincoli di non-negatività ($L_{ij} \geq 0$ e $R_{jk} \geq 0$), favorendo componenti additive interpretabili.
- **Regolarizzazione L1:** Aggiunge termini di penalità per forzare la sparsità (*sparseness*) dei pesi. La funzione di Loss diventa:

$$Loss = \Delta(X, \tilde{X}) + \lambda(\|L\|_1 + \|R\|_1)$$

11.2 Deep Auto-Encoder

Anziché passare direttamente allo spazio latente con un'unica trasformazione, un approccio **deep** scompone l'operazione in strati gerarchici. L'approssimazione di X avviene tramite una cascata di matrici $R_1, R_2, \dots, R_l$.

La codifica processa l'input attraverso i vari strati:

$$X R_1^{-1} R_2^{-1} \dots R_l^{-1} = L'$$

E la ricostruzione ripercorre gli strati in senso inverso (utilizzando architetture come le *Stacked DeConvolutional Layers* per ricreare lo spazio ad alta dimensionalità):

$$L' R_l \dots R_2 R_1 = \tilde{X}$$

Lavorare con reti profonde (Deep) è vantaggioso perché strati individuali producono matrici dei pesi più sparse e, soprattutto, permettono l'applicazione di **trasformazioni non lineari** tra uno strato e l'altro, garantendo l'apprendimento di comportamenti complessi.

11.3 Estrazione di Feature nelle Immagini

Per classificare un'immagine, la rete deve trasformarla dallo spazio grezzo dei pixel (spazio dei colori) a uno *spazio dei kernel* tramite **descrittori di feature** appresi (Learned Descriptors). In un task di classificazione, l'obiettivo matematico nella ricerca di questi descrittori è duplice:

1. **Massimizzare la distanza inter-classe:** Separare il più possibile i cluster di immagini appartenenti a categorie diverse.
2. **Minimizzare la distanza intra-classe:** Rendere compatti e vicini tra loro i vettori delle immagini appartenenti alla stessa categoria.

11.3.1 Operazione di Convoluzione e Livelli Stacked

Nelle Convolutional Neural Networks (CNN), l'operazione base è la **convoluzione**. Si scansiona l'immagine con una finestra scorrevole estraendo porzioni locali (patch) e confrontandole con un set di filtri o **kernel**.

Non tutti i descrittori locali hanno la stessa importanza discriminativa; la rete apprende i pesi dei kernel in modo da descrivere l'input esclusivamente in termini delle *feature* più rilevanti (es. bordi, texture). Per estrarre pattern complessi, le architetture moderne impilano molteplici livelli (**Stacked Convolutional Layers**).

Funzioni di Attivazione Non Lineari Subito dopo la convoluzione, si applica una funzione di attivazione non lineare per scartare le risposte negative e introdurre non-linearità.

- **ReLU (Rectified Linear Unit):** La più comune. Azzerà i valori negativi.

$$ReLU(x) = \max(0, x)$$

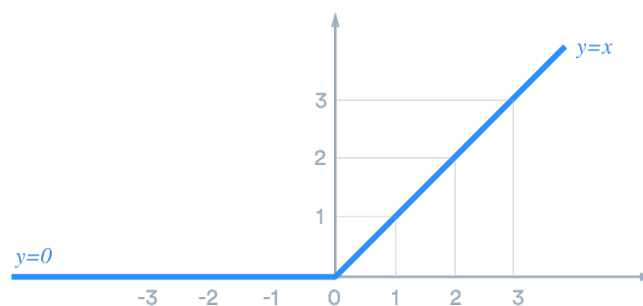


Figura 97: Funzione di attivazione ReLU

Il problema della ReLU è che non è differenziabile nello zero, complicando l'ottimizzatore in alcuni scenari.

- **SoftPlus:** Una formulazione alternativa e completamente derivabile della ReLU.

$$SoftPlus(x) = \log(1 + e^x)$$

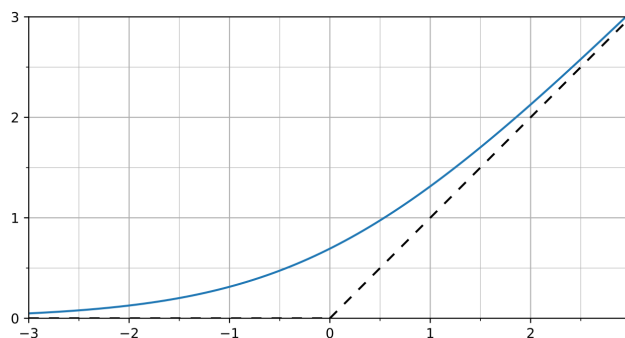


Figura 98: Funzione di attivazione SoftPlus

- **Logistic Sigmoid e Hyperbolic Tangent (HT):**

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad \text{e} \quad HT(x) = \frac{e^{2x} - 1}{e^{2x} + 1}$$

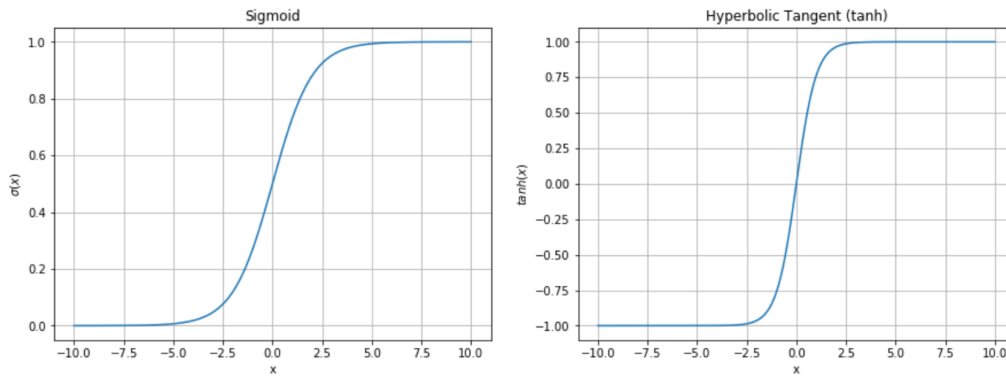


Figura 99: Funzione di attivazione Sigmoid e Hyperbolic Tangent

11.3.2 Operazione di Pooling (Sottocampionamento)

Intervallata ai livelli di convoluzione e attivazione, si applica l'operazione di **MaxPool** per ridurre drasticamente la quantità di dati e la risoluzione spaziale delle feature map. Il vantaggio fondamentale del pooling ripetuto è che la dimensione del kernel rimane costante, ma operando su una mappa ristretta, lo stesso filtro cattura **feature proporzionalmente sempre più grandi** e astratte rispetto all'immagine originale in input.

11.4 Classificazione Finale

Una volta che l'immagine ha attraversato i livelli convoluzionali e di pooling, i descrittori latenti devono essere tradotti in una predizione. Questo avviene tramite due layer finali:

Dense Linear Layer (Fully Connected Layer) I tensori multidimensionali vengono "appiattiti" (Flattening) e passati a uno strato denso. Si tratta di una trasformazione lineare rappresentata da una matrice dei pesi che mappa le feature estratte in un vettore di dimensione C , dove C è il **numero di classi** del problema.

SoftMax Layer I valori grezzi prodotti dal Dense Layer (es. 0.8, 1.1, 0.6) presentano un problema: non sono probabilità e non sommano a 1. Per poter interpretare l'output, si applica la funzione **SoftMax**, che trasforma il vettore in una vera e propria distribuzione di probabilità (es. 0.32, 0.43, 0.25), indicando la confidenza della rete per ciascuna classe.

11.5 Architetture Notevoli e Framework

Sfruttando i mattoni fondamentali appena descritti (Conv → ReLU → MaxPool → Dense → SoftMax), sono nate architetture profonde standardizzate divenute riferimento nello stato dell'arte. Tra le più famose:

- **VGG16:** Caratterizzata da una sequenza profonda di "Stacked Convolution + ReLU" usando kernel piccoli, seguita da strati densi finali.
- **ResNet50:** Introduce concetti avanzati come le **Skip Connections** (connessioni residue che bypassano alcuni strati per evitare la dissipazione del gradiente) e raggruppa le operazioni in blocchi convoluzionali complessi, includendo Zero Padding e trasformazioni di normalizzazione.

12 Relevance Feedback

Il Relevance Feedback (retroazione di rilevanza) è un meccanismo interattivo e iterativo che permette di delegare al sistema l'apprendimento delle caratteristiche ottimali di una query, sfruttando le valutazioni fornite direttamente dall'utente.

A differenza dei sistemi statici, in cui l'utente deve raffinare manualmente i termini di ricerca procedendo per tentativi, in questo scenario l'utente si limita a esaminare un primo insieme di documenti restituiti dal sistema, classificandoli (esplicitamente o implicitamente) come **rilevanti** (interessanti) o **non rilevanti** (non interessanti). Il sistema analizza quindi questo *training set* temporaneo per individuare pattern ricorrenti: estrae le *feature* maggiormente presenti nei documenti approvati e, contemporaneamente, penalizza quelle prevalenti nei documenti scartati. Sulla base di questa analisi, l'algoritmo riformula autonomamente la query originale (tramite espansione o riponderazione) per convergere verso uno spazio di ricerca più affine al reale intento informativo dell'utente.

12.1 Soggettività vs Oggettività

Il problema fondamentale dell'Information Retrieval è il divario semantico tra la percezione dell'utente e la metrica della macchina. Definiamo una query Q e l'evento R come l'effettiva rilevanza di un documento per uno specifico utente. In termini probabilistici, il sistema dovrebbe idealmente ordinare i documenti in base a $p(R | O_i)$, ovvero la probabilità a posteriori che l'oggetto O_i sia giudicato rilevante.

Un documento O_i dovrebbe precedere un documento O_j nei risultati se e solo se:

$$p(R | O_i) > p(R | O_j) \Leftrightarrow \text{sim}(Q, O_i) > \text{sim}(Q, O_j)$$

Tuttavia, le probabilità di rilevanza sono intrinsecamente **soggettive**: variano da utente a utente per la stessa query e non sono direttamente misurabili dal sistema. Il sistema possiede solo un modello **oggettivo** di similarità geometrica o vettoriale, $\text{sim}(Q, O_i)$. Il Relevance Feedback agisce esattamente come un ponte matematico: utilizza il feedback per manipolare la query Q e la funzione $\text{sim}()$ in modo che la classifica oggettiva approssimi il più possibile la distribuzione probabilistica soggettiva dell'utente.

12.2 Query e Manipolazione dello Spazio di Ricerca

Esistono diverse strategie geometriche per adattare la query in base al feedback. Immaginando di partire da una query di range:

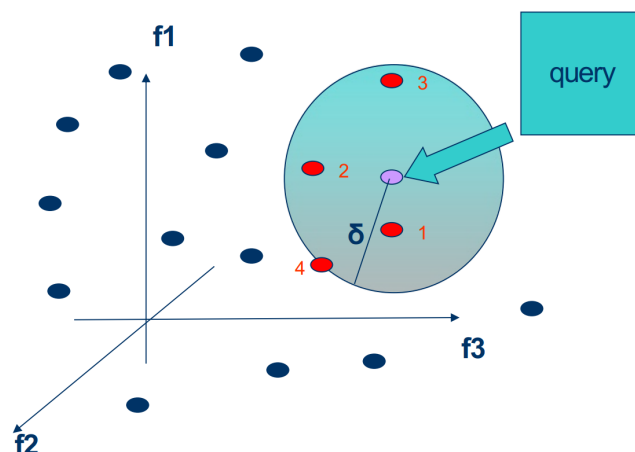


Figura 100: Query di range iniziale

Sottoponendo i risultati all'utente, il sistema può adottare varie tattiche di correzione:

1. Spostamento del baricentro (Query Point Movement): La tecnica più intuitiva consiste nel trascinare fisicamente il vettore query nello spazio vettoriale, avvicinandolo alla regione ad alta densità di documenti rilevanti e allontanandolo da quelli irrilevanti.

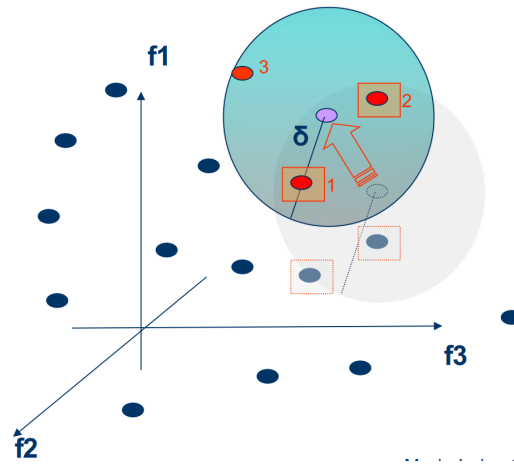


Figura 101: Query di range con centro spostato verso i documenti più rilevanti

2. Variazione del raggio di tolleranza: Si modifica la soglia di inclusione per restringere il focus (aumentando la precisione) o allargarlo (aumentando la recall) a seconda di quanti risultati rilevanti sono stati individuati nelle prossimità.

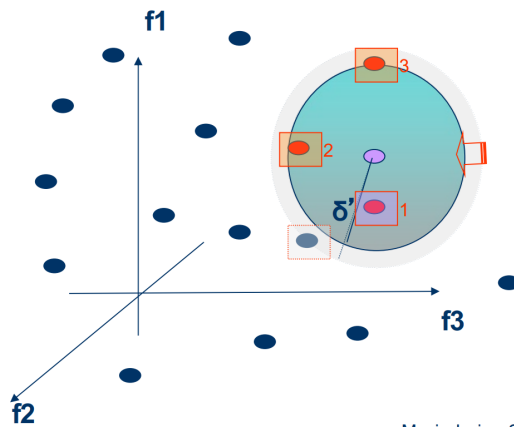


Figura 102: Query di range modificando il raggio

3. Ricalibrazione dei pesi (Feature Reweighting): Modificando l'importanza (peso) delle singole feature, lo spazio di ricerca subisce una distorsione geometrica. Feature molto discriminanti "stirano" lo spazio, rendendo la query più selettiva lungo determinati assi.

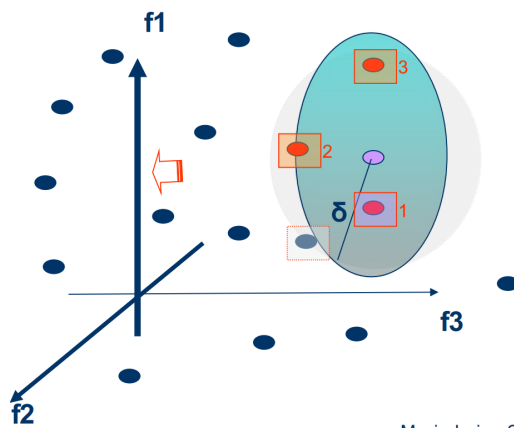


Figura 103: Query di range modificando l'importanza delle feature

4. Espansione e Riduzione dimensionale (Feature Selection): Feature influenti o fuorvianti possono essere azzerate (peso portato a zero), eliminando intere dimensioni. Al contempo, nuovi termini estratti dai documenti rilevanti possono essere aggiunti alla query.

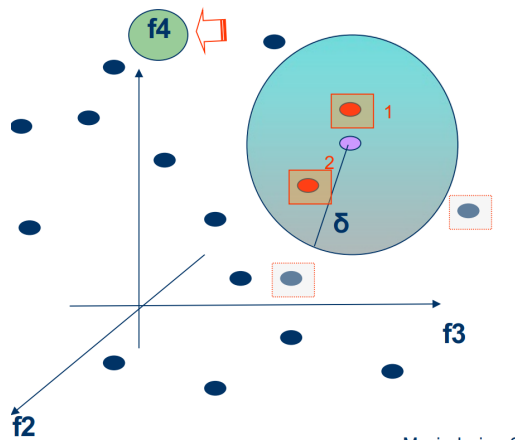


Figura 104: Query modificata aggiungendo e rimuovendo feature

5. Modifica della metrica di distanza: Se la distribuzione dei documenti rilevanti suggerisce una topologia particolare, il sistema può cambiare la formula matematica utilizzata per calcolare le distanze (es. passando da Euclidea a Mahalanobis).

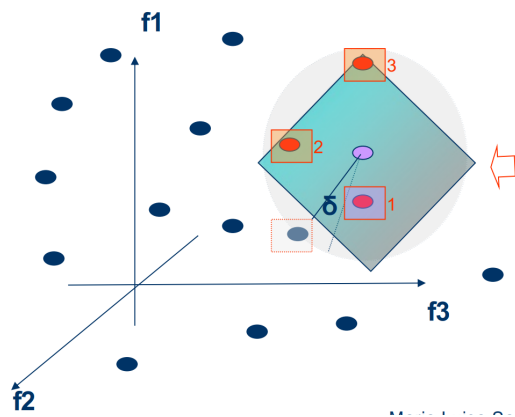


Figura 105: Query di range modificando la metrica di distanza

6. Riclusterizzazione spaziale: Nei casi più complessi, lo spazio può essere riorganizzato identificando nuovi cluster di rilevanza locali.

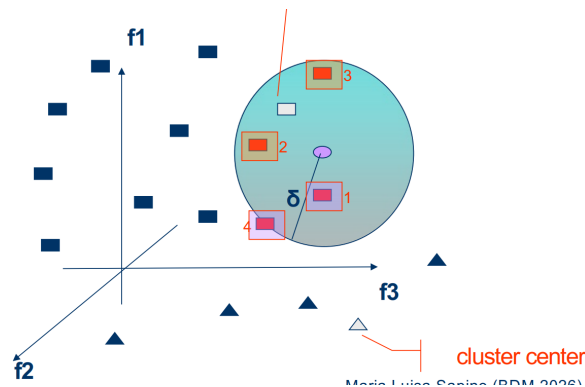


Figura 106: Riorganizzazione dello spazio tramite clustering

12.3 Spostamento della Query (Algoritmo di Rocchio)

L'approccio matematico più celebre per lo spostamento della query (Query Point Movement) si basa sulla massimizzazione della **separazione** tra classi. Vogliamo trovare un vettore query tale per cui la somma delle distanze dai documenti irrilevanti (Ir) sia massimizzata, e contemporaneamente la somma delle distanze dai documenti rilevanti (Rel) sia minimizzata:

$$sep = \left(\sum_{O_i \in Ir} dist(Q, O_i) \right) - \left(\sum_{O_i \in Rel} dist(Q, O_i) \right)$$

Sfruttando la linearità della funzione di distanza vettoriale, possiamo condensare le sommatorie calcolando i centroidi (i baricentri) dei due gruppi di documenti:

$$sep = dist \left(Q, \sum_{O_i \in Ir} O_i - \sum_{O_i \in Rel} O_i \right)$$

L'algoritmo formula la nuova query corretta Q' applicando una traslazione vettoriale alla query originale Q . Si aggiunge una frazione del baricentro dei documenti rilevanti e si sottrae una frazione del baricentro di quelli irrilevanti:

$$Q' = Q + \left(c_{rel} \times \sum_{O_i \in Rel} O_i \right) - \left(c_{ir} \times \sum_{O_i \in Ir} O_i \right)$$

Dove c_{rel} e c_{ir} sono i tassi di apprendimento (pesi) che regolano l'aggressività dello spostamento verso il target positivo o l'allontanamento dal target negativo.

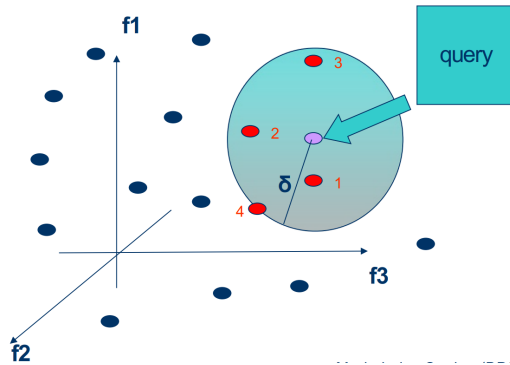


Figura 107: Query di range iniziale

Geometricamente, il processo parte dal calcolo della risultante dei vettori associati agli oggetti rilevanti e irrilevanti.

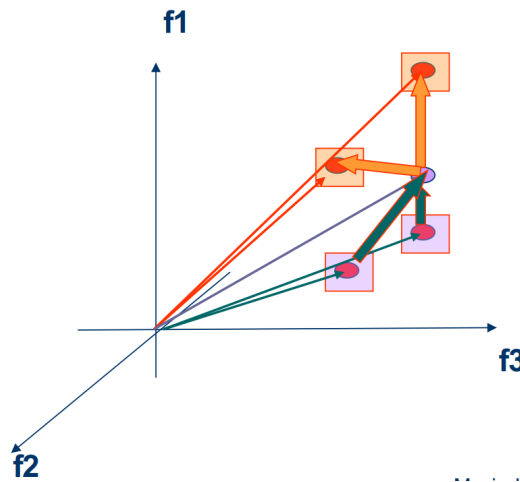


Figura 108: Calcolo vettoriale delle somme spaziali

Concatenando i vettori risultanti pesati, otteniamo il vettore direzione di correzione totale.

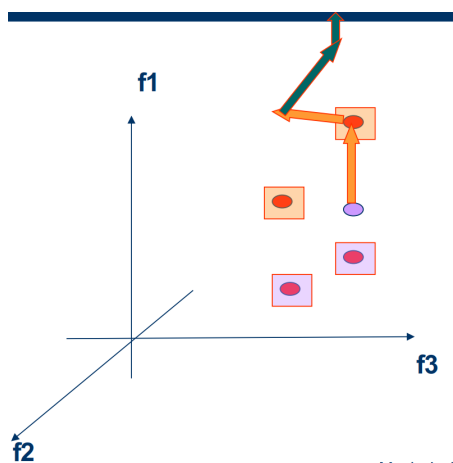


Figura 109: Concatenazione dei vettori differenza per ottenere il vettore di traslazione

Questa direzione è l'esatto percorso spaziale lungo il quale far scivolare la query originale per ottimizzare il recupero futuro.

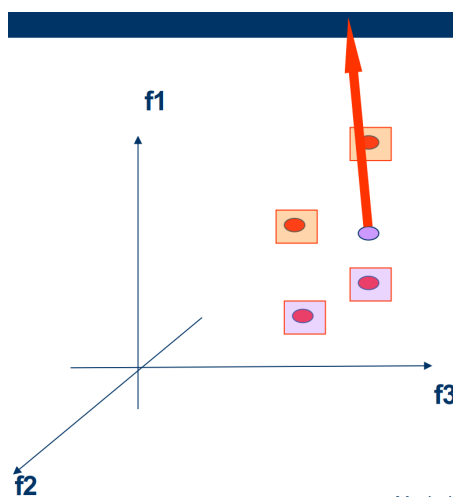


Figura 110: Applicazione della direzione di spostamento alla query Q

Applicando il vettore di traslazione, la query Q' raggiunge il suo nuovo posizionamento nello spazio semantico.

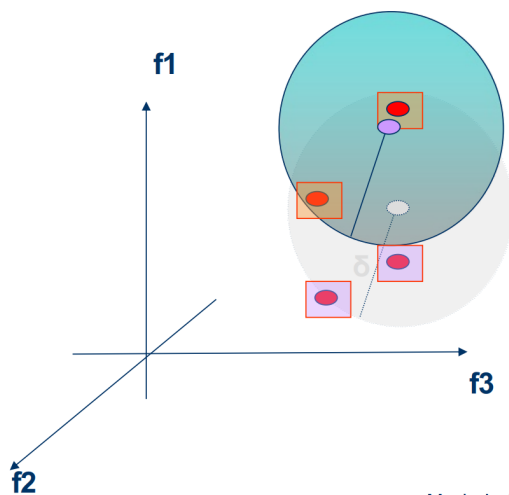


Figura 111: Nuova query Q' centrata nel cluster ottimale

12.4 Ricalibrazione dei Pesi delle Feature

Un'altra tecnica potente non sposta la query, ma ne deforma i confini alterando l'importanza delle singole feature in base alle prove statistiche ottenute dal feedback. Per ciascuna feature dello spazio vettoriale, viene calcolato un fattore di **significatività** (*sig*), basato sul rapporto tra le sue probabilità di occorrenza:

$$sig(f_k) = \frac{\log\left(\frac{p(f_k|R)}{1-p(f_k|R)}\right)}{\log\left(\frac{p(f_k|I)}{1-p(f_k|I)}\right)}$$

Il numeratore valuta l'*odds* (il rapporto di probabilità a favore) della feature negli oggetti rilevanti (*R*), mentre il denominatore ne valuta l'impatto in quelli irrilevanti (*I*). Se la feature appare sistematicamente nei documenti buoni e raramente in quelli scartati, il valore *sig* sarà estremamente elevato. Questo valore diventerà il nuovo peso w_k assegnato a quella specifica feature.

12.4.1 Come stimare le probabilità marginali

A priori, il sistema è cieco: non possiede le distribuzioni di *R* e *I*. Deve quindi ricavare delle stime empiriche basandosi esclusivamente sull'interazione in corso. La stima di $p(f_k | R)$ varia a seconda del ruolo originale della feature:

Caso A: Se f_k NON era un termine della query iniziale In questo scenario, la feature f_k non ha influenzato il motore di ricerca nella selezione iniziale dei documenti. Di conseguenza, possiamo considerare l'insieme dei documenti restituiti (Retrieved) e successivamente valutati, come un **campione statistico privo di bias** rispetto a f_k . Possiamo approssimare la probabilità globale calcolando la frequenza relativa all'interno del subset analizzato:

$$p(f_k | R) \approx p(f_k | \text{Retrieved \& Relevant})$$

Ad esempio, se sui 50 documenti rilevanti visionati dall'utente, 15 contengono la feature non cercata f_k , assegneremo $p(f_k | R) = 15/50 = 0.3$.

Caso B: Se f_k ERA un termine della query iniziale In questo caso, l'assunzione precedente crolla. Il dataset restituito è fortemente sbilanciato (*biased*), perché il sistema ha forzatamente selezionato i documenti *proprio in virtù* della presenza di f_k .

Per calcolare il reale impatto isolato di f_k , dobbiamo ricorrere a un artificio logico-matematico. Assumiamo che la similarità sia calcolata come una combinazione lineare (dot product) per superare una certa soglia di cut-off *T*:

$$sim(o, q) = \sum_{i=1}^n w_i f_i > T$$

Isoliamo il contributo specifico della feature f_j in esame rispetto al resto della query (che definiamo $sim_{(-j)}$):

$$sim(o, q) = w_j f_j + sim_{(-j)}(o, q) > T$$

Classifichiamo ora gli oggetti rilevanti e recuperati all'interno della seguente matrice di contingenza:

	$f_j = 1$ (Presente)	$f_j = 0$ (Assente)
$sim_{(-j)}(o, q) \leq T$	<i>a</i>	0
$sim(o, q) > T$	<i>b</i>	<i>c</i>

Tabella 1: Matrice di classificazione rispetto al cut-off *T* scorporato dalla feature *j*

- **Cella *a*:** Oggetti che, senza la feature *j*, sarebbero stati scartati. Sono stati recuperati **esclusivamente** grazie alla presenza determinante di *j*.

- **Cella 0:** Situazione impossibile (l'oggetto non ha la feature e non supera il threshold col resto della query, ergo non è tra gli oggetti recuperati e valutati).
- **Cella b:** Oggetti in cui j è presente, ma la cui assenza sarebbe stata ininfluenza, poiché il resto della query garantiva già il superamento della soglia T .
- **Cella c:** Oggetti in cui j è assente, ma recuperati grazie alla forza del resto della query.

Per neutralizzare il bias indotto dalla query originaria, limitiamo l'osservazione solo ai casi in cui il termine f_k è risultato **ininfluente** ai fini del superamento della soglia. Isoliamo cioè la probabilità condizionata alla seconda riga della tabella:

$$p(f_k \mid \text{Ret \& Rel}) = p(f_k = 1 \mid \text{sim}_{(-k)}(o, q) > T)$$

L'insieme universo diventa l'unione dei casi b e c . I casi in cui f_k è presente sono b . La probabilità epurata dal bias diventa quindi un semplice rapporto tra cardinalità:

$$p(f_k \mid R) \approx \frac{b}{b + c}$$

12.5 Ranking Bayesiano dei Documenti (Dimostrazione dei passaggi)

L'obiettivo ultimo di un sistema IR con relevance feedback è ordinare i documenti in modo decrescente rispetto alla loro reale probabilità di essere pertinenti. Come affermato inizialmente, il ranking ideale soddisfa la condizione:

$$p(R \mid O_i) > p(R \mid O_j)$$

Poiché questa probabilità a posteriori è complessa da stimare direttamente, utilizziamo manipolazioni algebriche e probabilistiche per ricondurla a variabili calcolabili empiricamente (le feature dei documenti). Il processo si snoda in quattro passaggi fondamentali:

Passaggio 1: Applicazione del Teorema di Bayes Scomponiamo la probabilità a posteriori usando la formula di Bayes, indicando con I l'evento complementare $\neg R$ (documento irrilevante):

$$p(R \mid O_i) = \frac{p(O_i \mid R)p(R)}{p(O_i \mid R)p(R) + p(O_i \mid I)p(I)}$$

L'equazione di ordinamento diventa quindi la comparazione tra le due forme bayesiane esplose per O_i e O_j . Poiché $p(R)$ e $p(I)$ sono probabilità a priori globali (costanti su tutti i documenti), il numeratore è influenzato unicamente dalla **verosimiglianza** (likelihood) $p(O_i \mid R)$.

Passaggio 2: Passaggio al rapporto delle probabilità (Odds Ratio) Trattare la forma frazionaria completa è computazionalmente inefficace e introduce instabilità. Fortunatamente, i sistemi IR non necessitano di conoscere il valore percentuale esatto di probabilità, ma solo il **rango** relativo (chi è maggiore di chi). Possiamo trasformare l'equazione in una metrica monotona crescente calcolando l'**Odds** di rilevanza, ovvero dividendo la probabilità che l'oggetto sia rilevante per la probabilità che sia irrilevante. Il ranking viene perfettamente preservato:

$$\frac{p(O_i \mid R)}{p(O_i \mid I)} > \frac{p(O_j \mid R)}{p(O_j \mid I)} \Leftrightarrow \text{sim}(Q, O_i) > \text{sim}(Q, O_j)$$

Questa proporzione ci indica esplicitamente di quante volte l'oggetto è più associabile alla classe "Rilevante" rispetto a quella "Irrilevante".

Passaggio 3: Ipotesi "Naive" di Indipendenza Condizionale L'oggetto O_i è un vettore complesso di centinaia o migliaia di feature (parole o attributi). Calcolare la distribuzione congiunta multivariata $p(O_i | R)$ è matematicamente impossibile a causa della scarsità dei dati (curse of dimensionality). Per rendere il calcolo computabile in tempo reale, introduciamo l'ipotesi del classificatore *Naive Bayes*: postuliamo che tutte le feature $f_{i,k}$ siano statisticamente indipendenti tra loro, data la classe di rilevanza. La probabilità di osservare l'intero documento si disintegra in una semplice moltiplicazione (produttoria) delle probabilità di osservare le sue singole feature separate:

$$p(O_i | R) \approx \prod_{k=1}^n p(f_{i,k} | R) \quad p(O_i | I) \approx \prod_{k=1}^n p(f_{i,k} | I)$$

Sostituendo queste produttorie nella disuguaglianza precedente, il problema di classificazione multiclasse collassa nel confronto tra fattori marginali che abbiamo precedentemente imparato a stimare empiricamente (vedi sezione 4.1):

$$\prod_{k=1}^n \frac{p(f_{i,k} | R)}{p(f_{i,k} | I)} > \prod_{k=1}^n \frac{p(f_{j,k} | R)}{p(f_{j,k} | I)}$$

Passaggio 4: Trasformazione Logaritmica per la linearizzazione Operare con moltiplicazioni di probabilità condizionate produce numeri infinitesimalmente piccoli (frequente causa di *underflow* computazionale nei calcolatori). Inoltre, i motori di ricerca utilizzano tradizionalmente funzioni di similarità additive (come il prodotto scalare). Poiché la funzione logaritmo naturale è strettamente monotona (se $A > B$, allora $\log(A) > \log(B)$), possiamo applicarla a entrambi i lati dell'equazione. Il logaritmo trasforma elegantemente le produttorie in sommatorie, offrendo una stabilità numerica assoluta e riconducendo l'ordinamento a un classico calcolo vettoriale lineare:

$$\sum_{k=1}^n \log \left(\frac{p(f_{i,k} | R)}{p(f_{i,k} | I)} \right) > \sum_{k=1}^n \log \left(\frac{p(f_{j,k} | R)}{p(f_{j,k} | I)} \right)$$

In questa forma finale, l'argomento del logaritmo all'interno della sommatoria rappresenta esattamente il "peso ideale" per la feature k -esima, permettendo al sistema di assegnare un punteggio assoluto (score) a ogni documento nel database mediante una semplice somma ponderata, allineando finalmente i calcoli oggettivi alle percezioni soggettive dell'utente.